

Verifying λ_{GT} Type System

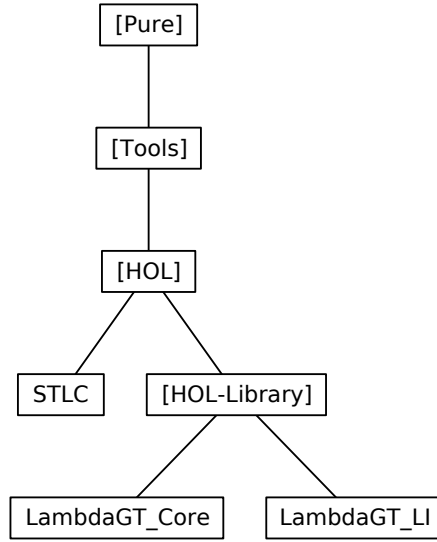
Jin Sano

June 15, 2026

Contents

1	Simply Typed Lambda-Calculus	1
1.1	Basic definitions of Untyped Call By Value Lambda-calculus	1
1.1.1	Lambda-terms in de Bruijn notation and substitution	1
1.1.2	Call-By-Value Reduction	2
1.1.3	Some examples	2
1.2	Type System	3
1.2.1	Environments	3
1.2.2	Types and Typing Rules	5
1.2.3	Some examples	5
1.3	Theorems	6
1.3.1	Substitution Lemma	6
1.3.2	Subject Reduction	7
1.3.3	Progress	7
1.3.4	Soundness	8
1.3.5	Closed	8
2	Introduction	9
3	Syntax, Semantics and Type System of the Language	10
3.1	Syntax of the Language	10
3.2	Link Substitution	11
3.3	Congruence Relation	11
3.4	Variable Lift and Graph Substitution	12
3.5	Value	13
3.6	Type System	14
3.7	Reduction Relations	16
4	Congruence Relations	17
5	Properties of Link Substitution	17
5.1	Basic Properties of Link Substitution	17
5.2	Basic Properties of Simultaneous Link Substitution	17
5.3	Link Substitution Before/After Inverse Link Substitution	18
5.4	Basic Property of Link Substitution Over Types	21
5.5	Link Substitution Over Link Substitution	23
6	Properties of Lift and Graph Substitution	25
6.1	Basic Properties of Lift	25
6.2	Basic Properties of Graph Substitution	26
7	Theorems of Congruence	28
7.1	Free Links Equivalence of Congruent Graphs	28
7.2	Multiset Invariant Over Congruent Graphs	30
7.3	Lemmata with Lift Graph	31
7.4	Lemmata with Subst Graph	32
7.5	Lemmata of Link substitutions	34
7.6	Graph Substitution Over Congruence Graphs and Expressions	37
8	Properties of Environments	39

9	Soundness	40
9.1	Substitution Lemma	40
9.2	Inversion Lemma	47
9.3	Subject Reduction	49
9.4	Canonical	52
9.5	Progress	53
9.6	Soundness	55
10	Introduction	55
11	Syntax, Semantics and Type System of the Language	56
11.1	Syntax of the Language	56
11.2	Link Substitution	56
11.3	Congruence Relation	57
11.4	Variable Lift and Graph Substitution	58
11.5	Value	59
11.6	Type System	59
11.7	Reduction Relations	64
12	Congruence Relations	65
13	Properties of Link Substitution	66
13.1	Basic Properties of Link Substitution	66
13.2	Basic Properties of Simultaneous Link Substitution	66
13.3	Link Substitution Before/After Inverse Link Substitution	67
13.4	Basic Property of Link Substitution Over Types	73
13.5	Link Substitution Over Link Substitution	75
14	Properties of Lift and Graph Substitution	77
14.1	Basic Properties of Lift	77
14.2	Basic Properties of Graph Substitution	78
15	Theorems of Congruence	81
15.1	Free Links Equivalence of Congruent Graphs	81
15.2	Multiset Invariant Over Congruent Graphs	83
15.3	Lemmata with Lift Graph	84
15.4	Lemmata with Subst Graph	86
15.5	Lemmata of Link substitutions	87
15.6	Graph Substitution Over Congruence Graphs and Expressions	93
16	Properties of Environments	94
17	Soundness	105
17.1	Substitution Lemma	105
17.2	Inversion Lemma	113
17.3	Subject Reduction	115
17.4	Canonical	118
17.5	Progress	120
17.6	Soundness	123



1 Simply Typed Lambda-Calculus

1.1 Basic definitions of Untyped Call By Value Lambda-calculus

```

theory STLC
imports Main
begin

```

```

declare [[syntax-ambiguity-warning = false]]

```

1.1.1 Lambda-terms in de Bruijn notation and substitution

We first define a link substitution and the operation to apply the substitutions over a link, a list of links, a link substitution and show some properties.

De Bruijn-style lambda-terms. - `Var n` : a variable represented by a natural number (de Bruijn index). - `App e1 e2` : application (written $e_1 \cdot e_2$). - `Abs t e` : abstraction $(\lambda t.e)$, which binds one variable. - `Val n s` : a constructor for base-type values of sort `s`.

```

datatype 'ty exp =
  Var nat
| App 'ty exp 'ty exp (infixl <·> 200)
| Abs 'ty 'ty exp
| Val nat nat

```

The lifting operation "`lift t k`" shifts free variables in `t` that are $\geq k$ by 1. This is used when we move under a binder (to avoid variable capture).

```

primrec
  lift :: ['ty exp, nat] => 'ty exp
where
  lift (Var i) k = (if i < k then Var i else Var (i + 1))

| lift (e1 · e2) k = lift e1 k · lift e2 k
| lift (Abs τ e) k = Abs τ (lift e (k + 1))

| lift (Val v s) k = Val v s

```

Capture-avoiding substitution "`e'[e/i]`": replace variable `i` in `e'` with `e`. - Variables above `k` are decremented (indices slide down). - At a binder (`Abs`), we lift `s` and substitute at `k+1` to account for the new binder.

```

primrec
  subst :: ['ty exp, 'ty exp, nat] => 'ty exp
  subst (Var i) (e, k) = (if i < k then Var i else e'[e/i])
where
  subst-Var: (Var j)[e/i] =

```

$(\text{if } i < j \text{ then Var } (j - 1) \text{ else if } j = i \text{ then } e \text{ else Var } j)$
 $| \text{subst-App: } (e1 \cdot e2)[e/i] = e1[e/i] \cdot e2[e/i]$
 $| \text{subst-Abs: } (\text{Abs } \tau \ e1)[e/i] = \text{Abs } \tau \ (e1[\text{lift } e \ 0 \ / \ i+1])$
 $| \text{subst-Val: } (\text{Val } v \ s)[e/i] = \text{Val } v \ s$

declare *subst-Var* [*simp del*]

declare *if-not-P* [*simp*] *not-less-eq* [*simp*]
 — don't add *r-into-rtranc1*[*intro!*]

1.1.2 Call-By-Value Reduction

Values are exactly abstractions and base-type literals.

inductive *is-val* :: 'ty exp $\Rightarrow$ bool **where**
 $v\text{-abs } [intro!]: \text{is-val } (\text{Abs } \tau \ e)$
 $v\text{-nat } [intro!]: \text{is-val } (\text{Val } v \ s)$

Call-by-value small-step relation. - *beta_v*: Perform beta only when the argument is already a value. - *appL*: Evaluate the function part first (left-to-right). - *appR*: Then evaluate the argument. No reduction happens inside an abstraction.

inductive *cbv* :: 'ty exp $\Rightarrow$ 'ty exp $\Rightarrow$ bool (**infixl** $\rightarrow_v$ 50) **where**
 $\text{beta-v } [simp, intro]:$
 $\text{is-val } v \implies$
 $\text{Abs } \tau \ e \cdot v \rightarrow_v e[v/0]$
 $| \text{appL } [simp, intro]:$
 $e1 \rightarrow_v e1' \implies$
 $e1 \cdot e2 \rightarrow_v e1' \cdot e2$
 $| \text{appR } [simp, intro]:$
 $\text{is-val } v \implies$
 $e \rightarrow_v e' \implies$
 $v \cdot e \rightarrow_v v \cdot e'$

Inversion rules for the step relation.

inductive-cases *cbv-cases* [*elim!*]:
 $\text{Var } i \rightarrow_v e$
 $\text{Abs } \tau \ e1 \rightarrow_v e$
 $e1 \cdot e2 \rightarrow_v e$
 $\text{Val } v \ s \rightarrow_v e$

Zero or more reductions.

abbreviation *cbv-star* ::
 $'ty \text{ exp } \Rightarrow 'ty \text{ exp } \Rightarrow \text{bool}$
 $(\text{infixl } \rightarrow_v^* \ 50) \text{ where}$
 $\text{cbv-star} \equiv \text{rtranc1 } \text{cbv}$

1.1.3 Some examples

We first define a link substitution and the operation to apply the substitutions over a link, a list of links, a link substitution and show some properties.

lemma *is-val* ($\text{Abs } \tau \ 1 \ (\text{Var } 0)$)
by (*simp add: v-abs*)

lemma ($\text{Abs } \tau \ 1 \ (\text{Var } 0)) \cdot \text{Val } 0 \ 0 \rightarrow_v \text{Val } 0 \ 0$
by (*metis beta-v is-val.simps less-irrefl subst-Var*)

lemma ($\text{Abs } \tau \ 1 \ (\text{Var } 0 \cdot \text{Val } 1 \ 2)) \cdot (\text{Abs } \tau \ 2 \ (\text{Var } 0))$
 $\rightarrow_v^* \text{Val } 1 \ 2$

proof —
have *P2*: ($\text{Var } 0 \cdot \text{Val } 1 \ 2$)[$(\text{Abs } \tau \ 2 \ (\text{Var } 0)) / 0$]
 $= ((\text{Abs } \tau \ 2 \ (\text{Var } 0)) \cdot \text{Val } 1 \ 2)$

```

by (simp add: subst-Var)

have P3: (Abs  $\tau 1$  (Var 0 · Val 1 2)) · (Abs  $\tau 2$  (Var 0))
   $\rightarrow_v$  ((Abs  $\tau 2$  (Var 0)) · Val 1 2)
by (metis P2 beta-v v-abs)

have P4: ((Abs  $\tau 2$  (Var 0)) · Val 1 2)  $\rightarrow_v$  Val 1 2
by (metis beta-v is-val.simps less-irrefl subst-Var)

show ?thesis
using P3 P4 by force
qed

```

1.2 Type System

We first define environments and then define typing relations.

1.2.1 Environments

Environments are represented as lists, where the head corresponds to the most recently inserted binding (de Bruijn-style contexts).

type-synonym ('a) env = 'a list

insert-at xs j x inserts element x at position j in xs.

```

fun insert-at :: 'a env  $\Rightarrow$  nat  $\Rightarrow$  'a  $\Rightarrow$  'a env where
  insert-at xs 0 x = x # xs |
  insert-at [] (Suc n) x = [x] |
  insert-at (y # ys) (Suc n) x = y # insert-at ys n x

```

Inserting at any position increases the environment length by exactly one.)

lemma length-insert-at:

length (insert-at xs j y) = length xs + 1

proof (induction xs arbitrary: j)

```

case Nil
then show ?case
apply (cases j)
apply auto
done

```

next

```

case (Cons a xs)
then show ?case
apply (cases j)
apply simp
by simp

```

qed

Relations between the before and the after inserted environments.

lemma shift-eq:

```

assumes j  $\leq$  length xs
shows (insert-at xs j y) ! j = y

```

using assms

proof (induction xs arbitrary: j)

```

case Nil
then show ?case
apply (cases j)
apply auto
done

```

next

```

case (Cons y ys)
then show ?case
apply (cases j)
apply auto
done

```

qed

```

lemma shift-gt:
  assumes  $i \leq \text{length } xs$ 
  shows  $j < i \implies (\text{insert-at } xs \ i \ y) ! j = xs ! j$ 
using assms
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  apply (cases i)
  apply simp
  by simp
next
  case (Cons a xs)
  then show ?case
  apply (cases i)
  apply auto
  using less-Suc-eq-0-disj by fastforce
qed

```

```

lemma shift-lt:
   $i < j \implies$ 
   $(\text{insert-at } xs \ i \ y) ! j = xs ! (j - 1)$ 
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  apply (cases i)
  apply simp
  by simp
next
  case (Cons a xs)
  then show ?case
  apply (cases i)
  apply auto
  done
qed

```

Inserting y at the front and then U at position $\text{Suc } i$ is the same as first inserting U at position i and then y at the front.

```

lemma shift-commute:
   $\text{insert-at } (\text{insert-at } xs \ i \ y1) \ 0 \ y2$ 
   $= \text{insert-at } (\text{insert-at } xs \ 0 \ y2) \ (\text{Suc } i) \ y1$ 
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  by simp
next
  case (Cons a xs)
  then show ?case
  by simp
qed

```

1.2.2 Types and Typing Rules

Simple types: - Base n : a base type identified by a natural number n . - Fun $\tau \ U$: function type from τ to U (written $\tau \Rightarrow U$).

```

datatype type =
  Base nat
  | Fun type type    (infixr  $\langle \Rightarrow \rangle$  200)

```

Typing environments map de Bruijn indices to types by list lookup. The head corresponds to the most recently bound variable.

```

type-synonym tyenv = type list

```

Typing Rules: - Var : if i is in range and has type τ under environment Γ , then variable i has type τ . - Abs : if t has type τ_2 under Γ extended with τ , then $\text{Abs } \tau' t$ has type $\tau \rightarrow \tau_2$. - App : if s has type $\tau \rightarrow \tau_2$ and t has type τ , then $s \cdot t$ has type τ_2 . - Val : numeric literals are assigned a fixed base type (here Base 1).

inductive typing ::

```

  tyenv  $\Rightarrow$  type exp  $\Rightarrow$  type  $\Rightarrow$  bool
  ( $\langle \vdash - : - \rangle$  [50, 50, 50] 50)
  where
    Var[intro!]:
       $i < \text{length } \Gamma \implies$ 
       $\Gamma ! i = \tau \implies$ 
       $\Gamma \vdash \text{Var } i : \tau$ 
  | Abs[intro!]:
       $\tau 1 \# \Gamma \vdash e : \tau 2 \implies$ 
       $\Gamma \vdash \text{Abs } \tau 1 e : \tau 1 \Rightarrow \tau 2$ 
  | App[intro!]:
       $\Gamma \vdash e 1 : \tau 2 \Rightarrow \tau 1 \implies$ 
       $\Gamma \vdash e 2 : \tau 2 \implies$ 
       $\Gamma \vdash e 1 \cdot e 2 : \tau 1$ 
  | Val[intro!]:
       $\Gamma \vdash (\text{Val } v s) : (\text{Base } s)$ 

```

Elimination (inversion) rules for the typing judgment. They allow case analysis on a typing derivation of a specific syntactic form.

inductive-cases typing-elim [elim!]:

```

   $\Gamma \vdash \text{Var } i : \tau$ 
   $\Gamma \vdash e 1 \cdot e 2 : \tau$ 
   $\Gamma \vdash \text{Abs } \tau 1 e : \tau$ 
   $\Gamma \vdash \text{Val } v s : \tau$ 

```

1.2.3 Some examples

schematic-goal $\Gamma \vdash \text{Abs } \tau 1 (\text{Var } 0) : ?\tau$
by force

schematic-goal $\Gamma \vdash \text{Abs } \tau 1 (\text{Abs } \tau 2 (\text{Var } 1)) : ?\tau$
by force

schematic-goal $\Gamma \vdash \text{Val } v s : ?\tau$
by force

schematic-goal $\square \vdash (\text{Abs } ?\tau 2 (\text{Var } 0)) \cdot \text{Val } v s : ?\tau$
by force

schematic-goal $\square \vdash \text{Abs } ?\tau 1 (\text{Abs } ?\tau 2 (\text{Var } 1 \cdot \text{Var } 0)) : ?\tau$
by force

schematic-goal $\Gamma \vdash$
 $\text{Abs } ?\tau 1 (\text{Abs } ?\tau 2 (\text{Abs } ?\tau 3 (\text{Var } 1 \cdot (\text{Var } 2 \cdot \text{Val } 1 \ 3 \cdot \text{Var } 0)))) : ?\tau$
by force

schematic-goal $\Gamma \vdash$
 $\text{Abs } ?\tau 1 (\text{Abs } ?\tau 2 (\text{Abs } ?\tau 3 (\text{Var } 2 \cdot \text{Var } 0 \cdot (\text{Var } 1 \cdot \text{Var } 0)))) : ?\tau$
by force

1.3 Theorems

1.3.1 Substitution Lemma

We first show that lifting preserves well-typedness. The lifting operation shifts free variable indices in an expression. This lemma shows that lifting preserves typing: if e is well-typed in environment Γ , then lifting e by i yields a well-typed term in the environment extended by inserting τ' at position i . This property is crucial for the substitution lemma, particularly under the abstraction case.

```

lemma lift-type:
  assumes  $\Gamma \vdash e : \tau$ 
    and  $i \leq \text{length } \Gamma$ 
  shows  $\text{insert-at } \Gamma \ i \ \tau' \vdash \text{lift } e \ i : \tau$ 
using assms
proof (induct arbitrary: i  $\tau'$  set: typing)
  case (Var j  $\Gamma \ \tau$ )
  then show ?case
  apply auto
  apply (simp add: length-insert-at)
  apply (simp add: shift-gt)
  apply (simp add: length-insert-at)
  apply (simp add: shift-lt)
  done
next
  case (Abs  $\tau 2' \ \tau \ \Gamma \ e \ \tau 2$ )
  then show ?case
  apply auto
  apply force
  done
next
  case (App  $\Gamma \ e1 \ \tau 2 \ \tau 1 \ e2$ )
  then show ?case
  apply auto
  done
next
  case (Val  $\Gamma \ s$ )
  then show ?case
  using typing.Val by auto
qed

```

Substitution lemma: If e_1 is well-typed under environment Γ_1 , and e_2 is well-typed under Γ_2 , and $\Gamma_1 = \text{insert_at } \Gamma_2 \ i \ \tau_2$, then substituting e_2 for variable i in e_1 yields a well-typed term under Γ_2 with the same type as e_1 . This formalises that well-typedness is preserved by substitution.

```

lemma subst-lemma:
  fixes  $i :: \text{nat}$ 
    and  $e1 \ e2 :: \text{type exp}$ 
    and  $\tau 1 \ \tau 2 :: \text{type}$ 
    and  $\Gamma 1 \ \Gamma 2 :: \text{tyenv}$ 
  shows  $\Gamma 1 \vdash e1 : \tau 1 \implies$ 
     $\Gamma 2 \vdash e2 : \tau 2 \implies$ 
     $i \leq \text{length } \Gamma 2 \implies$ 
     $\Gamma 1 = \text{insert-at } \Gamma 2 \ i \ \tau 2 \implies$ 
     $\Gamma 2 \vdash e1[e2/i] : \tau 1$ 
proof (induct arbitrary:  $\Gamma 2 \ i \ \tau 2 \ e2$  set: typing)
  case (Abs  $\tau 1' \ \Gamma' \ e' \ \tau 2'$ )
  have LIFT:  $\tau 1' \# \Gamma 2 \vdash \text{lift } e2 \ 0 : \tau 2$ 
    using Abs.premis(1) lift-type by fastforce
  have LE:  $\text{Suc } i \leq \text{length } (\tau 1' \# \Gamma 2)$  using Abs.premis(2) by simp
  have ENV:  $\tau 1' \# \Gamma' = \text{insert-at } (\tau 1' \# \Gamma 2) \ (\text{Suc } i) \ \tau 2$ 
    using Abs.premis(3) by (simp add: shift-commute)
  have IH:  $\tau 1' \# \Gamma 2 \vdash e'[\text{lift } e2 \ 0 / \text{Suc } i] : \tau 2'$ 
    using Abs.hyps(2) ENV LE LIFT by blast
  show ?case
    by (simp add: IH typing.Abs)
next
  case (Var j  $\Gamma \ \tau$ )
  then show ?case
  apply (simp add: subst-Var)
  apply auto
  apply (simp add: shift-eq)
  apply (simp add: length-insert-at)
  apply (simp add: shift-lt)
  by (simp add: shift-gt)
next
  case (App  $\Gamma \ s \ \tau \ \tau' \ t$ )
  then show ?case

```

```

  by auto
next
case (Val  $\Gamma$  s)
  then show ?case
  using typing.Val by fastforce
qed

```

1.3.2 Subject Reduction

Subject reduction: If a term e reduces to e' by a call-by-value step, and e is well-typed, then e' is also well-typed with the same type. This ensures that evaluation preserves typing.

```

lemma subject-reduction:
  assumes  $\Gamma \vdash e : \tau$ 
    and  $e \rightarrow_v e'$ 
  shows  $\Gamma \vdash e' : \tau$ 
using assms
proof (induct arbitrary: e' set: typing)
  case (Var  $i$   $\Gamma$   $\tau$ )
  then show ?case
  by auto
next
  case (Abs  $\tau' \tau \Gamma$   $e \tau 2$ )
  then show ?case
  by auto
next
  case (App  $\Gamma$  s  $\tau \tau 2$  t)
  then show ?case
  apply auto
  by (simp add: subst-lemma)
next
  case (Val  $\Gamma$  v)
  then show ?case
  using cbv.cases by blast
qed

```

Multi-step subject reduction: If e reduces to e' in zero or more steps, then e' has the same type as e .

```

theorem subject-reduction':
  assumes  $e \rightarrow_v^* e'$ 
    and  $\Gamma \vdash e : \tau$ 
  shows  $\Gamma \vdash e' : \tau$ 
using assms
  by (induct set: rtranclp) (iprover intro: subject-reduction)+

```

1.3.3 Progress

Canonical forms for function types under CBV: if a closed value has a function type, it must be an Abs.

```

lemma canonical-fun:
  assumes  $V$ : is-val v
    and  $T$ :  $\Box \vdash v : \tau 1 \Rightarrow \tau 2$ 
  shows  $\exists e. v = \text{Abs } \tau 1 e$ 
using  $V$   $T$ 
proof cases
  case (v-abs t)
  then show ?thesis
  using  $T$  by blast
next
  case (v-nat n e)
  hence  $\Box \vdash \text{Val } n e : \tau 1 \Rightarrow \tau 2$  using  $T$  by simp
  then have  $\tau \Rightarrow \tau 2 = \text{Base } 1$ 
  apply cases
  done
  thus ?thesis by simp
qed

```

Progress: closed, well-typed terms are values or can step.

```

lemma progress:

```

```

 $\Gamma \vdash e : \tau \implies$ 
 $\Gamma = [] \implies$ 
 $is\text{-}val\ e \vee (\exists e'. e \rightarrow_v e')$ 
proof (induction arbitrary:  $\tau$  set: typing)
  case ( $Var\ i\ \Gamma'\ \tau$ )
  then show  $?case$ 
  by simp
next
  case ( $Abs\ \tau1\ \Gamma\ t\ \tau2$ )
  then show  $?case$ 
  by auto
next
  case ( $Val\ \Gamma'\ s$ )
  then show  $?case$ 
  by auto
next
  case ( $App\ \Gamma'\ t1\ \tau2\ \tau1\ t2$ )
  have  $H1: \Gamma' \vdash t1 : \tau2 \Rightarrow \tau1$ 
  by (simp add: App.hyps(1))
  have  $H2: \Gamma' = []$ 
  by (simp add: App.premis)
  have  $P1: is\text{-}val\ t1 \vee (\exists t1'. t1 \rightarrow_v t1')$ 
  by (simp add: App.IH(1) H2)
  have  $P2: is\text{-}val\ t1 \implies$ 
     $(\exists t'. (t1 \cdot t2) \rightarrow_v t')$ 
  using App.IH(2) H1 H2 canonical-fun by blast
  show  $?case$ 
  using  $P1\ P2$  by blast
qed

```

Progress: a simpler corollary.

```

lemma progress':
  assumes  $[] \vdash e : \tau$ 
  shows  $is\text{-}val\ e \vee (\exists e'. e \rightarrow_v e')$ 
using assms progress by auto

```

1.3.4 Soundness

Type soundness theorem: If a closed term is well-typed and reduces (in zero or more steps) to some term e' , then e' is either a value or can take a further step. This combines subject reduction and progress to establish the standard type soundness property of the language.

```

theorem type-soundness:
  assumes  $WT: [] \vdash e : \tau$ 
  and  $STEPS: e \rightarrow_v^* e'$ 
  shows  $is\text{-}val\ e' \vee (\exists u. e' \rightarrow_v u)$ 
proof -
  have  $[] \vdash e' : \tau$  using STEPS WT subject-reduction' by blast
  thus  $?thesis$  using progress' by blast
qed

```

1.3.5 Closed

Set of free variables.

```

primrec fv :: type exp => nat set where
  fv-Var:  $fv\ (Var\ i) = \{i\}$ 
  | fv-App:  $fv\ (t \cdot u) = fv\ t \cup fv\ u$ 
  | fv-Abs:  $fv\ (Abs\ \tau'\ t) = (\lambda\ i. i - 1)\ ` (fv\ t - \{0\})$ 
  | fv-Val:  $fv\ (Val\ t\ s) = \{\}$ 

```

```

value  $fv\ (Abs\ \tau'\ (Abs\ \tau'\ (Abs\ \tau'\ (Var\ 5 \cdot (Var\ 2 \cdot Val\ 1\ 8 \cdot Var\ 0))))))$ 
value  $fv\ (Abs\ \tau'\ (Abs\ \tau'\ (Abs\ \tau'\ (Var\ 2 \cdot Var\ 0 \cdot (Var\ 1 \cdot Var\ 0))))))$ 

```

```

abbreviation is-closed :: type exp => bool where

```

$is-closed\ t \equiv fv\ t = \{\}$

The length of the typing environment corresponds the indices of free variables.

```

lemma envlen-fv:
  assumes Ty:  $\Gamma \vdash t : \tau$ 
  shows  $\forall\ i \in fv\ t. i < length\ \Gamma$ 
using assms
proof (induction arbitrary:  $\tau$  set: typing)
  case (Var  $i\ \Gamma'\ \tau$ )
  then show ?case
  apply auto
  done
next
  case (Abs  $\tau1\ \Gamma\ t\ \tau2$ )
  then show ?case
  apply auto
  done
next
  case (App  $\Gamma\ s\ \tau\ \tau2\ t$ )
  then show ?case
  apply auto
  done
next
  case (Val  $\Gamma\ s$ )
  then show ?case
  apply auto
  done
qed

```

If typable, then it is closed.

```

lemma emp-env-closed:
  assumes  $\square \vdash t : \tau$ 
  shows  $fv\ t = \{\}$ 
using assms envlen-fv by fastforce

```

end

2 Introduction

This is a proof for the soundness of the type system of the λ_{GT} language.

```

theory LambdaGT-Core
imports
  Main
  HOL-Library.Multiset
begin

```

```

declare  $[[syntax-ambiguity-warning = false]]$ 
declare  $[[smt-oracle = false]]$ 

```

3 Syntax, Semantics and Type System of the Language

3.1 Syntax of the Language

```

type-synonym link = nat
type-synonym links = link list

```

We use natural numbers as link identifiers. A value of type *link* represents a single link name, and a value of type *links* represents a finite list of such names. Links are handled using de Bruijn indices: the number 0 refers to the innermost bound link, 1 to the next one, and so on.

```

datatype 'ty graph =
  Zero
| Atom 'ty p links
| Fusion link link

```

```

| Mol 'ty graph 'ty graph
| Nu 'ty graph
and 'ty exp =
  Graph 'ty graph
| App 'ty exp 'ty exp (infixl <·> 200)
| Case 'ty exp (links * 'ty) list 'ty graph 'ty exp 'ty exp
and 'ty p =
  GConstr string
| GVar nat
| GAbs links 'ty 'ty exp

```

The syntax of HyperLMNtal graphs, expressions, and atomic patterns is given by the following mutually recursive datatypes.

- *'ty graph* is the type of graphs, parameterised by a type annotation *'ty*.
 - *Zero* is the empty graph.
 - *Atom p xs* is an atomic cell with principal atom *p* and a list *xs* of incident links.
 - *Fusion x y* represents a fusion (equality) constraint between links *x* and *y*.
 - *Mol g1 g2* is the parallel composition (multiset union) of graphs *g1* and *g2*.
 - *Nu g* introduces a fresh link scope around *g*. The binder *Nu* binds link index *0* in *g*; all other free link indices in *g* are shifted accordingly.
- *'ty exp* is the type of expressions over graphs.
 - *Graph g* embeds a graph *g* as a trivial expression.
 - *App e1 e2* (written *e1 · e2*) is application of *e1* to *e2*.
 - *Case e Γ T e1 e2* is a case expression: it inspects the value of *e*, tries to match it against the graph pattern *T* using the rule environment *Γ*, and continues with *e1* on success or *e2* on failure.
- *'ty p* is the type of principal atoms.
 - *GConstr s* is a constructor with name *s*.
 - *GVar i* is a de Bruijn index *i* for an expression variable.
 - *GAbs xs ty e* is an abstraction whose body is *e*, with a list of interface links *xs* and a type annotation *ty*. The bound variable is represented by index *0* in *e*, and outer variables are shifted as usual for de Bruijn indices.

Free links of graphs. The constructor *Nu* binds the link index 0 in its subgraph, and all other free links are shifted down by one.

```

primrec FL :: 'ty graph => link set
where
  FL Zero = {}
| FL (Atom p links) = set links
| FL (Fusion x y) = {x, y}
| FL (Mol g1 g2) = FL g1 ∪ FL g2
| FL (Nu g) = {n. Suc n ∈ FL g}

```

lemma *shift-down-alt*:

```

  {n. Suc n ∈ S} = (λx. x - 1) ` (S - {0})
apply auto
by (metis Zero-not-Suc diff-Suc-1' imageI insert-Diff-single insert-iff)

```

3.2 Link Substitution

Link Substitution Over Graphs

Free link substitution. Note that this is design not to change any local links.

```

primrec
  lmap :: (link => link) => 'ty graph => 'ty graph
where

```

```

  lmap f Zero = Zero
| lmap f (Atom p xs) = Atom p (map f xs)
| lmap f (Fusion x y) = Fusion (f x) (f y)
| lmap f (Mol g1 g2) = Mol (lmap f g1) (lmap f g2)
| lmap f (Nu g) = Nu (lmap (\x. if x = 0 then x else (f (x - 1) + 1)) g)

```

lemma *succ-after-before*:

```

{ n. Suc n ∈ (λx. if x = 0 then x else (f (x - 1) + 1)) ' S } =
  f ' { n. Suc n ∈ S }

```

apply *auto*

by *force*

Simultaneous Link Substitution

Definition of Simultaneous Link Substitution.

fun *lsubst* :: (link * link) list => link => link **where**

```

  lsubst [] x = x |
  lsubst ((v, n) # s) x = (if x = v then n else lsubst s x)

```

lemma *lsubst-simp*:

```

  lsubst (vn # s) x = (if x = fst vn then snd vn else lsubst s x)

```

by (*metis lsubst.simps(2) prod.collapse*)

3.3 Congruence Relation

We first define the congruence relation over graphs and prove some properties of the relation.

Inductive definition of graph equivalence.

inductive *gcong* :: 'ty graph => 'ty graph => bool (**infix** $\simeq$ 50)

where

```

  refl [intro!]: g ≃ g
| sym [intro]: g1 ≃ g2 ==> g2 ≃ g1
| trans[trans]: g1 ≃ g2 ==> g2 ≃ g3 ==> g1 ≃ g3

| mol-zero [simp]: Mol g Zero ≃ g
| mol-comm [simp]: Mol g1 g2 ≃ Mol g2 g1
| mol-assoc[simp]: Mol (Mol a b) c ≃ Mol a (Mol b c)

| mol-cong[intro]: g1 ≃ g1' ==> g2 ≃ g2' ==> Mol g1 g2 ≃ Mol g1' g2'
| nu-cong [intro]: g ≃ g' ==> Nu g ≃ Nu g'

```

```

| nu-subst-fusion1:
  y ∈ FL g ==> Nu (Mol (Fusion 0 y) g) ≃ Nu (lmap (\x. if x = 0 then y else x) g)
| nu-subst-fusion2:
  0 ∈ FL g ==> Nu (Mol (Fusion 0 y) g) ≃ Nu (lmap (\x. if x = 0 then y else x) g)

```

```

| nu-nu-fusion1 [simp]: Nu (Nu (Fusion 0 1)) ≃ Zero
| nu-nu-fusion2 [simp]: Nu (Fusion 0 0) ≃ Zero

```

```

| nu-zero [simp]: Nu Zero ≃ Zero

```

```

| nu-comm: Nu (Nu g) ≃ Nu (Nu (lmap (\x. if x = 0 then 1 else if x = 1 then x else x) g))

```

```

| nu-scope: Mol (Nu g1) g2 ≃ Nu (Mol g1 (lmap Suc g2))

```

Some helper functions for building a graph.

definition *Mols1*:

Mols1 g gs = foldl Mol g gs

primrec *Nus1* :: *nat* => *'ty graph* => *'ty graph* **where**

Nus1 0 g = g

| *Nus1 (Suc n) g = Nu (Nus1 n g)*

3.4 Variable Lift and Graph Substitution

The lifting operation *lift t k* shifts free de Bruijn indices stored in *GVar* that are greater than by 1. When we go under an *GAbs* binder, we increase the cutoff to *k + 1*. Graph-level binders *Nu* bind links, not expression variables, so they do not affect the cutoff for lifting expression variables.

primrec *lift* :: *nat* => *'ty exp* => *'ty exp*

and *lift-atom* :: *nat* => *'ty p* => *'ty p*

and *lift-graph* :: *nat* => *'ty graph* => *'ty graph*

where

lift k (Graph g) = Graph (lift-graph k g)

| *lift k (App e1 e2) = App (lift k e1) (lift k e2)*

| *lift k (Case e Γ T e1 e2) =*
Case (lift k e) Γ T (lift (k + length Γ) e1) (lift k e2)

| *lift-atom k (GConstr s) = GConstr s*

| *lift-atom k (GVar i) =*
(if i < k then GVar i else GVar (i + 1))

| *lift-atom k (GAbs τ xs e) =*
GAbs τ xs (lift (k + 1) e)

| *lift-graph k Zero = Zero*

| *lift-graph k (Atom a vs) = Atom (lift-atom k a) vs*

| *lift-graph k (Fusion x y) = Fusion x y*

| *lift-graph k (Mol g1 g2) =*
Mol (lift-graph k g1) (lift-graph k g2)

| *lift-graph k (Nu g) =*
Nu (lift-graph k g)

Capture-avoiding substitution *e' [e / i]* on expressions, atoms, and graphs. This is a structural skeleton; the behavior on variable occurrences (*GVar*) must be decided to make this a real substitution.

primrec *subst* :: *nat* => *links* => *'ty graph* => *'ty exp* => *'ty exp*

and *subst-graph* :: *nat* => *links* => *'ty graph* => *'ty graph* => *'ty graph*

and *subst-p* :: *nat* => *links* => *'ty graph* => *'ty p* => *links* => *'ty graph*

where

subst i xs G (Graph g) = Graph (subst-graph i xs G g)

| *subst i xs G (App e1 e2) = App (subst i xs G e1) (subst i xs G e2)*

| *subst i xs G (Case e0 Γ T e1 e2) =*
Case (subst i xs G e0) Γ T
(subst (i + length Γ) xs ((lift-graph 0 $\sim$ (length Γ)) G) e1)
(subst i xs G e2)

| *subst-p i xs G (GConstr s) ys = Atom (GConstr s) ys*

| *subst-p i xs G (GAbs zs ty e0) ys =*
Atom (GAbs zs ty (subst (i + 1) xs (lift-graph 0 G) e0)) ys

| *subst-p i xs G (GVar j) ys =*
(if i < j then (Atom (GVar (j - 1)) ys)
else if j = i $\wedge$ length xs = length ys then lmap (lsubst (zip xs ys)) G
else (Atom (GVar j) ys))

| *subst-graph i xs G Zero = Zero*

| *subst-graph i xs G (Atom p ys) = subst-p i xs G p ys*

| *subst-graph i xs G (Fusion x y) = Fusion x y*

| *subst-graph i xs G (Mol g1 g2) = Mol (subst-graph i xs G g1) (subst-graph i xs G g2)*

| *subst-graph* *i xs G* (*Nu g*) = *Nu* (*subst-graph i xs G g*)

definition *substs-graph*:

substs-graph *XsGs T* = *fold* ($\lambda ((xs, -), g). \text{subst-graph } 0 \text{ } xs \text{ } g$) *XsGs T*

definition *substs*:

substs *XsGs e* = *fold* ($\lambda ((xs, -), g). \text{subst } 0 \text{ } xs \text{ } g$) *XsGs e*

declare *if-not-P* [*simp*] *not-less-eq* [*simp*]
 — don't add *r-into-rtranc1*[*intro!*]

3.5 Value

Values are exactly abstractions and base-type literals.

primrec *is-atom-val* :: '*ty p* => *bool* **where**

is-atom-val (*GConstr s*) = *True*

| *is-atom-val* (*GAbs xs ty e*) = *True*

| *is-atom-val* (*GVar i*) = *False*

Values are exactly abstractions and base-type literals.

primrec *is-graph-val* :: '*ty graph* => *bool* **where**

is-graph-val *Zero* = *True*

| *is-graph-val* (*Atom p xs*) = *is-atom-val p*

| *is-graph-val* (*Fusion x y*) = *True*

| *is-graph-val* (*Mol g1 g2*) = (*is-graph-val g1* $\wedge$ *is-graph-val g2*)

| *is-graph-val* (*Nu g*) = *is-graph-val g*

lemma *is-graph-val-ignore-lmap*:

is-graph-val (*lmap f g*) = *is-graph-val g*

apply (*induct g arbitrary: f rule: graph.induct[of - %-. True %-. True]*)

by *auto*

lemma *cong-is-graph-val*:

assumes *g1* $\simeq$ *g2*

shows *is-graph-val g1* = *is-graph-val g2*

using *assms*

apply (*induct set: gcong*)

apply (*simp-all add: is-graph-val-ignore-lmap*)

by *auto*

Values are exactly abstractions and base-type literals.

primrec *is-val* :: '*ty exp* => *bool* **where**

is-val (*Graph g*) = *is-graph-val g*

| *is-val* (*App e1 e2*) = *False*

| *is-val* (*Case e0 Γ T e1 e2*) = *False*

lemma *cong-is-val*:

assumes *g1* $\simeq$ *g2*

shows *is-val* (*Graph g1*) = *is-val* (*Graph g2*)

using *assms*

by (*simp add: cong-is-graph-val*)

3.6 Type System

We first define environments and then define typing relations.

Environments

Environments are represented as lists, where the head corresponds to the most recently inserted binding (de Bruijn-style contexts).

type-synonym (*'a*) *env* = *'a list*

insert-at xs j x inserts element *x* at position *j* in *xs*.

fun *insert-at* :: *'a env => nat => 'a => 'a env* **where**
insert-at xs 0 x = x # xs |
insert-at [] (Suc n) x = [x] |
insert-at (y # ys) (Suc n) x = y # insert-at ys n x

Types and Typing Rules

Simple types:

- *TBase n xs* : base type identified by *n* with free links *xs*
- *TArrow ty1 ty2 xs* : function type *ty1 => ty2* with free links *xs*.

datatype *type* =
TBase nat links
| *TArrow type type links*

Typing environments map de Bruijn indices to (links, type) pairs. The head is the most recently bound variable.

type-synonym *tyenv* = (*links * type*) *list*

Right-hand side of a production rule.

type-synonym *prodrule-RHS* =
*nat * (string * links) * (link * link) list * type list*

Whole production rule: left-hand side tag / links and RHS.

type-synonym *prodrule* =
(*nat * links*) * *prodrule-RHS*

primrec *FLty* :: *type => link set* **where**
FLty (TBase i xs) = set xs
| *FLty (TArrow ty1 ty2 xs) = set xs*

Free links of a RHS: we shift by *i* to account for the *Nus1 i* binder.

definition *FLrhs* :: *prodrule-RHS => link set* **where**
FLrhs rhs $\equiv$
(*case rhs of (i, (C, zs), fusions, taus) =>*
{*n. n + i* $\in$
(*set zs*
 $\cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$
 $\cup (\bigcup (FLty \text{ ' set taus}))$ }})

Free-link substitution on types.

primrec *lmap-ty* :: *links => links => type => type* **where**
lmap-ty xs ys (TBase i zs) =
TBase i (map (lsubst (zip xs ys)) zs)
| *lmap-ty xs ys (TArrow ty1 ty2 zs) =*
TArrow ty1 ty2 (map (lsubst (zip xs ys)) zs)

Turn a list of link pairs (*x, y*) into a list of fusion graphs *Fusion x y*.

definition *fusions-of* :: (*link * link*) *list* => *type graph list* **where**
fusions-of fusions = *map ($\lambda(x, y). \text{Fusion } x \text{ } y$) fusions*

definition *taus-of-rhs* :: *prodrule-RHS => type list* **where**
taus-of-rhs rhs =
(*case rhs of (i, (C, zs), fusions, taus) => taus*)

Apply a production rule to argument graphs *Ts*.

definition *app-prodrule* ::

```

type graph list => prodrule-RHS => type graph where
app-prodrule Ts rhs =
  (case rhs of (i, (C, zs), fusions, taus) =>
    Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts))

```

Typing rules.

inductive *typing* ::

```

tyenv => prodrule list => type exp => type => bool
(⟨- ⊢ {-} - : -⟩ [50, 50, 50, 50] 50)

```

where

TyVar:

```

i < length Γ ==>
Γ ! i = (xs, ty) ==>
length xs = length ys ==>
distinct xs ==>
distinct ys ==>
set xs = FLty ty ==>
Γ ⊢ {P} Graph (Atom (GVar i) ys) : lmap-ty xs ys ty

```

| *TyArrow*:

```

(xs, ty1) # Γ ⊢ {P} e : ty2 ==>
distinct xs ==>
set xs = FLty ty1 ==>
Γ ⊢ {P} Graph (Atom (GAbs xs ty1 e) ys) : TArrow ty1 ty2 ys

```

| *TyApp*:

```

Γ ⊢ {P} e1 : TArrow ty1 ty2 xs ==>
Γ ⊢ {P} e2 : ty1 ==>
Γ ⊢ {P} e1 · e2 : ty2

```

| *TyProd*:

```

((a, xs), rhs) ∈ set P ==>
taus = taus-of-rhs rhs ==>
length taus = length Ts ==>
(∀ i < length Ts. Γ ⊢ {P} (Graph (Ts ! i)) : taus ! i) ==>
set xs = FLrhs rhs ==>
Γ ⊢ {P} (Graph (app-prodrule Ts rhs)) : TBase a xs

```

| *TyCong*:

```

Γ ⊢ {P} Graph g1 : ty ==>
g1 ≃ g2 ==>
Γ ⊢ {P} Graph g2 : ty

```

| *TyAlpha*:

```

Γ2 ⊢ {P} Graph g : ty ==>
set xs = FL g ==>
length xs = length ys ==>
distinct xs ==>
distinct ys ==>
Γ2 ⊢ {P} Graph (lmap (lsubst (zip xs ys)) g)
  : lmap-ty xs ys ty

```

| *TyCase*:

```

Γ ⊢ {P} e0 : ty1 ==>
Γ2 @ Γ ⊢ {P} e1 : ty2 ==>
Γ ⊢ {P} e2 : ty2 ==>
Γ ⊢ {P} Case e0 Γ2 T e1 e2 : ty2

```

Inversion rules for the typing judgment. They let us analyse typing derivations for particular syntactic forms.

inductive-cases *typing-GVarE* [elim!]:

```

Γ ⊢ {P} Graph (Atom (GVar i) ys) : ty

```

inductive-cases *typing-GAbsE* [elim!]:

```

Γ ⊢ {P} Graph (Atom (GAbs xs ty1 e) ys) : ty

```

inductive-cases *typing-AppE* [elim!]:

```

Γ ⊢ {P} e1 · e2 : ty

```

inductive-cases *typing-GraphE* [elim!]:

```

Γ ⊢ {P} (Graph g) : ty

```

3.7 Reduction Relations

Call-by-value small-step relation on expressions. - Perform beta only when the argument graph is already a value. - Case on a value graph that matches the pattern graph. - Case on a non-graph value (falls through to the else branch). - Evaluate the function part first (left-to-right). - Then evaluate the argument, once the function is a value. No reduction happens inside abstractions.

inductive *cbv-ty* :: *type exp* ==> *prodrule list* ==> *type exp* ==> *bool*
 (- ->{-} - [60,0,61] 60)

where

beta-v-ty [*simp*, *intro*]:
 is-graph-val v ==> *distinct xs* ==> *FL v = set xs* ==>
 lam $\simeq$ *Atom (GAbs xs ty e) ys* ==>
 Graph lam · *Graph v* ->{P} *subst 0 xs v e*
 | *case1-v-ty* [*simp*, *intro*]:
 is-graph-val v ==>
 length Γ = *length gs* ==>
 XsGs = *zip* Γ *gs* ==>
 v $\simeq$ *substs-graph XsGs T* ==>
 $\forall ((xs, ty), g) \in \text{set } XsGs. ($
 distinct xs $\wedge$ *FL g = set xs* $\wedge$
 $\square \vdash \{P\} (\text{Graph } g) : ty$
 $) ==>$
 Case (Graph v) Γ T e1 e2 ->{P} substs XsGs e1
 | *case2-v-ty* [*simp*, *intro*]:
 is-graph-val v ==>
 Case (Graph v) Γ T e1 e2 ->{P} e2
 | *appL-ty* [*simp*, *intro*]:
 e1 ->{P} e1' ==>
 e1 · *e2* ->{P} *e1'* · *e2*
 | *appR-ty* [*simp*, *intro*]:
 is-val v ==>
 e ->{P} e' ==>
 v · *e* ->{P} *v* · *e'*
 | *caseL-ty* [*simp*, *intro*]:
 e0 ->{P} e0' ==>
 Case e0 Γ T e1 e2 ->{P} Case e0' Γ T e1 e2

Inversion rules for the call-by-value step relation.

inductive-cases *cbv-ty-AppE* [*elim!*]:
 e1 · *e2* ->{P} *e*

inductive-cases *cbv-ty-CaseE* [*elim!*]:
 Case e0 Γ T e1 e2 ->{P} e

inductive-cases *cbv-ty-GraphE* [*elim!*]:
 Graph g ->{P} e

Zero or more reductions.

abbreviation *cbv-ty-star* ::
 type exp ==> *prodrule list* ==> *type exp* ==> *bool*
 (- ->{-})* - [60,0,61] 60)

where

e ->{P} e' $\equiv$ rtranchp ($\lambda e e'. \text{cbv-ty } e \text{ } P \text{ } e'$) e e'*

4 Congruence Relations

Show it's an equivalence relation (useful for rewriting/quotients).

lemma *gcong-reflp*: *reflp* ($\simeq$)
by (*simp add: gcong.refl reflpI*)

lemma *gcong-symp*: *symp* ($\simeq$)
by (*simp add: gcong.sym sympI*)

lemma *gcong-transp*: *transp* ($\simeq$)
using *gcong.trans transpI* **by** *blast*

lemma *gcong-equivp*: *equivp* ($\simeq$)
by (*simp add*: *gcong-reflp gcong-symp gcong-transp equivpI*)

5 Properties of Link Substitution

5.1 Basic Properties of Link Substitution

lemma *FL-lmap-commute*:
fixes *g* :: 'ty graph
shows *FL* (*lmap* *f* *g*) = *f* ' (*FL* *g*)
proof (*induct g arbitrary*: *f*)
case (*Nu g*)
have *FL* (*lmap* *f* (*Nu g*)) = *FL* (*Nu* (*lmap* (λx . if $x = 0$ then x else (f ($x - 1$) + 1)) *g*))
by *simp*
also have ... = {*n*. *Suc* *n* $\in$ *FL* (*lmap* (λx . if $x = 0$ then x else (f ($x - 1$) + 1)) *g*)}
by *simp*
also have ... = {*n*. *Suc* *n* $\in$ (λx . if $x = 0$ then x else (f ($x - 1$) + 1)) ' *FL g*}
using *Nu* **by** *blast*
also have ... = *f* ' {*n*. *Suc* *n* $\in$ *FL g*}
using *succ-after-before* **by** *presburger*
also have ... = *f* ' *FL* (*Nu g*)
by *simp*
finally show ?*case* .
qed *auto*

lemma *FL-lmap-deSuc-Suc*:
fixes *k n* :: nat
and *g* :: 'ty graph
shows {*n*. *Suc* *n* $\in$ *FL* (*lmap* *Suc g*)} = *FL g*
apply (*simp add*: *FL-lmap-commute*)
by *auto*

5.2 Basic Properties of Simultaneous Link Substitution

lemma *lsubst-cdom*:
assumes $x \in \text{set } (\text{map } \text{fst } xs)$
shows *lsubst* *xs* *x* $\in \text{set } (\text{map } \text{snd } xs)$
using *assms*
proof (*induct xs*)
case *Nil*
then show ?*case*
apply *auto*
done
next
case (*Cons a xs*)
then show ?*case*
apply (*cases* $x = \text{fst } a$)
using *lsubst-simp*
by *auto*
qed

lemma *lsubst-image-aux*:
assumes *distinct* (*map* *fst s*)
shows *map* (*lsubst s*) (*map* *fst s*) = *map* *snd s*
using *assms*
proof (*induct s*)
case *Nil*
then show ?*case*
apply *auto*
done
next
case (*Cons a s*)
then show ?*case*
apply *auto*

```

using lsubst-simp apply presburger
using lsubst-simp apply auto
by (metis fstI image-eqI)
qed

```

```

lemma lsubst-image:
  assumes distinct (map fst s)
  shows lsubst s ' set (map fst s) = set (map snd s)
by (metis assms list.set-map lsubst-image-aux)

```

```

lemma lsubst-zip-image:
  assumes length xs = length ys
  assumes distinct xs
  shows lsubst (zip xs ys) ' set xs = set ys
using assms
proof (induct xs arbitrary: ys)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  by (metis lsubst-image map-fst-zip map-snd-zip)
qed

```

5.3 Link Substitution Before/After Inverse Link Substitution

Swapping links.

```

abbreviation swap-list :: ('a * 'b) list => ('b * 'a) list
where swap-list xs ≡ map (λ(x, y). (y, x)) xs

```

```

lemma swap-swap-list [simp]:
  swap-list (swap-list xs) = xs
apply (induct xs)
by auto

```

```

lemma swap-map-fst [simp]:
  map fst (swap-list xs) = map snd xs
apply (induct xs)
by auto

```

```

lemma swap-map-snd [simp]:
  map snd (swap-list xs) = map fst xs
apply (induct xs)
by auto

```

```

lemma lsubst-inv-cdom:
  assumes y ∈ set (map snd xs)
  shows lsubst (swap-list xs) y ∈ set (map fst xs)
by (metis assms lsubst-cdom swap-map-fst swap-map-snd)

```

```

lemma lsubst-inv-inv:
  assumes distinct (map fst xs)
  assumes y ∈ set (map snd xs)
  shows lsubst xs (lsbst (swap-list xs) y) = y
using assms
proof (induct xs arbitrary: y)
  case Nil
  then show ?case
  apply auto
  done

```

```

next
case (Cons a xs)
obtain x y' where
  decomp: (x, y') = a
by (metis surj-pair)

then show ?case
proof (cases y = y')
case True
have lsubst ((x, y') # xs) (lsbst (swap-list ((x, y') # xs)) y) = y
  by (simp add: True)
then show ?thesis
  using decomp by fastforce
next
case False
have P0: y ∈ set (map snd xs)
  using Cons.prem1(2) False decomp by fastforce
then have lsubst (swap-list xs) y ∈ set (map fst xs)
  using lsubst-inv-edom by presburger
then have P3: lsubst (swap-list xs) y ≠ x
  using Cons.prem1(1) decomp by force

have lsubst (a # xs) (lsbst (swap-list (a # xs)) y) =
  lsubst ((x, y') # xs) (lsbst (swap-list ((x, y') # xs)) y)
  using decomp by simp
also have ... = lsubst ((x, y') # xs) (lsbst ((y', x) # swap-list xs) y)
  by simp
also have ... = lsubst ((x, y') # xs) (lsbst (swap-list xs) y)
  by (simp add: False)
also have ... = lsubst xs (lsbst (swap-list xs) y)
  by (simp add: P3)
also have ... = y
  using Cons.hyps Cons.prem1(1) P0 by auto
finally show ?thesis .
qed
qed

```

```

corollary lsubst-inv-inv2:
  assumes distinct (map snd xs)
  assumes y ∈ set (map fst xs)
  shows lsubst (swap-list xs) (lsbst xs y) = y
by (metis assms(1,2) lsubst-inv-inv swap-map-fst swap-swap-list)

```

```

lemma map-lsubst-inv-inv:
  assumes distinct (map fst xs)
  assumes set ys ⊆ set (map snd xs)
  shows map (lsbst xs o lsubst (swap-list xs)) ys = ys
using assms
proof (induct ys)
case Nil
then show ?case
  apply auto
done
next
case (Cons a ys)
then show ?case
  apply auto
  using Cons.prem2(2) lsubst-inv-inv by auto
qed

```

```

corollary map-lsubst-zip-inv-inv:
  assumes distinct xs
  assumes set ys' ⊆ set ys
  assumes length xs = length ys

```

shows $\text{map } (\text{lsubst } (\text{zip } xs \ ys) \ o \ \text{lsubst } (\text{zip } ys \ xs)) \ ys' = ys'$
by (*metis* *assms*(1,2,3) *map-fst-zip map-lsubst-inv-inv map-snd-zip zip-commute*)

lemma *lmap-lsubst-inv-inv-aux*:

assumes *distinct* (*map fst xs*)
assumes *distinct* (*map snd xs*)
assumes $\forall y \in FL \ g. \ y \geq k \longrightarrow y - k \in \text{set } (\text{map fst } xs)$
shows $\text{lmap } (\lambda x. \text{if } x < k \text{ then } x \text{ else } (\text{lsubst } (\text{swap-list } xs) \ (x - k) + k))$
 $(\text{lmap } (\lambda x. \text{if } x < k \text{ then } x \text{ else } (\text{lsubst } xs \ (x - k) + k)) \ g) = g$

using *assms*

proof (*induct g arbitrary: k rule: graph.induct[of - %-. True %-. True]*)

case *Zero*
then show *?case*
apply *auto*
done

next

case (*Atom x1 x2*)
then show *?case*
apply *auto*
by (*simp add: FL.simps*(2) *comp-apply lsubst-inv-inv2 map-idI*)

next

case (*Fusion x1 x2*)
then show *?case*
apply *auto*
using *lsubst-inv-inv2* **apply** *fastforce*
using *lsubst-inv-inv2* **apply** *fastforce*
using *lsubst-inv-inv2* **apply** *fastforce*
using *lsubst-inv-inv2* **apply** *fastforce*
using *lsubst-inv-inv2* **apply** *fastforce*
using *lsubst-inv-inv2* **apply** *fastforce*
done

next

case (*Mol x1 x2*)
then show *?case*
apply *auto*
done

next

case (*Nu g'*)
have $\forall y \in FL \ (Nu \ g'). \ y \geq k \longrightarrow y - k \in \text{set } (\text{map fst } xs)$
using *Nu.prem*s(3) **by** *blast*
then have *P2*: $\forall y \in FL \ g'. \ y \geq \text{Suc } k \longrightarrow y - (\text{Suc } k) \in \text{set } (\text{map fst } xs)$
by (*metis FL.simps*(5) *Suc-le-D Suc-le-mono diff-Suc-Suc mem-Collect-eq*)

have *P3*:

$(\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{if } x - 1 < k \text{ then } x - 1 \text{ else } \text{lsubst } (\text{swap-list } xs) \ (x - 1 - k) + k) + 1) =$
 $(\lambda x. \text{if } x < \text{Suc } k \text{ then } x \text{ else } \text{lsubst } (\text{swap-list } xs) \ (x - (\text{Suc } k)) + (\text{Suc } k))$
using *ab-semigroup-add-class.add-ac*(1) *add commute diff-diff-eq not0-implies-Suc not-less-eq*
by *fastforce*

have *P4*:

$(\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{if } x - 1 < k \text{ then } x - 1 \text{ else } \text{lsubst } xs \ (x - 1 - k) + k) + 1) =$
 $(\lambda x. \text{if } x < \text{Suc } k \text{ then } x \text{ else } \text{lsubst } xs \ (x - (\text{Suc } k)) + (\text{Suc } k))$
using *add-cancel-right-right add-diff-inverse-nat diff-Suc-eq-diff-pred less-Suc-eq less-imp-diff-less*
not-less-eq plus-1-eq-Suc **by** *fastforce*

have *P5*:

$\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{if } x - 1 < k \text{ then } x - 1 \text{ else } \text{lsubst } (\text{swap-list } xs) \ (x - 1 - k) + k) + 1)$
 $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{if } x - 1 < k \text{ then } x - 1 \text{ else } \text{lsubst } xs \ (x - 1 - k) + k) + 1) \ g') =$
 $\text{lmap } (\lambda x. \text{if } x < \text{Suc } k \text{ then } x \text{ else } \text{lsubst } (\text{swap-list } xs) \ (x - (\text{Suc } k)) + (\text{Suc } k))$
 $(\text{lmap } (\lambda x. \text{if } x < \text{Suc } k \text{ then } x \text{ else } \text{lsubst } xs \ (x - (\text{Suc } k)) + (\text{Suc } k)) \ g')$
using *P3 P4* **by** *argo*

show *?case*

apply *auto*

using *Nu.hyps Nu.prem*s(2) *P2 P5 assms*(1) **by** *presburger*

qed auto

corollary *lmap-lsubst-inv-inv*:
assumes *distinct* (*map fst xs*)
assumes *distinct* (*map snd xs*)
assumes $FL\ g \subseteq set\ (map\ fst\ xs)$
shows $lmap\ (lsbst\ (swap-list\ xs))\ (lmap\ (lsbst\ xs)\ g) = g$
proof –
have $\forall y \in FL\ g. y \geq 0 \longrightarrow y - 0 \in set\ (map\ fst\ xs)$
apply *auto*
using *assms(3)* **by** *auto*
then have $lmap\ (\lambda x. \text{if } x < 0 \text{ then } x \text{ else } (lsbst\ (swap-list\ xs)\ (x - 0) + 0))$
 $(lmap\ (\lambda x. \text{if } x < 0 \text{ then } x \text{ else } (lsbst\ xs\ (x - 0) + 0))\ g) = g$
using *assms(1,2)* *lmap-lsubst-inv-inv-aux* **by** *blast*
then show *?thesis*
by *simp*
qed

5.4 Basic Property of Link Substitution Over Types

lemma *lmap-ty-inv-inv*:
assumes $length\ xs = length\ ys$
assumes *distinct* *xs*
assumes *distinct* *ys*
assumes $FLty\ ty \subseteq set\ xs$
shows $lmap-ty\ ys\ xs\ (lmap-ty\ xs\ ys\ ty) = ty$
using *assms*
proof (*induct ty*)
case (*TBase x1 x2*)
then show *?case*
apply (*simp add: lmap-ty-def*)
using *map-lsubst-ty-inv-inv* **by** *force*
next
case (*TArrow ty1 ty2 x3*)
then show *?case*
apply (*simp add: lmap-ty-def*)
using *map-lsubst-ty-inv-inv* **by** *force*
qed

Free links of a left-folded *Mol*.

lemma *FL-Mols1*:
 $FL\ (Mols1\ g\ gs) = FL\ g \cup \bigcup (FL\ `set\ gs)$
apply (*induct gs arbitrary: g*)
apply (*simp-all add: Mols1*)
by *auto*

Free links after *i* nested *Nu* binders.

lemma *FL-Nus1*:
 $FL\ (Nus1\ i\ g) = \{n. n + i \in FL\ g\}$
by (*induct i arbitrary: g*) *auto*

lemma *app-prodrule-FL-equiv*:
assumes *A1*: $taus = taus-of-rhs\ rhs$
assumes *A3*: $map\ FLty\ taus = map\ FL\ Ts$
shows $FL\ (app-prodrule\ Ts\ rhs) = FLrhs\ rhs$
proof –
obtain *i C zs fusions taus'* **where**
 $prod-rhs: rhs = (i, (C, zs), fusions, taus')$
by (*metis prod.collapse*)
have $taus: taus' = taus$
by (*simp add: A1 prod-rhs taus-of-rhs-def*)
then have $prod-rhs2: rhs = (i, (C, zs), fusions, taus)$
by (*simp add: prod-rhs*)

```

have FL-fusions:
   $\bigcup (FL \text{ ' set (fusions-of fusions)}) = (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
  apply (simp add: fusions-of-def)
  by (simp add: case-prod-beta)

have
   $FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =$ 
   $\text{set } zs \cup \bigcup (FL \text{ ' set (fusions-of fusions)})$ 
  by (simp add: FL-Mols1)
also have
   $\dots = \text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
  by (simp add: FL-fusions)
finally have P11:
   $FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =$ 
   $\text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
  by simp

have  $\bigcup (FL \text{ ' set } Ts) = \bigcup (\text{set (map FL } Ts))$ 
  by simp
also have  $\dots = \bigcup (\text{set (map FLty taus)})$ 
  using A3 by presburger
also have  $\dots = \bigcup (FLty \text{ ' set taus})$ 
  by simp
finally have P6:  $\bigcup (FL \text{ ' set } Ts) = \bigcup (FLty \text{ ' set taus})$ 
  by simp

have  $FL (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts) =$ 
 $FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))$ 
 $\cup \bigcup (FL \text{ ' set } (Ts))$ 
  by (simp add: FL-Mols1)
also have
   $\dots = (\text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})) \cup \bigcup (FL \text{ ' set } Ts)$ 
  using P11 by presburger
also have
   $\dots = \text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\}) \cup \bigcup (FLty \text{ ' set taus})$ 
  using P6 by presburger
finally have
   $FL (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts) =$ 
 $\text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\}) \cup \bigcup (FLty \text{ ' set taus})$ 
  by blast
then have P30:
   $FL (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =$ 
 $\{n. n + i \in \text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
 $\cup (\bigcup ty \in \text{set taus. } FLty ty)\}$ 
  by (simp add: FL-Nus1)

have  $FL (\text{app-prodrule } Ts \text{ rhs}) = FL (\text{app-prodrule } Ts (i, (C, zs), \text{fusions}, \text{taus}))$ 
  using prod-rhs2 by auto
also have  $\dots =$ 
 $FL (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts))$ 
  by (simp add: app-prodrule-def)
also have  $\dots =$ 
 $\{n. n + i \in \text{set } zs \cup (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
 $\cup (\bigcup ty \in \text{set taus. } FLty ty)\}$ 
  using P30 by blast
also have  $\dots = FLrhs (i, (C, zs), \text{fusions}, \text{taus})$ 
  by (simp add: FLrhs-def)
also have  $\dots = FLrhs \text{ rhs}$ 
  using prod-rhs2 by blast
finally show ?thesis by simp
qed

```

5.5 Link Substitution Over Link Substitution

lemma *lmap-lmap-commute-link-aux*:

```

  assumes  $v \in \text{set } (\text{map } \text{fst } s)$ 
  shows  $f (\text{lsubst } s \ v) =$ 
     $\text{lsubst } (\text{map } (\lambda(x,y). (x, f \ y)) \ s) \ v$ 
using assms
proof (induct s)
  case Nil
  then show ?case
  by auto
next
  case (Cons a s)
  then show ?case
  apply auto
  apply (simp-all add: case-prod-beta lsubst-simp)
  by (simp add: rev-image-eqI)
qed

```

```

lemma lmap-lmap-commute-link:
  assumes  $v \in \text{set } xs$ 
  assumes  $\text{length } xs = \text{length } ys$ 
  shows  $f (\text{lsubst } (\text{zip } xs \ ys) \ v) =$ 
     $\text{lsubst } (\text{zip } xs \ (\text{map } f \ ys)) \ v$ 
proof -
  have  $\text{zip } xs \ (\text{map } f \ ys) = \text{map } (\lambda(x,y). (x, f \ y)) \ (\text{zip } xs \ ys)$ 
  by (simp add: zip-map2)
  then show ?thesis
  by (simp add: assms(1,2) lmap-lmap-commute-link-aux)
qed

```

```

lemma lmap-lmap-commute-links:
  assumes  $\text{set } vs \subseteq \text{set } xs$ 
  assumes  $\text{length } xs = \text{length } ys$ 
  shows  $\text{map } f \ (\text{map } (\text{lsubst } (\text{zip } xs \ ys)) \ vs) =$ 
     $\text{map } (\text{lsubst } (\text{zip } xs \ (\text{map } f \ ys))) \ vs$ 
using assms
proof (induct vs arbitrary: xs ys)
  case Nil
  then show ?case
  by simp
next
  case (Cons a vs)
  then show ?case
  by (simp add: lmap-lmap-commute-link)
qed

```

```

lemma lmap-shift:
  shows  $(\text{lsubst } s) \ x + 1 = \text{lsubst } (\text{map } (\lambda(z, y). (z + 1, y + 1)) \ s) \ (x + 1)$ 
proof (induct s)
  case Nil
  then show ?case
  by simp
next
  case (Cons a s)
  then show ?case
  by (simp-all add: lsubst-simp case-prod-beta)
qed

```

```

lemma lmap-shift2:
  ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } ((\text{lsubst } s) \ (x - 1) + 1)$ ) y =
     $\text{lsubst } ((0, 0) \# \text{map } (\lambda(x, y). (x + 1, y + 1)) \ s) \ y$ 
apply auto
using Suc-eq-plus1 Suc-pred lmap-shift by presburger

```

```

lemma lmap-lmap-commute:
  assumes FL  $G \subseteq \text{set } xs$ 
  assumes  $\text{length } xs = \text{length } ys$ 
  shows  $\text{lmap } f (\text{lmap } (\text{lsubst } (\text{zip } xs \text{ } ys)) \text{ } G) = \text{lmap } (\text{lsubst } (\text{zip } xs (\text{map } f \text{ } ys))) \text{ } G$ 
using assms
proof (induct  $G$  arbitrary:  $xs \text{ } ys \text{ } f$  rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by simp
next
  case (Atom  $p \text{ } vs$ )
  then show ?case
  using lmap-lmap-commute-links by force
next
  case (Fusion  $x1 \text{ } x2$ )
  then show ?case
  apply auto
  apply (simp add: lmap-lmap-commute-link)
  by (simp add: lmap-lmap-commute-link)
next
  case (Mol  $G1 \text{ } G2$ )
  then show ?case
  by auto
next
  case (Nu  $g$ )

  have SucMinus:  $\forall S. \text{Suc } '(\lambda x. x - 1) ' (S - \{0\}) = S - \{0\}$ 
  apply auto
  by (simp add: image-iff)

  have FL  $(Nu \text{ } g) \subseteq \text{set } xs$ 
  using Nu.prem1 by auto
  then have  $\{n. \text{Suc } n \in FL \text{ } g\} \subseteq \text{set } xs$ 
  by auto
  then have  $(\lambda x. x - 1) ' (FL \text{ } g - \{0\}) \subseteq \text{set } xs$ 
  using shift-down-alt by blast
  then have  $\text{Suc } '(\lambda x. x - 1) ' (FL \text{ } g - \{0\}) \subseteq \text{Suc } ' \text{set } xs$ 
  by (meson image-mono)
  then have  $(FL \text{ } g - \{0\}) \subseteq \text{Suc } ' \text{set } xs$ 
  using SucMinus by simp
  then have  $FL \text{ } g \subseteq \text{insert } 0 (\text{Suc } ' \text{set } xs)$ 
  by auto
  then have  $P1: FL \text{ } g \subseteq \text{set } (0 \# \text{map } \text{Suc } xs)$ 
  by simp

  have  $P5: \forall ys. \text{map } (\lambda(x, y). (x + 1, y + 1)) (\text{zip } xs \text{ } ys) = \text{zip } (\text{map } \text{Suc } xs) (\text{map } \text{Suc } ys)$ 
  apply auto
  by (simp add: zip-map-map)

  have IH:  $\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \text{ } (x - 1) + 1))$ 
     $(\text{lmap } (\text{lsubst } (\text{zip } (0 \# \text{map } \text{Suc } xs) (0 \# \text{map } \text{Suc } ys))) \text{ } g) =$ 
 $\text{lmap } (\text{lsubst } (\text{zip } (0 \# \text{map } \text{Suc } xs)$ 
     $(\text{map } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \text{ } (x - 1) + 1)) (0 \# \text{map } \text{Suc } ys)))) \text{ } g$ 
  by (metis (lifting) Nu.hyps Nu.prem2 P1 length-Cons length-map)

  have IH2:  $\text{map } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \text{ } (x - 1) + 1)) (0 \# \text{map } \text{Suc } ys) =$ 
     $0 \# \text{map } \text{Suc } (\text{map } f \text{ } ys)$ 
  apply (induct  $ys$ ) by auto

  have IH3:  $\forall ys. \text{lmap } (\text{lsubst } (\text{zip } xs \text{ } ys)) (Nu \text{ } g) =$ 
     $Nu (\text{lmap } (\text{lsubst } (\text{zip } (0 \# \text{map } \text{Suc } xs) (0 \# \text{map } \text{Suc } ys))) \text{ } g)$ 
proof
  fix  $ys$ 
  have  $\text{lmap } (\text{lsubst } (\text{zip } xs \text{ } ys)) (Nu \text{ } g) =$ 
     $Nu (\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{lsubst } (\text{zip } xs \text{ } ys) (x - 1) + 1)) \text{ } g)$ 
  using lmap.simps(5) by blast

```

```

also have ... =
  Nu (lmap (lsubst ((0, 0) # map (λ(x, y). (x + 1, y + 1)) (zip xs ys))) g)
using lmap-shift2 by presburger
also have ... =
  Nu (lmap (lsubst ((0, 0) # zip (map Suc xs) (map Suc ys))) g)
using P5 by presburger
also have ... =
  Nu (lmap (lsubst (zip (0 # map Suc xs) (0 # map Suc ys))) g)
by simp
finally show
  lmap (lsubst (zip xs ys)) (Nu g) =
  Nu (lmap (lsubst (zip (0 # map Suc xs) (0 # map Suc ys))) g) .
qed

```

```

have lmap f (lmap (lsubst (zip xs ys)) (Nu g)) =
  lmap f (Nu (lmap (lsubst (zip (0 # map Suc xs) (0 # map Suc ys))) g))
using IH3 by presburger
also have ... =
  Nu (lmap (λx. if x = 0 then x else (f (x - 1) + 1))
    (lmap (lsubst (zip (0 # map Suc xs) (0 # map Suc ys))) g))
using lmap.simps(5) by blast
also have ... =
  Nu ((lmap (lsubst (zip (0 # map Suc xs)
    (map (λx. if x = 0 then x else (f (x - 1) + 1)) (0 # map Suc ys)))) g))
using IH by blast
also have ... =
  Nu ((lmap (lsubst (zip (0 # map Suc xs) (0 # map Suc (map f ys)))) g))
using IH2 by argo
also have ... = lmap (lsubst (zip xs (map f ys))) (Nu g)
using IH3 by presburger
finally show ?case .
qed auto

```

6 Properties of Lift and Graph Substitution

6.1 Basic Properties of Lift

lemma lift-graph-lmap-commute:
 lift-graph i (lmap f g) = lmap f (lift-graph i g)
apply (induct g arbitrary: f i) **by** simp-all

lemma lift-graph-freelinks:
 FL (lift-graph i g) = FL g
apply (induct g arbitrary: i) **by** simp-all

lemma lifts-graph-freelinks:
 FL ((lift-graph i $\widehat{\sim}$ n) g) = FL g
apply (induct n)
using lift-graph-freelinks
by auto

lemma lsubst-graph-freelinks:
assumes length xs = length ys
assumes set xs = FL G
assumes distinct xs
shows FL (lmap (lsubst (zip xs ys)) G) = set ys
proof –
have P1: FL (lmap (lsubst (zip xs ys)) G) =
 (lsubst (zip xs ys)) ‘ FL G
by (simp add: FL-lmap-commute)
have P2: lsubst (zip xs ys) ‘ set xs = set ys
by (metis assms(1,3) lsubst-image map-fst-zip map-snd-zip)
then have P3: lsubst (zip xs ys) ‘ FL G = set ys
by (simp add: assms(2))

```

then show ?thesis
  by (simp add: FL-lmap-commute)
qed

```

6.2 Basic Properties of Graph Substitution

```

lemma subst-atom-freelinks:
  assumes set xs = FL g2
  assumes distinct xs
  shows FL (subst-p i xs g2 p ys) = set ys
proof (cases p)
  case (GConstr x1)
  then show ?thesis
  by simp
next
  case (GVar j)
  consider (lt) i < j | (eq) i = j | (gt) j < i
  by arith
  then show ?thesis
proof cases
  case lt
  then show ?thesis
  by (simp add: GVar)
  next
  case eq
  then show ?thesis
  by (simp add: GVar assms(1,2) lsubst-graph-freelinks)
  next
  case gt
  then show ?thesis
  by (simp add: GVar)
qed
next
  case (GAbs x31 x32 x33)
  then show ?thesis
  by simp
qed

```

```

lemma subst-graph-freelinks-equiv:
  assumes set xs = FL g2
  assumes distinct xs
  shows FL (subst-graph i xs g2 g) = FL g
using assms
proof (induct g arbitrary: i xs g2 rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by auto
next
  case (Atom x1 x2)
  then show ?case
  by (simp add: subst-atom-freelinks)
next
  case (Fusion x1 x2)
  then show ?case
  by simp
next
  case (Mol g1 g2)
  then show ?case
  by simp
next
  case (Nu g)
  then show ?case
  by simp
qed auto

```

```

lemma lift-graph-cong:
  fixes g1 g2 :: 'ty graph
  assumes g1  $\simeq$  g2
  shows lift-graph i g1  $\simeq$  lift-graph i g2
  using assms
proof (induction arbitrary: i rule: gcong.induct)
  case (refl g)
  then show ?case
  apply auto
  done
next
  case (sym g1 g2)
  then show ?case
  apply auto
  done
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast
next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
  case (nu-cong g g')
  then show ?case
  by (simp add: gcong.nu-cong)
next
  case (nu-subst-fusion1 y g)
  have P1: lift-graph i (Nu (lmap ( $\lambda x$ . if x = 0 then y else x) g)) =
    Nu (lift-graph i (lmap ( $\lambda x$ . if x = 0 then y else x) g))
    using lift-graph.simps(5) by blast
  have P2: ... = Nu (lmap ( $\lambda x$ . if x = 0 then y else x) (lift-graph i g))
    by (simp add: lift-graph-lmap-commute)
  have P3: y  $\in$  FL (lift-graph i g)
    by (simp add: lift-graph-freelinks local.nu-subst-fusion1)
  then have P5: Nu (lmap ( $\lambda x$ . if x = 0 then y else x) (lift-graph i g))  $\simeq$ 
    Nu (Mol (Fusion 0 y) (lift-graph i g))
    using gcong.nu-subst-fusion1 by blast
  have P6: ... = lift-graph i (Nu (Mol (Fusion 0 y) g))
    by simp
  then show ?case
    by (simp add: P2 P5 gcong.sym)
next
  case (nu-subst-fusion2 g y)
  have P1: lift-graph i (Nu (lmap ( $\lambda x$ . if x = 0 then y else x) g)) =
    Nu (lift-graph i (lmap ( $\lambda x$ . if x = 0 then y else x) g))
    using lift-graph.simps(5) by blast
  have P2: ... = Nu (lmap ( $\lambda x$ . if x = 0 then y else x) (lift-graph i g))
    by (simp add: lift-graph-lmap-commute)
  have P3: 0  $\in$  FL (lift-graph i g)
    by (simp add: lift-graph-freelinks local.nu-subst-fusion2)
  then have P5: Nu (lmap ( $\lambda x$ . if x = 0 then y else x) (lift-graph i g))  $\simeq$ 
    Nu (Mol (Fusion 0 y) (lift-graph i g))
    using gcong.nu-subst-fusion2 by blast

```

```

have P6: ... = lift-graph i (Nu (Mol (Fusion 0 y) g))
  by simp
then show ?case
  by (simp add: P2 P5 gcong.sym)
next
case nu-nu-fusion1
then show ?case
using gcong.nu-nu-fusion1 by auto
next
case nu-nu-fusion2
then show ?case
by simp
next
case nu-zero
then show ?case
by simp
next
case (nu-comm g)
then show ?case
by (simp add: gcong.nu-comm lift-graph-lmap-commute)
next
case (nu-scope g2 g1)
then show ?case
apply auto
by (simp add: gcong.nu-scope lift-graph-lmap-commute)
qed

```

7 Theorems of Congruence

7.1 Free Links Equivalence of Congruent Graphs

We show that the set of free links over congruent graphs are equivalent.

lemma *freelinks-equiv-E6A*:

```

assumes A1: G1 = Nu (Mol (Fusion 0 y) g)
  and A2: G2 = Nu (lmap (λ x. if x = 0 then y else x) g)
  and A3: y ∈ FL g
shows FL G1 = FL G2
proof -
have {n. Suc n ∈ (λ x. if x = 0 then y else x) ‘ FL g} =
  {n. Suc n ∈ FL (Mol (Fusion 0 y) g)}
  apply simp
  using A3 by blast
then show ?thesis
  by (simp add: A1 A2 FL-lmap-commute)
qed

```

lemma *freelinks-equiv-E6B*:

```

assumes A1: G1 = Nu (Mol (Fusion 0 y) g)
  and A2: G2 = Nu (lmap (λ x. if x = 0 then y else x) g)
  and A3: 0 ∈ FL g
shows FL G1 = FL G2
proof -
have P1: FL (lmap (λ x. if x = 0 then y else x) g) =
  (λ x. if x = 0 then y else x) ‘ FL g
  using FL-lmap-commute by blast
have P2: {n. Suc n ∈ (λ x. if x = 0 then y else x) ‘ FL g} =
  {n. Suc n ∈ FL (Mol (Fusion 0 y) g)}
  apply auto
  using A3 by blast
then show ?thesis
  by (simp add: A1 A2 FL-lmap-commute)
qed

```

lemma *freelinks-equiv-E9*:

```

assumes  $G1 = Nu\ (Nu\ g)$ 
and  $G2 = Nu\ (Nu\ (lmap\ (\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g))$ 
shows  $FL\ G1 = FL\ G2$ 
proof –
  have  $P1: FL\ (lmap\ (\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g) =$ 
     $(\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) ' FL\ g$ 
    using FL-lmap-commute by blast

  have  $P2: FL\ (Nu\ (Nu\ (lmap\ (\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g))) =$ 
     $\{n. Suc\ (Suc\ n) \in (\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) ' FL\ g\}$ 
    using Collect-cong FL.simps(5) P1 mem-Collect-eq by auto

  have  $P3: \dots = \{n. Suc\ (Suc\ n) \in (\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) ' FL\ g\}$ 
    using  $P1$  by simp

  have  $P4: \dots = \{n. Suc\ (Suc\ n) \in FL\ g\}$ 
    using diff-Suc-1' by fastforce

  have  $P5: \dots = FL\ (Nu\ (Nu\ g))$ 
    by auto

  show ?thesis
    using  $P2\ P4\ P5\ \text{assms}(1,2)$  by argo
qed

```

```

lemma freelinks-equiv-E10:
  assumes  $A1: G1 = Mol\ (Nu\ g1)\ g2$ 
  and  $A2: G2 = Nu\ (Mol\ g1\ (lmap\ Suc\ g2))$ 
  shows  $FL\ G1 = FL\ G2$ 
proof –
  have  $FL\ G2 = FL\ (Nu\ (Mol\ g1\ (lmap\ Suc\ g2)))$ 
    by (simp add: A2)
  also have  $\dots = \{n. Suc\ n \in FL\ (Mol\ g1\ (lmap\ Suc\ g2))\}$ 
    by simp
  also have  $\dots = \{n. Suc\ n \in (FL\ g1 \cup FL\ (lmap\ Suc\ g2))\}$ 
    by simp
  also have  $\dots = \{n. Suc\ n \in FL\ g1\} \cup \{n. Suc\ n \in FL\ (lmap\ Suc\ g2)\}$ 
    by auto
  also have  $\dots = FL\ (Nu\ g1) \cup \{n. Suc\ n \in FL\ (lmap\ Suc\ g2)\}$ 
    by simp
  also have  $\dots = FL\ (Nu\ g1) \cup FL\ (Nu\ (lmap\ Suc\ g2))$ 
    by simp
  also have  $\dots = FL\ (Nu\ g1) \cup FL\ g2$ 
    by (simp add: FL-lmap-deSuc-Suc)
  also have  $\dots = FL\ (Mol\ (Nu\ g1)\ g2)$ 
    by simp
  also have  $\dots = FL\ G1$ 
    by (simp add: A1)
  finally show ?thesis
    by simp
qed

```

Equivalence of free links over congruent graphs.

```

lemma freelinks-equiv:
  assumes  $g1 \simeq g2$ 
  shows  $FL\ g1 = FL\ g2$ 
using assms
proof (induction rule: gcong.induct)
  case (refl g)
    then show ?case by simp
next
  case (sym g1 g2)
    then show ?case by simp
next
  case (trans g1 g2 g3)
    then show ?case
      by presburger
next
  case (mol-zero g)

```

```

  then show ?case by simp
next
case (mol-comm g1 g2)
  then show ?case
  by (simp add: sup-commute)
next
case (mol-assoc a b c)
  then show ?case by (simp add: sup-assoc)
next
case (mol-cong g1 g2 g1' g2')
  then show ?case
  using FL.simps(4) by blast
next
case (nu-cong g g')
  then show ?case
  using FL.simps(5) by blast
next
case (nu-subst-fusion1 y g)
  then show ?case
  using freelinks-equiv-E6A by blast
next
case (nu-subst-fusion2 y g)
  then show ?case
  using freelinks-equiv-E6B by blast
next
case (nu-nu-fusion1)
  then show ?case
  by auto
next
case (nu-nu-fusion2)
  then show ?case by simp
next
case (nu-zero)
  then show ?case by simp
next
case (nu-comm g)
  then show ?case
  using freelinks-equiv-E9 by blast
next
case (nu-scope g1 g2)
  then show ?case
  using freelinks-equiv-E10 by blast
qed

```

7.2 Multiset Invariant Over Congruent Graphs

Checks that the left-most atom is a constructor atom. Used to distinguish the graph created by `app_prod` and an abstraction atom.

primrec *f-of* :: '*ty* graph => ('*ty* p * nat) multiset

where

```

  f-of Zero = {#}
| f-of (Atom p xs) = {# (p, length xs) #}
| f-of (Fusion x y) = {#}
| f-of (Mol g1 g2) = f-of g1 + f-of g2
| f-of (Nu g) = f-of g

```

lemma *lmap-id*: *lmap id g = g*

apply (*induct g*)

apply *auto*

by (*metis (full-types, lifting) ext Suc-eq-plus1 Suc-pred' neq0-conv*)

lemma *f-lmap-ignore[simp]*:

```

  f-of (lmap f g) = f-of g
by (induct g arbitrary: f) auto

```

```

lemma f-cong-gcong:
  assumes  $g1 \simeq g2$ 
  shows  $f\text{-of } g1 = f\text{-of } g2$ 
using assms
apply (induct set: gcong)
by auto

```

```

lemma abs-cong-inv:
  assumes  $\text{Atom } (GAbs\ xs1\ ty1\ e1)\ ys1 \simeq \text{Atom } (GAbs\ xs2\ ty2\ e2)\ ys2$ 
  shows  $xs1 = xs2 \wedge ty1 = ty2 \wedge e1 = e2 \wedge \text{length } ys1 = \text{length } ys2$ 
proof -
  have  $P1: f\text{-of } (\text{Atom } (GAbs\ xs1\ ty1\ e1)\ ys1) = f\text{-of } (\text{Atom } (GAbs\ xs2\ ty2\ e2)\ ys2)$ 
  using assms f-cong-gcong by blast
  have  $P2: f\text{-of } (\text{Atom } (GAbs\ xs1\ ty1\ e1)\ ys1) = \{\# (GAbs\ xs1\ ty1\ e1, \text{length } ys1) \#\}$ 
  by simp
  have  $P3: f\text{-of } (\text{Atom } (GAbs\ xs2\ ty2\ e2)\ ys2) = \{\# (GAbs\ xs2\ ty2\ e2, \text{length } ys2) \#\}$ 
  by simp
  have  $P4: \{\# (GAbs\ xs1\ ty1\ e1, \text{length } ys1) \#\} = \{\# (GAbs\ xs2\ ty2\ e2, \text{length } ys2) \#\}$ 
  using  $P1\ P2\ P3$  by argo
  then show ?thesis
  by fastforce
qed

```

```

lemma abs-var-cong-inv:
   $\neg(\text{Atom } (GVar\ i)\ ys' \simeq \text{Atom } (GAbs\ xs\ ty1\ e)\ ys)$ 
proof -
  have  $\neg((GVar\ i, \text{length } ys') = (GAbs\ xs\ ty1\ e, \text{length } ys))$ 
  by force
  then have  $\neg(\{\# (GVar\ i, \text{length } ys') \#\} = \{\# (GAbs\ xs\ ty1\ e, \text{length } ys) \#\})$ 
  by simp
  then have  $\neg f\text{-of } (\text{Atom } (GVar\ i)\ ys') = f\text{-of } (\text{Atom } (GAbs\ xs\ ty1\ e)\ ys)$ 
  by simp
  then show ?thesis
  using f-cong-gcong by blast
qed

```

7.3 Lemmata with Lift Graph

```

lemma lift-graph-Nus1-equiv:
   $\text{lift-graph } i\ (Nus1\ j\ G) = Nus1\ j\ (\text{lift-graph } i\ G)$ 
apply (induct j)
by simp-all

```

```

lemma lift-graph-Mols1-equiv:
   $\text{lift-graph } i\ (Mols1\ G\ Gs) = Mols1\ (\text{lift-graph } i\ G)\ (\text{map } (\text{lift-graph } i)\ Gs)$ 
apply (induct Gs arbitrary: G)
by (simp-all add: Mols1)

```

```

lemma lift-graph-fusions-of:
   $\text{map } (\text{lift-graph } i)\ (\text{fusions-of } fusions) = \text{fusions-of } fusions$ 
proof (induction fusions)
  case Nil
  then show ?case
  by (simp add: fusions-of-def)
next
  case (Cons p ps)
  obtain  $x\ y$  where  $p\text{-eq}: p = (x, y)$ 
  by (cases p) auto
  then show ?case
  by (simp add: case-prod-beta fusions-of-def)
qed

```

```

lemma lift-graph-app-prodrule-equiv:
  lift-graph i (app-prodrule Ts rhs) =
    app-prodrule (map (lift-graph i) Ts) rhs
proof –
  obtain j C zs fusions taus
    where rhs-eq: rhs = (j, (C, zs), fusions, taus)
    by (cases rhs) auto

  have P1: lift-graph i (Atom (GConstr C) zs) = Atom (GConstr C) zs
    by simp

  have P2: map (lift-graph i) (fusions-of fusions) = fusions-of fusions
    by (simp add: lift-graph-fusions-of)

  have P3:
    lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
      Mols1 (lift-graph i (Atom (GConstr C) zs))
        (map (lift-graph i) (fusions-of fusions))
    by (simp add: lift-graph-Mols1-equiv)

  from P1 P2 P3 have P4:
    lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
      Mols1 (Atom (GConstr C) zs) (fusions-of fusions)
    by (simp add: lift-graph-fusions-of)

  have P5:
    lift-graph i
      (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
      Nus1 j
        (Mols1 (lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)))
          (map (lift-graph i) Ts))
    by (simp add: lift-graph-Mols1-equiv lift-graph-Nus1-equiv)

  from P4 P5 have P6:
    lift-graph i
      (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
      Nus1 j
        (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
          (map (lift-graph i) Ts))
    by metis

  show ?thesis
    unfolding rhs-eq app-prodrule-def
    using P6 by simp
qed

```

7.4 Lemmata with Subst Graph

```

lemma subst-graph-Nus1-equiv:
  subst-graph i xs g2 (Nus1 j G) = Nus1 j (subst-graph i xs g2 G)
apply (induct j)
by simp-all

lemma subst-graph-Mols1-equiv:
  subst-graph i xs g2 (Mols1 G Gs) = Mols1 (subst-graph i xs g2 G) (map (subst-graph i xs g2) Gs)
apply (induct Gs arbitrary: G)
by (simp-all add: Mols1)

lemma subst-graph-fusions-of:
  map (subst-graph i xs g2) (fusions-of fusions) = fusions-of fusions
proof (induct fusions)
  case Nil
  then show ?case

```

```

by (simp add: fusions-of-def)
next
case (Cons a fusions)
then show ?case
apply (simp-all add: fusions-of-def)
by (simp add: case-prod-beta)
qed

```

lemma *subst-typrod*:

```

subst-graph i xs g2 (app-prodrule Ts rhs) =
  app-prodrule (map (subst-graph i xs g2) Ts) rhs
proof –
obtain j C zs fusions taus
  where rhs-eq: rhs = (j, (C, zs), fusions, taus)
  by (cases rhs) auto

have P1: subst-graph i xs g2 (Atom (GConstr C) zs) = Atom (GConstr C) zs
  by simp

have P2: map (subst-graph i xs g2) (fusions-of fusions) = fusions-of fusions
  by (simp add: subst-graph-fusions-of)

have P3:
  subst-graph i xs g2 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    Mols1 (subst-graph i xs g2 (Atom (GConstr C) zs))
      (map (subst-graph i xs g2) (fusions-of fusions))
  by (simp add: subst-graph-Mols1-equiv)

from P1 P2 P3 have P4:
  subst-graph i xs g2 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    Mols1 (Atom (GConstr C) zs) (fusions-of fusions)
  by simp

```

```

have P5:
  subst-graph i xs g2
    (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
  Nus1 j
    (Mols1 (subst-graph i xs g2 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)))
      (map (subst-graph i xs g2) Ts))
  by (simp add: subst-graph-Mols1-equiv subst-graph-Nus1-equiv)

```

```

from P4 P5 have P6:
  subst-graph i xs g2
    (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
  Nus1 j
    (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
      (map (subst-graph i xs g2) Ts))
  by metis
then show ?thesis
  unfolding rhs-eq app-prodrule-def
  using P6 by simp
qed

```

7.5 Lemmata of Link substitutions

lemma *subst-p-lmap-commute*:

```

assumes FL G ⊆ set xs
shows lmap f (subst-p i xs G p ys) = subst-p i xs G p (map f ys)
using assms
proof (cases p)
  case (GConstr x1)
  then show ?thesis
  by simp
next

```

```

case (GVar j)
consider (lt) i < j | (eq) i = j | (gt) j < i
  by arith
then show ?thesis
proof cases
  case lt
    then show ?thesis
    by (simp add: GVar)
  next
  case eq
    then show ?thesis
    by (simp add: GVar assms(1) lmap-lmap-commute)
  next
  case gt
    then show ?thesis
    by (simp add: GVar)
  qed
next
case (GAbs x31 x32 x33)
then show ?thesis
by simp
qed

```

```

lemma subst-graph-lmap-commute:
  assumes FL G  $\subseteq$  set xs
  shows lmap f (subst-graph i xs G g1) = subst-graph i xs G (lmap f g1)
using assms
proof (induct g1 arbitrary: f rule: graph.induct[of - %-. True %-. True])
  case Zero
    then show ?case
    by auto
  next
  case (Atom x1 x2)
    then show ?case
    apply auto
    by (simp add: subst-p-lmap-commute)
  next
  case (Fusion x1 x2)
    then show ?case
    by auto
  next
  case (Mol g11 g12)
    then show ?case
    by auto
  next
  case (Nu g1)
    then show ?case
    by auto
  qed auto

```

```

lemma lmap-compose:
  lmap f (lmap g G) = lmap (f o g) G
proof (induct G arbitrary: f g rule: graph.induct[of - %-. True %-. True])
  case Zero
    then show ?case
    by simp
  next
  case (Atom x1 x2)
    then show ?case
    by auto
  next
  case (Fusion x1 x2)
    then show ?case
    by auto
  next

```

```

case (Mol G1 G2)
then show ?case
by auto
next
case (Nu G)
have P2: ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1$ ) o ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } g (x - 1) + 1$ ) =
  ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (g (x - 1)) + 1$ )
  using Suc-eq-plus1 Zero-not-Suc diff-Suc-1 by auto
show ?case
apply auto
using Nu P2 by presburger
qed auto

```

```

lemma lmap-lim:
   $\forall x \in FL\ G. f\ x = g\ x ==> lmap\ f\ G = lmap\ g\ G$ 
apply (induct G arbitrary: f g)
by simp-all

```

```

lemma lmap-graph-cong:
   $g1 \simeq g2 ==> lmap\ f\ g1 \simeq lmap\ f\ g2$ 
proof (induct arbitrary: f rule: gcong.induct)
  case (refl g)
  then show ?case
  by blast
next
  case (sym g1 g2)
  then show ?case
  by blast
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast
next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
  case (nu-cong g g')
  then show ?case
  by auto
next
  case (nu-subst-fusion1 y g)

```

```

have  $y \in FL\ g$ 
  by (simp add: local.nu-subst-fusion1)
then have P4: ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1$ ) y
   $\in FL\ (lmap\ (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)\ g)$ 
  using FL-lmap-commute by blast

```

```

have P6: ( $\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)\ y \text{ else } x$ )
   $\circ (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) =$ 
  ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1$ )  $\circ (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x)$ 
  by auto

```

```

have T1:  $\text{lmap } f \text{ (Nu (Mol (Fusion 0 y) g))} =$ 
   $\text{Nu (Mol (Fusion 0 ((\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y))}$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) g))$ 
by auto
also have T2:  $\dots \simeq$ 
   $\text{Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y \text{ else } x)$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) g))$ 
using P4 gcong.nu-subst-fusion1 by blast
have T3:  $\dots =$ 
   $\text{Nu (lmap}$ 
   $((\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y \text{ else } x)$ 
   $o (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)) g)$ 
using lmap-compose by blast
have T4:  $\dots =$ 
   $\text{Nu (lmap}$ 
   $((\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) o (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x)) g)$ 
by (simp add: P6)
have T5:  $\dots =$ 
   $\text{Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x) g))$ 
by (simp add: lmap-compose)
have T6:  $\dots = \text{lmap } f \text{ (Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x) g))$ 
by simp
show ?case
using P6 T2 T3 T5 T6 calculation by argo
next
case (nu-subst-fusion2 g y)
have  $0 \in FL \text{ } g$ 
by (simp add: local.nu-subst-fusion2)
then have P4:  $0$ 
   $\in FL (\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) g)$ 
by (simp add: FL-lmap-commute)

have P6:  $(\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y \text{ else } x)$ 
   $o (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) =$ 
   $(\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) o (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x)$ 
by auto

have T1:  $\text{lmap } f \text{ (Nu (Mol (Fusion 0 y) g))} =$ 
   $\text{Nu (Mol (Fusion 0 ((\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y))}$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) g))$ 
by auto
also have T2:  $\dots \simeq$ 
   $\text{Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y \text{ else } x)$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) g))$ 
using P4 gcong.nu-subst-fusion2 by blast
have T3:  $\dots =$ 
   $\text{Nu (lmap}$ 
   $((\lambda x. \text{if } x = 0 \text{ then } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) y \text{ else } x)$ 
   $o (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)) g)$ 
using lmap-compose by blast
have T4:  $\dots =$ 
   $\text{Nu (lmap}$ 
   $((\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1) o (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x)) g)$ 
by (simp add: P6)
have T5:  $\dots =$ 
   $\text{Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f (x - 1) + 1)$ 
   $(\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x) g))$ 
by (simp add: lmap-compose)
have T6:  $\dots = \text{lmap } f \text{ (Nu (lmap } (\lambda x. \text{if } x = 0 \text{ then } y \text{ else } x) g))$ 
by simp
show ?case
using P6 T2 T3 T5 T6 calculation by argo
next
case nu-nu-fusion1
then show ?case

```

```

apply auto
using gcong.nu-nu-fusion1 by auto
next
  case nu-nu-fusion2
  then show ?case
  by auto
next
  case nu-zero
  then show ?case
  by auto
next
  case (nu-comm g)

have P1:
  ( $\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x$ )
  o ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else (if } x - 1 = 0 \text{ then } x - 1 \text{ else } f(x - 1 - 1) + 1) + 1$ ) =
  ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else (if } x - 1 = 0 \text{ then } x - 1 \text{ else } f(x - 1 - 1) + 1) + 1$ )
  o ( $\lambda x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x$ )
  by auto

show ?case
apply auto
by (metis (no-types, lifting) P1 gcong.nu-comm lmap-compose)
next
  case (nu-scope g1 g2)
  have lmap (Suc o f) g2 = lmap (( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) o Suc) g2
  by (simp add: lmap-lim)
  then have P3: lmap Suc (lmap f g2) = lmap ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) (lmap Suc g2)
  by (simp add: lmap-compose)

have E1:
  lmap f (Mol (Nu g1) g2) =
  Mol (Nu (lmap ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) g1)) (lmap f g2)
  by simp
have E2: ...  $\simeq$ 
  Nu (Mol (lmap ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) g1)
  (lmap ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) (lmap Suc g2)))
  by (metis gcong.nu-scope P3)
have E3: ... = Nu (lmap ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } f(x - 1) + 1$ ) (Mol g1 (lmap Suc g2)))
  by simp
have E4: ... = lmap f (Nu (Mol g1 (lmap Suc g2)))
  by simp

show ?case
using E1 E2 E3 E4 by argo
qed

```

7.6 Graph Substitution Over Congruence Graphs and Expressions

```

lemma subst-graph-cong:
  assumes g1  $\simeq$  g2
  assumes set xs = FL G
  assumes distinct xs
  shows subst-graph i xs G g1  $\simeq$  subst-graph i xs G g2
using assms
proof (induction arbitrary: i xs G rule: gcong.induct)
  case (refl g)
  then show ?case
  by (simp add: gcong.refl)
next
  case (sym g1 g2)
  then show ?case
  by blast
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast

```

```

next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
  case (nu-cong g g')
  then show ?case
  by (simp add: gcong.nu-cong)
next
  case (nu-subst-fusion1 y g)
  then show ?case
  apply auto
  by (metis (no-types, lifting) gcong.nu-subst-fusion1 order-refl subst-graph-freelinks-equiv
    subst-graph-lmap-commute)
next
  case (nu-subst-fusion2 g y)
  then show ?case
  apply auto
  by (metis (no-types, lifting) dual-order.refl gcong.nu-subst-fusion2 subst-graph-freelinks-equiv
    subst-graph-lmap-commute)
next
  case nu-nu-fusion1
  then show ?case
  apply auto
  using gcong.nu-nu-fusion1 by fastforce
next
  case nu-nu-fusion2
  then show ?case
  by auto
next
  case nu-zero
  then show ?case
  by simp
next
  case (nu-comm g)
  then show ?case
  apply auto
  by (metis (no-types, lifting) gcong.nu-comm subsetI subst-graph-lmap-commute)
next
  case (nu-scope g2 g1)
  then show ?case
  apply auto
  by (metis gcong.nu-scope subset-refl subst-graph-lmap-commute)
qed

```

8 Properties of Environments

lemma *insert-at-shift*:

$\text{insert-at } (xs \text{ @ } ys) \ (j + \text{length } xs) \ y = xs \text{ @ } \text{insert-at } ys \ j \ y$
apply (*induction xs*)
by *auto*

Inserting at any position increases the environment length by exactly one.)

lemma *length-insert-at*:

$\text{length } (\text{insert-at } xs \ j \ y) = \text{length } xs + 1$

```

proof (induction xs arbitrary: j)
  case Nil
  then show ?case
  apply (cases j)
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  apply (cases j)
  apply simp
  by simp
qed

```

Relations between the before and the after inserted environments.

```

lemma shift-eq:
  assumes  $j \leq \text{length } xs$ 
  shows  $(\text{insert-at } xs \ j \ y) ! j = y$ 
using assms
proof (induction xs arbitrary: j)
  case Nil
  then show ?case
  apply (cases j)
  apply auto
  done
next
  case (Cons y ys)
  then show ?case
  apply (cases j)
  apply auto
  done
qed

```

```

lemma shift-gt:
  assumes  $i \leq \text{length } xs$ 
  shows  $j < i \implies (\text{insert-at } xs \ i \ y) ! j = xs ! j$ 
using assms
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  apply (cases i)
  apply simp
  by simp
next
  case (Cons a xs)
  then show ?case
  apply (cases i)
  apply auto
  using less-Suc-eq-0-disj by fastforce
qed

```

```

lemma shift-lt:
   $i < j \implies$ 
   $(\text{insert-at } xs \ i \ y) ! j = xs ! (j - 1)$ 
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  apply (cases i)
  apply simp
  by simp
next
  case (Cons a xs)
  then show ?case

```

```

apply (cases i)
apply auto
done
qed

```

Inserting y at the front and then U at position $\text{Suc } i$ is the same as first inserting U at position i and then y at the front.

lemma *shift-commute*:

```

insert-at (insert-at xs i y1) 0 y2
= insert-at (insert-at xs 0 y2) (Suc i) y1

```

proof (induction xs arbitrary: i j)

```

case Nil
then show ?case
by simp

```

next

```

case (Cons a xs)
then show ?case
by simp

```

qed

9 Soundness

9.1 Substitution Lemma

We first show that lifting preserves well-typedness. The lifting operation shifts free variable indices in an expression. This lemma shows that lifting preserves typing: if e is well-typed in environment Γ , then lifting e by i yields a well-typed term in the environment extended by inserting τ' at position i . This property is crucial for the substitution lemma, particularly under the abstraction case.

lemma *lift-type*:

```

assumes  $T: \Gamma \vdash \{P\} \ e : ty$ 
and  $LE: i \leq \text{length } \Gamma$ 
shows  $\text{insert-at } \Gamma \ i \ (ys, ty2) \vdash \{P\} \ (\text{lift } i \ e) : ty$ 
using  $T \ LE$ 
proof (induction arbitrary: i ys ty2 rule: typing.induct)
case (TyVar j  $\Gamma \ xs \ ty \ vs \ P$ )
note  $j\text{-lt} = \text{TyVar.hyps}(1)$ 
note  $\text{lookup} = \text{TyVar.hyps}(2)$ 
note  $\text{len-vs} = \text{TyVar.hyps}(3)$ 
note  $\text{le-i} = \text{TyVar.prem}s$ 

```

show ?case

proof (cases $j < i$)

```

case True
then have  $P1: \text{lift-atom } i \ (GVar \ j) = GVar \ j$ 
by simp

```

```

have  $\text{len-ins}: \text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2)) = \text{length } \Gamma + 1$ 
by (simp add: length-insert-at)

```

```

from  $j\text{-lt}$  have  $j\text{-lt}': j < \text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2))$ 
by (simp add: len-ins)

```

```

from  $\text{le-i}$  True have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! j = \Gamma ! j$ 
by (simp add: shift-gt)

```

```

with  $\text{lookup}$  have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! j = (xs, ty)$ 
by simp

```

with $j\text{-lt}' \ \text{len-vs}$

show ?thesis

using $P1 \ \text{TyVar.hyps}(4,5,6) \ \text{typing.TyVar}$ **by force**

next

case False

then have $i\text{-le-j}: i \leq j$ **by simp**

have $P1: \text{lift-atom } i \ (GVar \ j) = GVar \ (\text{Suc } j)$

```

  by (simp add: False)

have len-ins: length (insert-at  $\Gamma$   $i$  (ys, ty2)) = length  $\Gamma$  + 1
  by (simp add: length-insert-at)

from j-lt have Suc j  $\leq$  length  $\Gamma$ 
  by simp
then have Sucj-lt-ins: Suc j < length (insert-at  $\Gamma$   $i$  (ys, ty2))
  by (simp add: len-ins)

from i-le-j have i < Suc j by simp
then have (insert-at  $\Gamma$   $i$  (ys, ty2)) ! Suc j =  $\Gamma$  ! j
  by (simp add: shift-lt)

with lookup have (insert-at  $\Gamma$   $i$  (ys, ty2)) ! Suc j = (xs, ty)
  by simp

with Sucj-lt-ins len-vs
show ?thesis
  using P1 TyVar.hyps(4,5,6) typing.TyVar by auto
qed

next
case (TyArrow xs ty1  $\Gamma$  P e ty2' vs)
note IH = TyArrow.IH
note le = TyArrow.prem

have Suc-le: Suc i  $\leq$  length ((xs, ty1) #  $\Gamma$ )
  using le by simp

have body-typed:
  insert-at ((xs, ty1) #  $\Gamma$ ) (Suc i) (ys, ty2)  $\vdash$  {P} lift (Suc i) e : ty2'
  using IH Suc-le by blast

have env-eq:
  insert-at ((xs, ty1) #  $\Gamma$ ) (Suc i) (ys, ty2)
  = insert-at (insert-at  $\Gamma$   $i$  (ys, ty2)) 0 (xs, ty1)
  by (simp add: shift-commute)

have (xs, ty1) # insert-at  $\Gamma$   $i$  (ys, ty2)  $\vdash$  {P} lift (Suc i) e : ty2'
  using body-typed by auto

then show ?case
  by (simp add: TyArrow.hyps(2,3) typing.TyArrow)

next
case (TyApp  $\Gamma$  P e1 ty3 ty1 xs e2)
note IH1 = TyApp.IH(1)
note IH2 = TyApp.IH(2)
note le = TyApp.prem

have
  insert-at  $\Gamma$   $i$  (ys, ty2')  $\vdash$  {P} lift i e1 : TArrow ty3 ty1 xs
  insert-at  $\Gamma$   $i$  (ys, ty2')  $\vdash$  {P} lift i e2 : ty3
  apply (simp add: IH1 le)
  by (simp add: IH2 le)

then show ?case
  apply auto
  by (meson IH1 IH2 le typing.TyApp)

next
case (TyCong  $\Gamma$  P g1 ty g2)
note IH = TyCong.IH
note cong = TyCong.hyps(2)
note le = TyCong.prem

```

```

have insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2'$ )  $\vdash \{P\}$  lift  $i$  ( $Graph$   $g1$ ) :  $ty$  for  $ys$   $ty2'$ 
  using  $IH$   $le$  by blast
then show ?case
  by (metis lift.simps(1) lift-graph-cong local.cong typing.TyCong)

next
case (TyAlpha  $\Gamma2$   $P$   $g$   $ty$   $xs$   $vs$ )
note  $IH = TyAlpha.IH$ 
note  $le = TyAlpha.premis(1)$ 
note  $xs-FL = TyAlpha.hyps(2)$ 
note  $len-xs-vs = TyAlpha.hyps(3)$ 

have insert-at  $\Gamma2$   $i$  ( $ys$ ,  $ty2'$ )  $\vdash \{P\}$  lift  $i$  ( $Graph$   $g$ ) :  $ty$  for  $ys$   $ty2'$ 
  using  $IH$   $le$  by blast

then show ?case
  by (simp add: TyAlpha.hyps(4,5) len-xs-vs lift-graph-freelinks lift-graph-lmap-commute typing.TyAlpha
      xs-FL)
next
case (TyProd  $a$   $xs$   $rhs$   $P$   $taus$   $Ts$   $\Gamma$ )

have  $P2$ :
  lift-graph  $i$  (app-prodrule  $Ts$   $rhs$ ) = app-prodrule (map (lift-graph  $i$ )  $Ts$ )  $rhs$ 
  by (simp add: lift-graph-app-prodrule-equiv)

then show ?case
  using  $P2$  TyProd.IH TyProd.hyps(1,2,3,4) TyProd.premis typing.TyProd by auto
next
case (TyCase  $\Gamma$   $P$   $e0$   $ty1$   $\Gamma2$   $e1$   $ty2'$   $e2$   $T$ )
have  $IHe0$ : insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$   $e0$  :  $ty1$ 
  by (simp add: TyCase.IH(1) TyCase.premis)

have insert-at ( $\Gamma2$  @  $\Gamma$ ) ( $i + length$   $\Gamma2$ ) ( $ys$ ,  $ty2$ )
   $\vdash \{P\}$  lift ( $i + length$   $\Gamma2$ )  $e1$  :  $ty2'$ 
  by (simp add: TyCase.IH(2) TyCase.premis)
then have  $IHe1$ :
   $\Gamma2$  @ insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )
   $\vdash \{P\}$  lift ( $i + length$   $\Gamma2$ )  $e1$  :  $ty2'$ 
  by (simp add: insert-at-shift)

have  $IHe2$ : insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$   $e2$  :  $ty2'$ 
  by (simp add: TyCase.IH(3) TyCase.premis)

have
  insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$ 
  Case (lift  $i$   $e0$ )  $\Gamma2$   $T$  (lift ( $i + length$   $\Gamma2$ )  $e1$ ) (lift  $i$   $e2$ ) :  $ty2'$ 
  using  $IHe0$   $IHe1$   $IHe2$  typing.TyCase by blast
then have insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$  (Case  $e0$   $\Gamma2$   $T$   $e1$   $e2$ ) :  $ty2'$ 
  by simp
then show ?case .
qed

lemma weakening-left:
  assumes  $A1$ :  $\Gamma \vdash \{P\}$   $e$  :  $ty$ 
  shows  $x \# \Gamma \vdash \{P\}$  (lift 0  $e$ ) :  $ty$ 
by (metis asms insert-at.simps(1) lift-type surj-pair zero-le)

lemma lifts-graph2:
  (lift 0  $\sim n$ ) (Graph  $g$ ) = Graph ((lift-graph 0  $\sim n$ )  $g$ )
proof (induct  $n$  arbitrary:  $g$ )
  case 0
  then show ?case
  apply auto

```

```

done
next
case (Suc n)
then show ?case
apply auto
done
qed

```

```

lemma weakening-lefts:
  assumes A1:  $\Gamma \vdash \{P\} \ e : ty$ 
  shows  $\Gamma 2 @ \Gamma \vdash \{P\} \ (lift\ 0 \ \sim \ length\ \Gamma 2) \ e : ty$ 
using assms
proof (induct  $\Gamma 2$  arbitrary: e)
  case Nil
  then show ?case
  apply auto
  done
next
case (Cons a  $\Gamma 2$ )
  then show ?case
  apply auto
  by (simp add: weakening-left)
qed

```

```

lemma subst-lemma:
  fixes i :: nat
  and e1 :: type exp
  and g2 :: type graph
  and ty1 ty2 :: type
  and  $\Gamma 1 \ \Gamma 2 :: tyenv$ 
  assumes A1:  $\Gamma 1 \vdash \{P\} \ e1 : ty1$ 
  and A2:  $\Gamma 2 \vdash \{P\} \ Graph\ g2 : ty2$ 
  and A3:  $i \leq length\ \Gamma 2$ 
  and A4:  $\Gamma 1 = insert-at\ \Gamma 2\ i\ (xs, ty2)$ 
  and A5:  $set\ xs = FL\ g2$ 
  and A6:  $distinct\ xs$ 
  shows  $\Gamma 2 \vdash \{P\} \ subst\ i\ xs\ g2\ e1 : ty1$ 
using assms
proof (induct arbitrary:  $\Gamma 2\ i\ ty2\ g2\ xs\ set: typing$ )

  case (TyCong  $\Gamma' \ P\ g1' \ ty' \ g2'$ )

  have IHFL1:  $FL\ g2 = set\ xs$ 
  by (simp add: TyCong.prem5(4))

  have P1:  $\Gamma' \vdash \{P\} \ (Graph\ g1') : ty'$ 
  by (simp add: TyCong.hyps(1))
  have P2:  $\Gamma 2 \vdash \{P\} \ (subst\ i\ xs\ g2\ ((Graph\ g1')) : ty'$ 
  using IHFL1 TyCong.hyps(2) TyCong.prem5(1,2,3,5) by presburger
  have P5:  $\Gamma' \vdash \{P\} \ (Graph\ g2') : ty'$ 
  using P1 TyCong.hyps(3) typing.TyCong by presburger

  have IH:  $g1' \simeq g2'$ 
  by (simp add: TyCong.hyps(3))
  then show ?case
  by (metis IHFL1 P2 TyCong.prem5(5) subst.simp(1) subst-graph-cong typing.TyCong)
next
case (TyAlpha  $\Gamma 2 \ P\ g\ ty\ xs\ ys$ )
  then show ?case
  by (metis subset-refl subst.simp(1) subst-graph-freelinks-equiv subst-graph-lmap-commute
    typing.TyAlpha)
next
case (TyArrow  $xs' \ ty1' \ \Gamma' \ P \ e' \ ty2' \ ys'$ )

```

```

have PremT:  $\Gamma 2 \vdash \{P\}$  (Graph g2) : ty2
by (simp add: TyArrow.prem(1))
have PremLE:  $0 \leq \text{length } \Gamma 2$ 
by simp

then have LIFT1: insert-at  $\Gamma 2$  0 (xs', ty1')  $\vdash \{P\}$  (lift 0 (Graph g2)) : ty2
using PremT PremLE lift-type
by presburger
then have LIFT: (xs', ty1')  $\# \Gamma 2 \vdash \{P\}$  (lift 0 (Graph g2)) : ty2
by simp

have LE: Suc i  $\leq \text{length } ((xs', ty1') \# \Gamma 2)$ 
by (simp add: TyArrow.prem(2,4) le-SucI length-insert-at)
have ENV: (xs', ty1')  $\# \Gamma' = \text{insert-at } ((xs', ty1') \# \Gamma 2)$  (Suc i) (xs, ty2)
by (simp add: TyArrow.prem(3))

have IH: (xs', ty1')  $\# \Gamma 2 \vdash \{P\}$  (subst (Suc i) xs (lift-graph 0 g2)) e' : ty2'
by (metis ENV LE LIFT1 TyArrow.hyps(2) TyArrow.prem(4,5) insert-at.simps(1) lift.simps(1)
lift-graph-freelinks)
show ?case
by (metis IH Suc-eq-plus1 TyArrow.hyps(3,4) subst.simps(1) subst-graph.simps(2) subst-p.simps(2)
typing.TyArrow)

next
case (TyVar j  $\Gamma$  xs' ty' ys P)
note TV = TyVar

have len- $\Gamma 1$ :  $\text{length } \Gamma = \text{length } \Gamma 2 + 1$ 
by (simp add: TyVar.prem(3) length-insert-at)

have subst-GVar:
subst i xs g2 (Graph (Atom (GVar j) ys)) =
(if i < j then Graph (Atom (GVar (j - 1)) ys)
else if j = i  $\wedge$   $\text{length } xs = \text{length } ys$ 
then Graph (lmap (lsubst (zip xs ys)) g2)
else Graph (Atom (GVar j) ys))
by simp

consider (lt) i < j | (eq) i = j | (gt) j < i
by arith
then show ?case
proof cases
case lt

then have subst-ij:
subst i xs g2 (Graph (Atom (GVar j) ys))
= Graph (Atom (GVar (j - 1)) ys)
by (simp add: subst-GVar)

from TyVar.hyps(1) TyVar.prem(4) len- $\Gamma 1$  lt
have j1-lt-len- $\Gamma 2$ : j - 1 <  $\text{length } \Gamma 2$ 
by (simp add: less-diff-conv)

from TyVar.prem(3) lt
have lookup-shift:
 $\Gamma ! j = \Gamma 2 ! (j - 1)$ 
by (simp add: shift-lt)

from TyVar.hyps(2) lookup-shift
have  $\Gamma 2 ! (j - 1) = (xs', ty')$  by simp

moreover have  $\text{length } xs' = \text{length } ys$ 
using TyVar.hyps(3) .

ultimately have
 $\Gamma 2 \vdash \{P\}$  Graph (Atom (GVar (j - 1)) ys) : lmap-ty xs' ys ty'

```

```

using TyVar.hyps(4,5,6) j1-lt-len-Γ2 typing.TyVar by blast

thus ?thesis
  using subst-ij by simp
next
case eq
have subst-eq:
  subst i xs g2 (Graph (Atom (GVar j) ys)) =
    (if length xs = length ys
     then Graph (lmap (lsubst (zip xs ys)) g2)
     else Graph (Atom (GVar j) ys))
  by (simp add: eq)

from TyVar.hyps(2) TyVar.prem(4) eq
have (xs', ty') = (xs, ty2)
proof -
  from TyVar.prem(3)
  have Γ ! j = (insert-at Γ 2 i (xs, ty2)) ! j
  by simp
  with eq TyVar.hyps(2) show ?thesis
    using TyVar.prem(2) shift-eq by fastforce
qed
then have xs'-xs: xs' = xs and ty'-ty2: ty' = ty2
  by auto

have len-xs-ys: length xs = length ys
  using TyVar.hyps(3) xs'-xs by simp

have subst-eq':
  subst i xs g2 (Graph (Atom (GVar j) ys))
    = Graph (lmap (lsubst (zip xs ys)) g2)
  using subst-eq len-xs-ys by simp

have Γ2 ⊢ {P} Graph (lmap (lsubst (zip xs ys)) g2)
  : lmap-ty xs ys ty2
  using TyVar.prem(2) TyVar.prem(4) len-xs-ys
  by (simp add: TyAlpha TyVar.hyps(5) TyVar.prem(1,5))

then show ?thesis
  using subst-eq' ty'-ty2 xs'-xs by presburger
next
case gt

have subst-gt:
  subst i xs g2 (Graph (Atom (GVar j) ys))
    = Graph (Atom (GVar j) ys)
  using gt subst-GVar by simp

from TyVar.prem(4) gt
have lookup-same:
  Γ ! j = Γ2 ! j
  by (simp add: TyVar.prem(2,3) shift-gt)

from TyVar.hyps(2) lookup-same
have lookup2: Γ2 ! j = (xs', ty') by simp

have j-gt-len-Γ2: j < length Γ2
proof -
  from TyVar.hyps(1) len-Γ1 have j < length Γ2 + 1 by simp
  with gt show ?thesis
    using TyVar.prem(2) dual-order.strict-trans1 by blast
qed

have length xs' = length ys using TyVar.hyps(3) .
then have Γ2 ⊢ {P} Graph (Atom (GVar j) ys) : lmap-ty xs' ys ty'
  by (simp add: TyVar.hyps(4,5,6) j-gt-len-Γ2 lookup2 typing.TyVar)

```

```

thus ?thesis
using subst-gt by simp
qed
next
case (TyApp  $\Gamma$   $P$   $e1$   $ty2$   $ty1$   $xs$   $e2$ )
show ?case
apply auto
by (meson TyApp.hyps(2,4) TyApp.prem(1,2,3,4,5) typing.TyApp)

next
case (TyProd  $a$   $zs$   $rhs$   $P$   $taus$   $Ts$   $\Gamma$ )
have  $P1$ : subst-graph  $i$   $xs$   $g2$  (app-prodrule  $Ts$   $rhs$ ) =
  app-prodrule (map (subst-graph  $i$   $xs$   $g2$ )  $Ts$ )  $rhs$ 
by (simp add: subst-typrod)

have  $P10$ :
 $\forall j < \text{length } Ts. \Gamma 2 \vdash \{P\} \text{ subst } i \text{ } xs \text{ } g2 \text{ } (\text{Graph } (Ts ! j)) : \text{taus} ! j$ 
proof (intro allI impI)
  fix  $j$  assume  $Hj$ :  $j < \text{length } Ts$ 

  from TyProd.hyps(4)[rule-format, of  $j$ ]  $Hj$ 
  have  $IHj$ :
     $\Gamma \vdash \{P\} \text{ Graph } (Ts ! j) : \text{taus} ! j \wedge$ 
     $(\forall x \text{ } xa \text{ } xb \text{ } xc.$ 
       $x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$ 
       $(\forall xd. \text{set } xd = FL \text{ } xc \longrightarrow$ 
         $\text{set } xd = FLty \text{ } xb \longrightarrow$ 
         $\Gamma = \text{insert-at } x \text{ } xa \text{ } (xd, xb) \longrightarrow$ 
         $\text{distinct } xd \longrightarrow$ 
         $x \vdash \{P\} \text{ subst } xa \text{ } xd \text{ } xc \text{ } (\text{Graph } (Ts ! j)) : \text{taus} ! j))$ 
    by auto

  then have  $IHj\text{-subst}$ :
     $\forall x \text{ } xa \text{ } xb \text{ } xc.$ 
     $x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$ 
     $(\forall xd. \text{set } xd = FL \text{ } xc \longrightarrow$ 
       $\text{set } xd = FLty \text{ } xb \longrightarrow$ 
       $\Gamma = \text{insert-at } x \text{ } xa \text{ } (xd, xb) \longrightarrow$ 
       $\text{distinct } xd \longrightarrow$ 
       $x \vdash \{P\} \text{ subst } xa \text{ } xd \text{ } xc \text{ } (\text{Graph } (Ts ! j)) : \text{taus} ! j)$ 
    by auto

  show  $\Gamma 2 \vdash \{P\} \text{ subst } i \text{ } xs \text{ } g2 \text{ } (\text{Graph } (Ts ! j)) : \text{taus} ! j$ 
  using  $Hj$  TyProd.prem(1,2,3,4,5)
   $\langle j < \text{length } Ts \implies \Gamma \vdash \{P\} \text{ Graph } (Ts ! j) : \text{taus} ! j \wedge (\forall x \text{ } xa \text{ } xb \text{ } xc. x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$ 
     $(\forall xd. \Gamma = \text{insert-at } x \text{ } xa \text{ } (xd, xb) \longrightarrow \text{set } xd = FL \text{ } xc \longrightarrow \text{distinct } xd \longrightarrow x \vdash \{P\} \text{ subst } xa \text{ } xd \text{ } xc \text{ } (\text{Graph } (Ts ! j)) : \text{taus} ! j)) \rangle$ 
  by presburger
qed

then show ?case
apply auto
using  $P1$  TyProd.hyps(1,2,3,5) typing.TyProd by auto
next
case (TyCase  $\Gamma$   $P$   $e0$   $ty1'$   $\Gamma 2'$   $e1$   $ty2'$   $e2$   $T$ )

have  $IHe0$ :  $\Gamma 2 \vdash \{P\} \text{ subst } i \text{ } xs \text{ } g2 \text{ } e0 : ty1'$ 
using TyCase.hyps(2) TyCase.prem(1,2,3,4,5) by presburger

have  $\Gamma 2 \vdash \{P\} \text{ Graph } g2 : ty2$ 
by (simp add: TyCase.prem(1))
then have  $P5$ :  $\Gamma 2' @ \Gamma 2 \vdash \{P\} (\text{lift } 0 \text{ } \sim \text{length } \Gamma 2') (\text{Graph } g2) : ty2$ 
by (simp add: weakening-lefts)

have  $i \leq \text{length } \Gamma 2$ 

```

```

  by (simp add: TyCase.prem1(2))
then have P4:  $i + \text{length } \Gamma 2' \leq \text{length } (\Gamma 2' @ \Gamma 2)$ 
  by auto

have  $\Gamma = \text{insert-at } \Gamma 2 \ i \ (xs, ty2)$ 
  by (simp add: TyCase.prem1(3))
then have IHe1:
   $\Gamma 2' @ \Gamma = \text{insert-at } (\Gamma 2' @ \Gamma 2) \ (i + \text{length } \Gamma 2') \ (xs, ty2)$ 
  by (simp add: insert-at-shift)
then have IHe1B:
   $\Gamma 2' @ \Gamma 2 \vdash \{P\}$ 
  subst (i + length  $\Gamma 2'$ ) xs ((lift-graph 0  $\rightsquigarrow$  (length  $\Gamma 2'$ )) g2) e1 : ty2'
  by (metis P4 P5 TyCase.hyps(4) TyCase.prem1(4,5) lifts-graph2 lifts-graph-freelinks)

have IHe2:  $\Gamma 2 \vdash \{P\}$  subst i xs g2 e2 : ty2'
  using TyCase.hyps(6) TyCase.prem1(1,2,3,4,5) by presburger

have T1:  $\Gamma 2 \vdash \{P\}$ 
  Case (subst i xs g2 e0)  $\Gamma 2'$  T
    (subst (i + length  $\Gamma 2'$ ) xs ((lift-graph 0  $\rightsquigarrow$  (length  $\Gamma 2'$ )) g2) e1)
  (subst i xs g2 e2)
  : ty2'
  using IHe0 IHe1B IHe2 typing.TyCase by blast
then show ?case
  by simp
qed

```

9.2 Inversion Lemma

```

lemma FL-ty-lmap:
  assumes A1:  $FLty \ ty = \text{set } xs$ 
  assumes A2:  $\text{length } xs = \text{length } ys$ 
  assumes A3:  $\text{distinct } xs$ 
  shows  $FLty \ (lmap-ty \ xs \ ys \ ty) = \text{set } ys$ 
proof (cases ty)
  case (TBase x11 x12)
  then show ?thesis
  apply (simp add: FLty-def)
  using A1 A2 A3 lsubst-zip-image by auto
next
  case (TArrow x21 x22 x23)
  then show ?thesis
  apply (simp add: FLty-def)
  using A1 A2 A3 lsubst-zip-image by auto
qed

```

```

lemma FL-typing-LRHS:
  assumes  $\Gamma \vdash \{P\}$   $G : ty$ 
  assumes  $G = \text{Graph } g$ 
  shows  $FL \ g = FLty \ ty$ 
using assms
proof (induction arbitrary: g set: typing)
  case (TyVar i  $\Gamma$  xs ty ys P)
  then show ?case
  using FL-ty-lmap by force
next
  case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
  then show ?case
  apply auto
  done
next
  case (TyApp  $\Gamma$  P e1 ty2 ty1 xs e2)
  then show ?case
  apply auto
  done

```

```

next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  have  $\forall i < \text{length } Ts. FLty (\text{taus-of-rhs } rhs \ ! \ i) = FL (Ts \ ! \ i)$ 
    by (simp add: TyProd.IH TyProd.hyps(2))
  then have  $P2: \text{map } FLty (\text{taus-of-rhs } rhs) = \text{map } FL Ts$ 
    using TyProd.hyps(2,3) map-equality-iff by blast
  then show ?case
  apply auto
  using TyProd.hyps(4) TyProd.prem1 app-prodrule-FL-equiv apply blast
  using TyProd.hyps(4) TyProd.prem1 app-prodrule-FL-equiv by blast
next
  case (TyCong  $\Gamma P g1 ty g2$ )
  then show ?case
  apply simp
  by (simp add: freelinks-equiv)
next
  case (TyAlpha  $\Gamma2 P g ty xs ys$ )
  then show ?case
  by (metis (mono-tags, lifting) FL-ty-lmap exp.inject(1) lsubst-graph-freelinks)
next
  case (TyCase  $\Gamma P e0 ty1 \Gamma2 e1 ty2 e2 T$ )
  then show ?case
  by auto
qed

lemma typing-inv-graph-abs-aux:
  assumes  $T: \Gamma \vdash \{P\} \ e' : ty$ 
  assumes  $ty: ty = TArrow \ ty1 \ ty2 \ zs$ 
    and  $E: e' = Graph \ g'$ 
    and  $C: g' \simeq Atom (GAbs \ xs \ ty3 \ e) \ ys$ 
  shows  $(xs, ty1) \# \Gamma \vdash \{P\} \ e : ty2$ 
using assms
proof (induction arbitrary:  $xs \ ty1 \ ty2 \ zs \ e \ ys \ g'$  rule: typing.induct)
  case (TyVar  $i \Gamma xs ty ys P$ )
  then show ?case
  apply auto
  by (simp add: abs-var-cong-inv)
next
  case (TyArrow  $xs' \ ty1' \Gamma P e2' ty2' ys'$ )
  then show ?case
  by (metis abs-cong-inv exp.inject(1) type.inject(2))
next
  case (TyApp  $\Gamma P e1 ty1 ty2 xs e2$ )
  then show ?case
  apply auto
  done
next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  then show ?case
  apply auto
  done
next
  case (TyCong  $\Gamma P e1 ty e2$ )
  then show ?case
  apply simp
  by (meson gcong.trans)
next
  case (TyAlpha  $\Gamma2' P g ty' xs' ys'$ )

  have  $P1: \text{set } xs' = FL \ g$ 
    using TyAlpha.hyps(2) by fastforce
  have  $P2: \text{set } ys' = FL (\text{lmap } (\text{lsbst } (\text{zip } xs' \ ys')) \ g)$ 
    by (simp add: P1 TyAlpha.hyps(3,4) lsubst-graph-freelinks)
  have  $P3: \text{lmap } (\text{lsbst } (\text{zip } xs' \ ys')) \ g \simeq Atom (GAbs \ xs \ ty3 \ e) \ ys$ 
    using TyAlpha.prem1(2,3) by blast
  then have  $P4: FL (\text{lmap } (\text{lsbst } (\text{zip } xs' \ ys')) \ g) = FL (Atom (GAbs \ xs \ ty3 \ e) \ ys)$ 

```

```

  by (simp add: freelinks-equiv)
then have P7: set ys' = set ys
  by (simp add: P2)

then have P8: distinct xs'
  by (simp add: TyAlpha.hyps(4))
then have P9: distinct ys'
  by (simp add: TyAlpha.hyps(5))

have P6: lmap (lsubst (zip ys' xs')) (lmap (lsubst (zip xs' ys')) g)  $\simeq$ 
  lmap (lsubst (zip ys' xs')) (Atom (GAbs xs ty3 e) ys)
  using P3 lmap-graph-cong by blast
have P7: lmap (lsubst (zip ys' xs')) (lmap (lsubst (zip xs' ys')) g) = g
  by (metis P1 P8 P9 TyAlpha.hyps(3) dual-order.refl lmap-lsubst-inv-inv map-fst-map-snd-map-ty3-commute)
have P8: g  $\simeq$  lmap (lsubst (zip ys' xs')) (Atom (GAbs xs ty3 e) ys)
  using P6 P7 by auto
then have P9: g  $\simeq$  Atom (GAbs xs ty3 e) (map (lsubst (zip ys' xs')) ys)
  by simp

have P14: FLty ty' = FL g
  using FL-typing-LRHS TyAlpha.hyps(1) by presburger
then have P15: FLty ty' = set xs'
  by (simp add: P1)

have P10: lmap-ty xs' ys' ty' = TArrow ty1 ty2 zs
  by (simp add: TyAlpha.prem(1))
have P12: lmap-ty xs' xs' (lmap-ty xs' ys' ty') = lmap-ty xs' xs' (TArrow ty1 ty2 zs)
  using P10 by presburger
have P13: ty' = lmap-ty xs' xs' (TArrow ty1 ty2 zs)
  by (metis P10 P15 TyAlpha.hyps(3,4,5) dual-order.refl lmap-ty-inv-inv)
then have P16: ty' = TArrow ty1 ty2 (map (lsubst (zip ys' xs')) zs)
  by auto

show ?case
  using P16 P9 TyAlpha.IH by blast
next
case (TyCase  $\Gamma$  P e0 ty1  $\Gamma$ 2 e1 ty2 e2 T)
then show ?case
  by auto
qed

```

corollary *typing-inv-graph-abs*:

```

assumes T:  $\Gamma \vdash \{P\}$  Graph (Atom (GAbs xs ty3 e) ys) : TArrow ty1 ty2 zs
shows (xs, ty1)  $\#$   $\Gamma \vdash \{P\}$  e : ty2
by (meson assms gcong.refl typing-inv-graph-abs-aux)

```

9.3 Subject Reduction

Subject reduction: If a term e reduces to e' by a call-by-value step, and e is well-typed, then e' is also well-typed with the same type. This ensures that evaluation preserves typing.

lemma *weakening-right*:

```

assumes  $\Gamma \vdash \{P\}$  e :  $\tau$ 
shows  $\Gamma @ \Gamma^2 \vdash \{P\}$  e :  $\tau$ 
using assms
proof (induct set: typing)
case (TyVar i  $\Gamma$  xs ty ys P)
then show ?case
  by (simp add: nth-append-left typing.TyVar)
next
case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
then show ?case
  by (simp add: typing.TyArrow)
next
case (TyApp  $\Gamma$  P e1 ty1 ty2 xs e2)

```

```

then show ?case
by (simp add: typing.TyApp)
next
case (TyProd a xs rhs P taus Ts  $\Gamma$ )
then show ?case
by (simp add: typing.TyProd)
next
case (TyCong  $\Gamma$  P g1 ty g2)
then show ?case
using typing.TyCong by presburger
next
case (TyAlpha  $\Gamma$ 2 P g ty xs ys)
then show ?case
by (simp add: typing.TyAlpha)
next
case (TyCase  $\Gamma$  P e0 ty1  $\Gamma$ 2 e1 ty2 e2 T)
then show ?case
by (simp add: typing.TyCase)
qed

```

```

lemma substs-lemma0:
  assumes A1:  $(xs, ty) \# \Gamma 1 \vdash \{P\}$  e1 : ty1
  assumes A2: distinct xs
  assumes A3: FL g = set xs
  assumes A4:  $\square \vdash \{P\}$  (Graph g) : ty
  shows  $\Gamma 1 \vdash \{P\}$  subst 0 xs g e1 : ty1
proof -
  have P1:  $\Gamma 1 \vdash \{P\}$  Graph g : ty
  by (metis A4 append-Nil weakening-right)
  then show ?thesis
  using A1 A2 A3 A4 subst-lemma by auto
qed

```

```

lemma substs-lemma-aux:
  assumes A1:  $\Gamma = \text{map fst } XsGs$ 
  assumes A2:  $\Gamma @ \Gamma 1 \vdash \{P\}$  e1 : ty1
  assumes A3:  $\forall ((xs, ty), g) \in \text{set } XsGs. ($ 
    distinct xs  $\wedge$  FL g = set xs  $\wedge$ 
     $\square \vdash \{P\}$  Graph g : ty)
  shows  $\Gamma 1 \vdash \{P\}$  substs XsGs e1 : ty1
using assms
proof (induct XsGs arbitrary:  $\Gamma$   $\Gamma 1$  e1 ty1)
  case Nil
  then show ?case
  by (simp add: substs)
next
  case (Cons a  $\Gamma$ )
  then show ?case
  apply (simp add: substs)
  using substs-lemma0 by force
qed

```

```

lemma substs-lemma:
  assumes A1:  $\Gamma @ \Gamma 1 \vdash \{P\}$  e1 : ty1
  assumes A2: length  $\Gamma$  = length gs
  assumes A3: XsGs = zip  $\Gamma$  gs
  assumes A4:  $\forall ((xs, ty), g) \in \text{set } XsGs. ($ 
    distinct xs  $\wedge$  FL g = set xs  $\wedge$ 
     $\square \vdash \{P\}$  Graph g : ty)
  shows  $\Gamma 1 \vdash \{P\}$  substs XsGs e1 : ty1
using substs-lemma-aux apply auto
using A1 A2 A3 A4 map-fst-zip split-beta by auto

```

Subject reduction: If a term e reduces to e' by a call-by-value step, and e is well-typed, then e' is also well-typed with the same type. This ensures that evaluation preserves typing.

```

lemma subject-reduction:
  assumes  $\Gamma \vdash \{P\} e : \tau$ 
    and  $e \rightarrow_{\{P\}} e'$ 
    shows  $\Gamma \vdash \{P\} e' : \tau$ 
using assms
proof (induct arbitrary: e' set: typing)
  case (TyVar  $i \Gamma xs ty ys P$ )
    then show ?case
    apply auto
    done
next
  case (TyArrow  $xs ty1 \Gamma P e ty2 ys$ )
    then show ?case
    apply auto
    done
next
  case (TyApp  $\Gamma P e1 ty1 ty2 zs e2$ )

    note  $T\text{-fun} = \text{TyApp.hyps}(1)$ 
    have  $\Gamma \vdash \{P\} e1 : T\text{Arrow } ty1 ty2 zs$ 
      by (simp add: T-fun)

    note  $T\text{-arg} = \text{TyApp.hyps}(3)$ 
    have  $\Gamma \vdash \{P\} e2 : ty1$ 
      by (simp add: T-arg)

    note  $IH\text{-fun} = \text{TyApp.hyps}(2)$ 
    note  $IH\text{-arg} = \text{TyApp.hyps}(4)$ 
    note  $Step = \text{TyApp.prem}$ 

    show ?case
    proof (cases rule: cbv-ty-AppE[OF Step])
      case ( $1 v xs lam ty e ys$ )

        have  $P1: (xs, ty1) \# \Gamma \vdash \{P\} e : ty2 ==>$ 
           $\Gamma \vdash \{P\} \text{Graph } (Atom (GAbs xs ty1 e) ys) : T\text{Arrow } ty1 ty2 ys$ 
          using  $1(3,4,7)$  FL-typing-LRHS T-arg TyArrow by auto

        have  $P2: \Gamma \vdash \{P\} \text{Graph } (Atom (GAbs xs ty e) ys) : T\text{Arrow } ty1 ty2 zs$ 
          using  $1(5)$  T-fun
          by (metis  $1(6)$  TyCong)
        have  $P3: (xs, ty1) \# \Gamma \vdash \{P\} e : ty2$ 
          using  $P2$  typing-inv-graph-abs by auto
        then show ?thesis
          using  $1(1,3,4,7)$  T-arg subst-lemma by force
      next
        case ( $2 e1'$ )
        then show ?thesis
        apply auto
        using  $IH\text{-fun } T\text{-arg } typing.TyApp$  by blast
      next
        case ( $3 e'$ )
        then show ?thesis
        apply auto
        using  $IH\text{-arg } T\text{-fun } typing.TyApp$  by blast
    qed
next
  case (TyProd  $a xs rhs P taus Ts \Gamma$ )
    then show ?case
    apply auto
    done
next
  case (TyCong  $\Gamma P g1 ty g2$ )
    then show ?case
    by auto
next

```

```

case (TyAlpha  $\Gamma$  2 P g ty xs ys)
then show ?case
apply auto
done
next
case (TyCase  $\Gamma$  P e0' ty1'  $\Gamma$  2' e1' ty2' e2' T)

note Step = TyCase.premss

have P1:  $\Gamma \vdash \{P\} \text{ e0}' : \text{ty1}'$ 
  by (simp add: TyCase.hyps(1))
have P3:  $\Gamma$  2' @  $\Gamma \vdash \{P\} \text{ e1}' : \text{ty2}'$ 
  by (simp add: TyCase.hyps(3))
have P5:  $\Gamma \vdash \{P\} \text{ e2}' : \text{ty2}'$ 
  by (simp add: TyCase.hyps(5))
have P7: Case e0'  $\Gamma$  2' T e1' e2'  $\rightarrow \{P\} \text{ e}'$ 
  by (simp add: Step)

show ?case
proof (cases rule: cbv-ty-CaseE[OF Step])
case (1 v XsGs)
then show ?thesis
  using P3 substs-lemma by presburger
next
case (2 v)
then show ?thesis
  apply auto
  using P5 by blast
next
case (3 e0')
then show ?thesis
  apply auto
  using P3 P5 TyCase.hyps(2) typing.TyCase by blast
qed
qed

```

Multi-step subject reduction: If e reduces to e' in zero or more steps, then e' has the same type as e .

```

corollary subject-reduction':
  assumes e  $\rightarrow^* \{P\} \text{ e}'$ 
  and  $\Gamma \vdash \{P\} \text{ e} : \tau$ 
  shows  $\Gamma \vdash \{P\} \text{ e}' : \tau$ 
using assms
  by (induct set: rtranclp) (iprover intro: subject-reduction)+

```

9.4 Canonical

```

lemma canonical-forms-arrow:
  assumes T:  $\Gamma \vdash \{P\} v : \text{ty}$ 
  assumes V: is-val v
  assumes A3:  $\Gamma = []$ 
  assumes A4: ty = TArrow ty1 ty2 zs
  shows  $\exists \text{xs e}' \text{ys g.}$ 
    (v = Graph g  $\wedge$ 
      g  $\simeq$  Atom (GAbs xs ty1 e') ys  $\wedge$ 
      distinct xs  $\wedge$ 
      set xs = FLty ty1)
using assms
proof (induct arbitrary: zs set: typing)
case (TyVar i  $\Gamma$  xs ty ys P)
then show ?case
  apply auto
  done
next
case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
then show ?case
  by auto
next

```

```

  case (TyApp  $\Gamma$  P e1 ty1 ty2 xs e2)
  then show ?case
  apply auto
  done
next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  then show ?case
  apply auto
  done
next
  case (TyCong  $\Gamma$  P g1 ty g2)
  show ?case
  apply auto
  by (metis TyCong.hyps(2,3) TyCong.prem(1,2,3) cong-is-val exp.inject(1) gcong.sym gcong.trans)
next
  case (TyAlpha  $\Gamma$ 2 P g ty xs ys)
  have P1: is-val (Graph g)
    using TyAlpha.prem(1) is-graph-val-ignore-lmap is-val.simp(1) by blast
  then show ?case
  apply auto
  by (metis (no-types, lifting) FL-typing-LRHS P1 TyAlpha.hyps(1,2,3,4,5,6) TyAlpha.prem(2,3) dual-order.refl
    exp.inject(1) lmap.simp(2) lmap-graph-cong lmap-ty.simp(2) lmap-ty-inv-inv)
next
  case (TyCase  $\Gamma$  P e0 ty1  $\Gamma$ 2 e1 ty2 e2 T)
  then show ?case
  apply auto
  done
qed

```

9.5 Progress

lemma *Nus-is-val*:
 $is-graph-val\ g = is-graph-val\ (Nus1\ i\ g)$
apply (induct i arbitrary: g)
by auto

lemma *Mols1-is-val*:
 $(is-graph-val\ g \wedge (\forall g \in set\ gs.\ is-graph-val\ g)) = is-graph-val\ (Mols1\ g\ gs)$
proof (induct gs arbitrary: g)
 case Nil
 then show ?case
 apply auto
 apply (simp-all add: Mols1)
 done
next
 case (Cons a gs)
 then show ?case
 apply (simp-all add: Mols1)
 by (metis is-graph-val.simp(4))
qed

lemma *fusions-is-val*:
 $\forall fus \in set\ (fusions-of\ fs).\ is-graph-val\ fus$
apply (simp add: fusions-of-def)
by auto

lemma *app-prodrule-is-val*:
assumes A1: $\forall i < length\ Ts.\ is-val\ (Graph\ (Ts\ !\ i))$
shows $is-graph-val\ (app-prodrule\ Ts\ rhs)$
proof –
have P0: $\forall T \in set\ Ts.\ is-graph-val\ T$
by (metis assms in-set-conv-nth is-val.simp(1))

obtain i C zs fusions taus **where**

```

  prod-rhs: rhs = (i, (C, zs), fusions, taus)
by (metis prod.collapse)

have is-graph-val (Atom (GConstr C) zs)
  by simp

then have is-graph-val (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
  using Mols1-is-val fusions-is-val by blast

then have is-graph-val (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)
  using Mols1-is-val P0 by blast

then have is-graph-val (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts))
  using Nus-is-val by blast

then show ?thesis
  apply (simp add: app-prodrule-def)
  using prod-rhs by auto
qed

```

Progress: closed, well-typed terms are values or can step.

```

lemma progress:
assumes A1:  $\Gamma \vdash \{P\} e : ty$ 
assumes A2:  $\Gamma = []$ 
shows is-val  $e \vee (\exists e'. e \rightarrow \{P\} e')$ 
using assms
proof (induction arbitrary: ty e' set: typing)
  case (TyVar i  $\Gamma$  xs ty ys P)
  then show ?case
  apply auto
  done
next
  case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
  then show ?case
  apply auto
  done
next
  case (TyApp  $\Gamma$  P e1 ty1 ty2 xs e2)
  have P1: is-val e1  $\vee (\exists a. e1 \rightarrow \{P\} a)$ 
  by (simp add: TyApp.IH(1) TyApp.prem)
  have P2: is-val e2  $\vee (\exists a. e2 \rightarrow \{P\} a)$ 
  by (simp add: TyApp.IH(2) TyApp.prem)

  have Step:
    (is-val e1  $\wedge$  is-val e2)  $\vee$ 
    ( $\exists a. e1 \rightarrow \{P\} a$ )  $\vee$  ( $\exists a. e2 \rightarrow \{P\} a$ )
  using P1 P2 by blast

  from Step show ?case
proof (elim disjE)
  assume VV: is-val e1  $\wedge$  is-val e2
  then have V1: is-val e1 and V2: is-val e2 by auto

  from V1 obtain g1 where E1: e1 = Graph g1
  by (cases e1) auto
  from V2 obtain g2 where E2: e2 = Graph g2
  by (cases e2) auto

  have P3C:
     $\exists xs e' ys. g1 \simeq \text{Atom } (GAbs xs ty1 e') ys \wedge \text{set } xs = FLty ty1 \wedge \text{distinct } xs$ 
  by (metis E1 TyApp.hyps(1) TyApp.prem V1 exp.inject(1) canonical-forms-arrow)

  then show ?case
  by (metis E1 E2 FL-typing-LRHS TyApp.hyps(2) V2 beta-v-ty is-val.simps(1))
next
  assume S: ( $\exists a. e1 \rightarrow \{P\} a$ )
  then show ?case

```

```

  by force
next
  assume S: (∃ a. e2 ->{P} a)
  then show ?case
  using P1 by blast
qed
next
  case (TyProd a xs rhs P taus Ts Γ)
  have P1: ∀ i < length Ts. is-val (Graph (Ts ! i))
    by (meson TyProd.IH TyProd.premis cbv-ty-GraphE)
  then show ?case
  apply auto
  by (simp add: app-prodrule-is-val)
next
  case (TyCong Γ P e1 ty e2)
  then show ?case
  by (metis cbv-ty-GraphE cong-is-val)
next
  case (TyAlpha Γ2 P g ty xs ys)
  then show ?case
  by (metis cbv-ty-GraphE is-graph-val-ignore-lmap is-val.simps(1))
next
  case (TyCase Γ P e0 ty1 Γ2 e1 ty2 e2 T)
  then show ?case
  apply auto
  apply (metis cbv-ty.intros(3) exp.exhaust is-val.simps(1,2,3))
  by (metis cbv-ty.intros(3) exp.exhaust is-val.simps(1,2,3))
qed

```

Progress: a simpler corollary.

```

corollary progress':
  assumes [] ⊢ {P} e : τ
  shows is-val e ∨ (∃ e'. e ->{P} e')
using assms progress by auto

```

9.6 Soundness

Type soundness theorem: If a closed term is well-typed and reduces (in zero or more steps) to some term e' , then e' is either a value or can take a further step. This combines subject reduction and progress to establish the standard type soundness property of the language.

```

theorem type-soundness:
  assumes WT: [] ⊢ {P} e : τ
  and STEPS: e ->{P}* e'
  shows is-val e' ∨ (∃ u. e' ->{P} u)
proof -
  have [] ⊢ {P} e' : τ using STEPS WT subject-reduction' by blast
  thus ?thesis using progress' by blast
qed

```

end

10 Introduction

This is a proof for the soundness of the type system of the λ_{GT} language.

```

theory LambdaGT-LI

```

```

imports

```

```

  Main

```

```

  HOL-Library.Multiset

```

```

  HOL-Library.FSet

```

```

begin

```

```

declare [[syntax-ambiguity-warning = false]]

```

```

declare [[smt-oracle = false]]

```

11 Syntax, Semantics and Type System of the Language

11.1 Syntax of the Language

type-synonym *link* = *nat*
type-synonym *links* = *link list*

We use natural numbers as link identifiers. A value of type *link* represents a single link name, and a value of type *links* represents a finite list of such names. Links are handled using de Bruijn indices: the number 0 refers to the innermost bound link, 1 to the next one, and so on.

datatype 'ty graph =
 Zero
 | Atom 'ty p links
 | Fusion link link
 | Mol 'ty graph 'ty graph
 | Nu 'ty graph
and 'ty exp =
 Graph 'ty graph
 | App 'ty exp 'ty exp (**infixl** <> 200)
 | Case 'ty exp (links * 'ty) list 'ty graph 'ty exp 'ty exp
and 'ty p =
 GConstr string
 | GVar nat
 | GAbs links 'ty 'ty exp

The syntax of HyperLMNtal graphs, expressions, and atomic patterns is given by mutually recursive datatypes.

Free links of graphs. The constructor Nu binds the link index 0 in its subgraph, and all other free links are shifted down by one.

primrec FL :: 'ty graph => link set
where
 FL Zero = {}
 | FL (Atom p links) = set links
 | FL (Fusion x y) = {x, y}
 | FL (Mol g1 g2) = FL g1 ∪ FL g2
 | FL (Nu g) = {n. Suc n ∈ FL g}

lemma shift-down-alt:
 $\{n. \text{Suc } n \in S\} = (\lambda x. x - 1) \text{ ` } (S - \{0\})$
apply auto
by (metis Zero-not-Suc diff-Suc-1' imageI insert-Diff-single insert-iff)

11.2 Link Substitution

Link Substitution Over Graphs

Free link substitution. Note that this is design not to change any local links.

primrec
 lmap :: (link => link) => 'ty graph => 'ty graph
where
 lmap f Zero = Zero
 | lmap f (Atom p xs) = Atom p (map f xs)
 | lmap f (Fusion x y) = Fusion (f x) (f y)
 | lmap f (Mol g1 g2) = Mol (lmap f g1) (lmap f g2)
 | lmap f (Nu g) = Nu (lmap ($\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f (x - 1) + 1)$) g)

lemma succ-after-before:
 $\{n. \text{Suc } n \in (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f (x - 1) + 1)) \text{ ` } S\} =$
 $f \text{ ` } \{n. \text{Suc } n \in S\}$
apply auto
by force

Simultaneous Link Substitution

Definition of Simultaneous Link Substitution.

```

fun lsubst :: (link * link) list => link => link where
  lsubst [] x = x |
  lsubst ((v, n) # s) x = (if x = v then n else lsubst s x)

```

lemma lsubst-simp:

```

  lsubst (vn # s) x = (if x = fst vn then snd vn else lsubst s x)
by (metis lsubst.simps(2) prod.collapse)

```

11.3 Congruence Relation

We first define the congruence relation over graphs and prove some properties of the relation.

Inductive definition of graph equivalence.

```

inductive gcong :: 'ty graph => 'ty graph => bool (infix  $\simeq$  50)
where

```

```

  refl [intro!]: g  $\simeq$  g
| sym [intro]: g1  $\simeq$  g2 ==> g2  $\simeq$  g1
| trans[trans]: g1  $\simeq$  g2 ==> g2  $\simeq$  g3 ==> g1  $\simeq$  g3

```

```

| mol-zero [simp]: Mol g Zero  $\simeq$  g
| mol-comm [simp]: Mol g1 g2  $\simeq$  Mol g2 g1
| mol-assoc[simp]: Mol (Mol a b) c  $\simeq$  Mol a (Mol b c)

```

```

| mol-cong[intro]: g1  $\simeq$  g1' ==> g2  $\simeq$  g2' ==> Mol g1 g2  $\simeq$  Mol g1' g2'
| nu-cong [intro]: g  $\simeq$  g' ==> Nu g  $\simeq$  Nu g'

```

```

| nu-subst-fusion1:
  y ∈ FL g ==> Nu (Mol (Fusion 0 y) g)  $\simeq$  Nu (lmap (λx. if x = 0 then y else x) g)
| nu-subst-fusion2:
  0 ∈ FL g ==> Nu (Mol (Fusion 0 y) g)  $\simeq$  Nu (lmap (λx. if x = 0 then y else x) g)

```

```

| nu-nu-fusion1 [simp]: Nu (Nu (Fusion 0 1))  $\simeq$  Zero
| nu-nu-fusion2 [simp]: Nu (Fusion 0 0)  $\simeq$  Zero

```

```

| nu-zero [simp]: Nu Zero  $\simeq$  Zero

```

```

| nu-comm: Nu (Nu g)  $\simeq$  Nu (Nu (lmap (λx. if x = 0 then 1 else if x = 1 then x else x) g))

```

```

| nu-scope: Mol (Nu g1) g2  $\simeq$  Nu (Mol g1 (lmap Suc g2))

```

Some helper functions for building a graph.

definition Mols1:

```

  Mols1 g gs = foldl Mol g gs

```

primrec Nus1 :: nat => 'ty graph => 'ty graph **where**

```

  Nus1 0 g = g
| Nus1 (Suc n) g = Nu (Nus1 n g)

```

11.4 Variable Lift and Graph Substitution

The lifting operation $\text{lift } t \ k$ shifts free de Bruijn indices stored in $GVar$ that are greater than by 1. When we go under an $GAbs$ binder, we increase the cutoff to $k + 1$. Graph-level binders Nu bind links, not expression variables, so they do not affect the cutoff for lifting expression variables.

```

primrec lift :: nat => 'ty exp => 'ty exp
and lift-atom :: nat => 'ty p => 'ty p
and lift-graph :: nat => 'ty graph => 'ty graph

```

where

```

lift k (Graph g) = Graph (lift-graph k g)
| lift k (App e1 e2) = App (lift k e1) (lift k e2)
| lift k (Case e  $\Gamma$  T e1 e2) =
  Case (lift k e)  $\Gamma$  T (lift (k + length  $\Gamma$ ) e1) (lift k e2)

```

```

| lift-atom k (GConstr s) = GConstr s
| lift-atom k (GVar i) =
  (if i < k then GVar i else GVar (i + 1))
| lift-atom k (GAbs  $\tau$  xs e) =
  GAbs  $\tau$  xs (lift (k + 1) e)

```

```

| lift-graph k Zero = Zero
| lift-graph k (Atom a vs) = Atom (lift-atom k a) vs
| lift-graph k (Fusion x y) = Fusion x y
| lift-graph k (Mol g1 g2) =
  Mol (lift-graph k g1) (lift-graph k g2)
| lift-graph k (Nu g) =
  Nu (lift-graph k g)

```

Capture-avoiding substitution $e' [e / i]$ on expressions, atoms, and graphs. This is a structural skeleton; the behavior on variable occurrences (GVar) must be decided to make this a real substitution.

```

primrec subst :: nat => links => 'ty graph => 'ty exp => 'ty exp
and subst-graph :: nat => links => 'ty graph => 'ty graph => 'ty graph
and subst-p :: nat => links => 'ty graph => 'ty p => links => 'ty graph
where

```

```

  subst i xs G (Graph g) = Graph (subst-graph i xs G g)
| subst i xs G (App e1 e2) = App (subst i xs G e1) (subst i xs G e2)
| subst i xs G (Case e0  $\Gamma$  T e1 e2) =
  Case (subst i xs G e0)  $\Gamma$  T
    (subst (i + length  $\Gamma$ ) xs ((lift-graph 0  $\sim$  (length  $\Gamma$ )) G) e1)
  (subst i xs G e2)

```

```

| subst-p i xs G (GConstr s) ys = Atom (GConstr s) ys
| subst-p i xs G (GAbs zs ty e0) ys =
  Atom (GAbs zs ty (subst (i + 1) xs (lift-graph 0 G) e0)) ys
| subst-p i xs G (GVar j) ys =
  (if i < j then (Atom (GVar (j - 1)) ys)
   else if j = i  $\wedge$  length xs = length ys then lmap (lsubst (zip xs ys)) G
   else (Atom (GVar j) ys))

```

```

| subst-graph i xs G Zero = Zero
| subst-graph i xs G (Atom p ys) = subst-p i xs G p ys
| subst-graph i xs G (Fusion x y) = Fusion x y
| subst-graph i xs G (Mol g1 g2) = Mol (subst-graph i xs G g1) (subst-graph i xs G g2)
| subst-graph i xs G (Nu g) = Nu (subst-graph i xs G g)

```

definition subst-graph:

```

subst-graph XsGs T = fold ( $\lambda ((xs, -), g). \text{subst-graph } 0 \text{ xs } g$ ) XsGs T

```

definition subst:

```

subst XsGs e = fold ( $\lambda ((xs, -), g). \text{subst } 0 \text{ xs } g$ ) XsGs e

```

declare if-not-P [simp] not-less-eq [simp]
— don't add r-into-rtranc1[intro!]

11.5 Value

Values are exactly abstractions and base-type literals.

```
primrec is-atom-val :: 'ty p => bool where
  is-atom-val (GConstr s) = True
| is-atom-val (GAbs xs ty e) = True
| is-atom-val (GVar i) = False
```

Values are exactly abstractions and base-type literals.

```
primrec is-graph-val :: 'ty graph => bool where
  is-graph-val Zero = True
| is-graph-val (Atom p xs) = is-atom-val p
| is-graph-val (Fusion x y) = True
| is-graph-val (Mol g1 g2) = (is-graph-val g1 ∧ is-graph-val g2)
| is-graph-val (Nu g) = is-graph-val g
```

```
lemma is-graph-val-ignore-lmap:
  is-graph-val (lmap f g) = is-graph-val g
apply (induct g arbitrary: f rule: graph.induct[of - %-. True %-. True])
by auto
```

```
lemma cong-is-graph-val:
  assumes g1 ≃ g2
  shows is-graph-val g1 = is-graph-val g2
using assms
apply (induct set: gcong)
apply (simp-all add: is-graph-val-ignore-lmap)
by auto
```

Values are exactly abstractions and base-type literals.

```
primrec is-val :: 'ty exp => bool where
  is-val (Graph g) = is-graph-val g
| is-val (App e1 e2) = False
| is-val (Case e0 Γ T e1 e2) = False
```

```
lemma cong-is-val:
  assumes g1 ≃ g2
  shows is-val (Graph g1) = is-val (Graph g2)
using assms
by (simp add: cong-is-graph-val)
```

11.6 Type System

We first define environments and then define typing relations.

Environments

Environments are represented as lists, where the head corresponds to the most recently inserted binding (de Bruijn-style contexts).

```
type-synonym ('a) env = 'a list
```

insert-at xs j x inserts element x at position j in xs.

```
fun insert-at :: 'a env => nat => 'a => 'a env where
  insert-at xs 0 x = x # xs
| insert-at [] (Suc n) x = [x]
| insert-at (y # ys) (Suc n) x = y # insert-at ys n x
```

Types and Typing Rules

Simple types:

- *TBase* n xs : base type identified by n with free links xs

- $T\text{Arrow } ty1 \ ty2 \ xs$: function type $ty1 \Rightarrow ty2$ with free links xs .
- $TLI \ tys \ ty \ xs$: linear implication type with hided links xs .

```
datatype type =
  TBase nat links
| TArrow type type links
| TLI (type multiset) nat links link fset
```

Typing environments map de Bruijn indices to (links, type) pairs. The head is the most recently bound variable. Normally: $\text{var} \rightarrow \text{var}$ This: $\text{var}, \text{links} \rightarrow \text{var}, \text{links}$. Thus: $\text{links} \rightarrow \text{type}$.

```
type-synonym tyenv = (links * type) list
```

Right-hand side of a production rule.

```
type-synonym prodrule-RHS =
  nat * (string * links) * (link * link) list * type list
```

Whole production rule: left-hand side tag / links and RHS.

```
type-synonym prodrule =
  (nat * links) * prodrule-RHS
```

A set of the free links of a type.

```
primrec FLty :: type => link set where
  FLty (TBase i xs) = set xs
| FLty (TArrow ty1 ty2 xs) = set xs
| FLty (TLI tys a xs ys) = fset ys
```

A set of the all the free links of a type.

```
primrec ALty :: type => link set where
  ALty (TBase i xs) = set xs
| ALty (TArrow ty1 ty2 xs) = set xs
| ALty (TLI tys a xs ys) =  $\bigcup$  (set-mset (image-mset ALty tys))  $\cup$  set xs  $\cup$  fset ys
```

```
lemma FLty-subset-ALty:
  FLty ty  $\subseteq$  ALty ty
apply (induct ty) by auto
```

```
lemma finite-FLty:
  finite (FLty ty)
apply (induct ty) by auto
```

```
lemma finite-ALty:
  finite (ALty ty)
apply (induct ty) by auto
```

```
definition FLfusion-set :: (link * link) list => link set where
  FLfusion-set fusions ==  $\bigcup$  (x,y)  $\in$  set fusions. {x, y}
```

```
definition FLfusion-fset :: (link * link) list => link fset where
  FLfusion-fset fusions == Abs-fset ( $\bigcup$  (x,y)  $\in$  set fusions. {x, y})
```

```
lemma finite-FLfusion-set:
  finite (FLfusion-set fusions)
apply (simp add: FLfusion-set-def)
by fastforce
```

definition $FLfusion :: (link * link) list \Rightarrow links$ **where**

$FLfusion\ fusions ==$
 $foldr (\lambda(x,y) acc. [x,y] @ acc) fusions []$

lemma $fusions-freelinks-equiv2B$:

$set (FLfusion\ fusions) = FLfusion-set\ fusions$

proof ($induction\ fusions$)

case Nil

then show $?case$

by ($simp\ add: FLfusion-def\ FLfusion-set-def$)

next

case ($Cons\ a\ fusions$)

obtain $x\ y$ **where** $a-def: a = (x,y)$

by ($cases\ a$)

have $rhs-step$:

$FLfusion-set\ (a \# fusions) = \{x,y\} \cup FLfusion-set\ fusions$

unfolding $FLfusion-set-def\ a-def$

by $auto$

from $Cons.IH$

have $set\ (foldr\ (\lambda(x,y) acc. [x,y] @ acc) fusions []) = FLfusion-set\ fusions$

unfolding $FLfusion-def$ **by** $simp$

with $step\ rhs-step$

show $?case$

unfolding $FLfusion-def$

using $a-def$ **by** $auto$

qed

Free links of RHS on production rules: we shift by i to account for the $Nus1\ i$ binder.

definition $FLrhs :: prodrule-RHS \Rightarrow link\ set$ **where**

$FLrhs\ rhs ==$
 $(case\ rhs\ of\ (i, (C, zs), fusions, taus) \Rightarrow$
 $\{n. n + i \in set\ zs \cup FLfusion-set\ fusions \cup (\bigcup (FLty\ 'set\ taus)))\})$

Free-link substitution on types.

primrec $lmap-ty :: (link \Rightarrow link) \Rightarrow type \Rightarrow type$ **where**

$lmap-ty\ f\ (TBase\ i\ zs) =$

$TBase\ i\ (map\ f\ zs)$

$| lmap-ty\ f\ (TArrow\ ty1\ ty2\ zs) =$

$TArrow\ ty1\ ty2\ (map\ f\ zs)$

$| lmap-ty\ f\ (TLI\ tys\ a\ xs\ ys) =$

$TLI\ (image-mset\ (lmap-ty\ f)\ tys)\ a\ (map\ f\ xs)\ (f\ |\cdot| ys)$

Free-link substitution on types. Note: this substitutes only free links.

definition $lsubst-ty :: links \Rightarrow links \Rightarrow type \Rightarrow type$ **where**

$lsubst-ty\ xs\ ys\ ty = lmap-ty\ (lsubst\ (zip\ xs\ ys))\ ty$

Turn a list of link pairs (x, y) into a list of fusion graphs $Fusion\ x\ y$.

definition $fusions-of :: (link * link) list \Rightarrow type\ graph\ list$ **where**

$fusions-of\ fusions = map\ (\lambda(x, y). Fusion\ x\ y)\ fusions$

definition $taus-of-rhs :: prodrule-RHS \Rightarrow type\ list$ **where**

$taus-of-rhs\ rhs =$

$(case\ rhs\ of\ (i, (C, zs), fusions, taus) \Rightarrow taus)$

Apply a production rule to argument graphs Ts .

definition $app-prodrule ::$

$type\ graph\ list \Rightarrow prodrule-RHS \Rightarrow type\ graph$ **where**

$app-prodrule\ Ts\ rhs =$

$(case\ rhs\ of\ (i, (C, zs), fusions, taus) \Rightarrow$

$Nus1\ i\ (Mols1\ (Mols1\ (Atom\ (GConstr\ C)\ zs)\ (fusions-of\ fusions))\ Ts))$

Make linear implication type when the application of TyLIIntro.

definition *make-diffgraph* ::
 $prodrule-RHS \Rightarrow type\ graph$ **where**
 $make-diffgraph\ rhs =$
 $(case\ rhs\ of\ (i,\ (C,\ zs),\ fusions,\ taus) \Rightarrow$
 $Mols1\ (Atom\ (GConstr\ C)\ zs)\ (fusions-of\ fusions))$

Make graph when application of TyLIIntro.

definition *make-LI* ::
 $prodrule \Rightarrow type$ **where**
 $make-LI\ prodrule =$
 $(case\ prodrule\ of\ ((a,\ xs),\ (i,\ (C,\ zs),\ fusions,\ taus)) \Rightarrow$
 $TLI\ (mset\ taus)\ a\ xs\ (fset-of-list\ zs\ |\cup|\ FLfusion-fset\ fusions))$

lemma *FL-Mols1-decomp*:
 $FL\ (Mols1\ a\ bs) = FL\ a \cup \bigcup (FL\ 'set\ bs)$

proof (*induct bs arbitrary: a*)

case *Nil*
then show *?case*
apply (*simp-all add: Mols1*)
done

next

case (*Cons a bs*)
then show *?case*
apply (*simp-all add: Mols1*)
apply *auto*
done

qed

lemma *FLfusion-set-equiv*:
 $FLfusion-set\ fusions = \bigcup (FL\ 'set\ (fusions-of\ fusions))$

proof (*induct fusions*)

case *Nil*
then show *?case*
apply *auto*
apply (*simp-all add: Mols1 FLfusion-set-def fusions-of-def*)
done

next

case (*Cons a fusions*)
then show *?case*
apply *auto*
apply (*simp-all add: Mols1 FLfusion-set-def fusions-of-def*)
apply (*simp add: case-prod-beta*)
by *fastforce*

qed

lemma *FLfusion-fset-set-equiv*:
 $fset\ (FLfusion-fset\ fusions) = FLfusion-set\ fusions$
apply (*simp-all add: FLfusion-fset-def FLfusion-set-def*)

proof (*induct fusions*)

case *Nil*
then show *?case*
apply *auto*
using *bot-fset.abs-eq* **by** *force*

next

case (*Cons a fusions*)
then show *?case*
by (*metis FLfusion-set-def fset-inverse fset-of-list.rep-eq*
 $fusions-freelinks-equiv2B$)

qed

```

lemma FLfusion-fset-equiv:
  fset (FLfusion-fset fusions) =  $\bigcup$  (FL ' set (fusions-of fusions))
apply (simp-all add: FLfusion-fset-def fusions-of-def)
proof (induct fusions)
  case Nil
  then show ?case
  apply auto
  using bot-fset-def by force
next
  case (Cons a fusions)
  then show ?case
  by (metis (mono-tags, lifting) FL.simps(3) FLfusion-fset-def
    FLfusion-fset-set-equiv FLfusion-set-def Sup.SUP-cong
    case-prod-beta)
qed

```

```

lemma TyLIIntro-freelinks-equiv:
  assumes rhs = (i, (C, zs), fusions, taus)
  assumes prodrule = ((a, xs), rhs)
  shows FLty (make-LI prodrule) = FL (make-diffgraph rhs)
apply (simp-all add: make-diffgraph-def make-LI-def FLty-def)
apply (simp-all add: FL-Mols1-decomp assms)
using FLfusion-fset-equiv
by (simp add: fset-of-list.rep-eq)

```

Bind free link in a graph.

```

definition lbind :: link => 'ty graph => 'ty graph where
  lbind x g = Nu ((lmap ( $\lambda y$ . if  $y = \text{Suc } x$  then 0 else  $y$ )) (lmap Suc g))

```

Bind free links in a graph.

```

definition lbinds :: links => 'ty graph => 'ty graph where
  lbinds xs g = fold lbind xs g

```

Typing rules.

```

inductive typing ::
  tyenv => prodrule list => type exp => type => bool
  (<-  $\vdash$  {-} - :  $\rightarrow$  [50, 50, 50, 50] 50)
where
  TyVar:
    i < length  $\Gamma$  ==>
    xs = map fst s ==>
    ys = map snd s ==>
     $\Gamma ! i = (xs, ty) ==>$ 
    distinct xs ==>
    distinct ys ==>
    set xs = FLty ty ==>
    set ys  $\cap$  ALty ty - FLty ty = {} ==>
     $\Gamma \vdash \{P\}$  Graph (Atom (GVar  $i$ ) ys) : lmap-ty (lsubst s) ty
  | TyArrow:
    (xs, ty1) #  $\Gamma \vdash \{P\}$  e : ty2 ==>
    distinct xs ==>
    set xs = FLty ty1 ==>
     $\Gamma \vdash \{P\}$  Graph (Atom (GAbs xs ty1 e) ys) : TArrow ty1 ty2 ys
  | TyApp:
     $\Gamma \vdash \{P\}$  e1 : TArrow ty1 ty2 xs ==>
     $\Gamma \vdash \{P\}$  e2 : ty1 ==>
     $\Gamma \vdash \{P\}$  e1  $\cdot$  e2 : ty2
  | TyProd:
    ((a, xs), rhs)  $\in$  set P ==>
    taus = taus-of-rhs rhs ==>
    length taus = length Ts ==>
    ( $\forall i < \text{length } Ts. \Gamma \vdash \{P\} (\text{Graph } (Ts ! i)) : \text{taus} ! i$ ) ==>

```

```

    set xs = FLrhs rhs ==>
     $\Gamma \vdash \{P\} \text{ (Graph (app-prodrule } Ts \text{ rhs)) : } TBase \ a \ xs$ 
| TyCong:
     $\Gamma \vdash \{P\} \text{ Graph } g1 : ty ==>$ 
     $g1 \simeq g2 ==>$ 
     $\Gamma \vdash \{P\} \text{ Graph } g2 : ty$ 
| TyAlpha:
     $\Gamma \vdash \{P\} \text{ Graph } g : ty ==>$ 
    set xs = ALty ty ==>
    length xs = length ys ==>
    distinct xs ==>
    distinct ys ==>
    FL g  $\subseteq$  set xs ==>
     $\Gamma \vdash \{P\} \text{ Graph (lmap (lsubst (zip xs ys)) g)$ 
      : lsubst-ty xs ys ty
| TyCase:
     $\Gamma \vdash \{P\} \ e0 : ty1 ==>$ 
     $\Gamma \vdash \{P\} \ e1 : ty2 ==>$ 
     $\Gamma \vdash \{P\} \ e2 : ty2 ==>$ 
     $\Gamma \vdash \{P\} \text{ Case } e0 \ \Gamma \vdash T \ e1 \ e2 : ty2$ 
| TyLIIntro:
    (lhs, rhs)  $\in$  set P ==>
     $\Gamma \vdash \{P\} \text{ Graph (make-diffgraph rhs) : make-LI (lhs, rhs)}$ 
| TyLITrans:
     $\Gamma \vdash \{P\} \text{ Graph } T1 : TLI (\{\#TBase \ a2 \ xs2\# \} + tys1) \ a1 \ xs1 \ ys1 ==>$ 
     $\Gamma \vdash \{P\} \text{ Graph } T2 : TLI \ tys2 \ a1 \ xs2 \ ys2 ==>$ 
    ys = (ys1  $\cup$  ys2) ==>
     $\Gamma \vdash \{P\} \text{ Graph (Mol } T1 \ T2) : TLI (tys1 + tys2) \ a1 \ xs2 \ ys$ 
| TyLIElim0:
    ys = fset-of-list ys' ==>
    set xs  $\subseteq$  FL T ==>
     $\Gamma \vdash \{P\} \text{ Graph } T : TLI \ 0 \ a \ xs \ ys ==>$ 
     $\Gamma \vdash \{P\} \text{ Graph (lbinds (filter } (\lambda y. y \notin \text{set xs}) \ ys') \ T)$ 
      : TBase a xs
| TyLIIntro0:
     $\Gamma \vdash \{P\} \text{ Graph } T : TBase \ a \ xs ==>$ 
     $\Gamma \vdash \{P\} \text{ Graph } T : TLI \ 0 \ a \ xs \ (\text{fset-of-list } xs)$ 

```

Inversion rules for the typing judgment. They let us analyse typing derivations for particular syntactic forms.

inductive-cases typing-GVarE [elim!]:

$\Gamma \vdash \{P\} \text{ Graph (Atom (GVar } i) \ ys) : ty$

inductive-cases typing-GAbsE [elim!]:

$\Gamma \vdash \{P\} \text{ Graph (Atom (GAbs } xs \ ty1 \ e) \ ys) : ty$

inductive-cases typing-AppE [elim!]:

$\Gamma \vdash \{P\} \ e1 \cdot e2 : ty$

inductive-cases typing-GraphE [elim!]:

$\Gamma \vdash \{P\} \text{ (Graph } g) : ty$

11.7 Reduction Relations

Call-by-value small-step relation on expressions. - Perform beta only when the argument graph is already a value. - Case on a value graph that matches the pattern graph. - Case on a non-graph value (falls through to the else branch). - Evaluate the function part first (left-to-right). - Then evaluate the argument, once the function is a value. No reduction happens inside abstractions.

inductive cbv-ty :: type exp => prodrule list => type exp => bool

(- ->{-} - [60,0,61] 60)

where

beta-v-ty [simp, intro]:

is-graph-val v ==> distinct xs ==> FL v = set xs ==>

lam $\simeq$ Atom (GAbs xs ty e) ys ==>

Graph lam $\cdot$ Graph v ->{P} subst 0 xs v e

| case1-v-ty [simp, intro]:

is-graph-val v ==>

```

length  $\Gamma$  = length  $gs$  ==>
XsGs = zip  $\Gamma$   $gs$  ==>
 $v \simeq \text{substs-graph } XsGs \ T$  ==>
 $\forall ((xs, ty), g) \in \text{set } XsGs. ($ 
  distinct  $xs \wedge FL \ g = \text{set } xs \wedge$ 
   $\llbracket \vdash \{P\} (\text{Graph } g) : ty$ 
 $) ==>$ 
Case (Graph  $v$ )  $\Gamma \ T \ e1 \ e2 \rightarrow \{P\} \text{substs } XsGs \ e1$ 
| case2-v-ty [simp, intro]:
  is-graph-val  $v$  ==>
  Case (Graph  $v$ )  $\Gamma \ T \ e1 \ e2 \rightarrow \{P\} \ e2$ 
| appL-ty [simp, intro]:
   $e1 \rightarrow \{P\} \ e1' ==>$ 
   $e1 \cdot e2 \rightarrow \{P\} \ e1' \cdot e2$ 
| appR-ty [simp, intro]:
  is-val  $v$  ==>
   $e \rightarrow \{P\} \ e' ==>$ 
   $v \cdot e \rightarrow \{P\} \ v \cdot e'$ 
| caseL-ty [simp, intro]:
   $e0 \rightarrow \{P\} \ e0' ==>$ 
  Case  $e0 \ \Gamma \ T \ e1 \ e2 \rightarrow \{P\} \text{Case } e0' \ \Gamma \ T \ e1 \ e2$ 

```

Inversion rules for the call-by-value step relation.

```

inductive-cases cbv-ty-AppE [elim!]:
   $e1 \cdot e2 \rightarrow \{P\} \ e$ 

```

```

inductive-cases cbv-ty-CaseE [elim!]:
  Case  $e0 \ \Gamma \ T \ e1 \ e2 \rightarrow \{P\} \ e$ 

```

```

inductive-cases cbv-ty-GraphE [elim!]:
  Graph  $g \rightarrow \{P\} \ e$ 

```

Zero or more reductions.

```

abbreviation cbv-ty-star ::
  type exp => prodrule list => type exp => bool
  (- -> {-})* - [60,0,61] 60)
where
   $e \rightarrow \{P\}^* \ e' \equiv \text{rtranclp } (\lambda e \ e'. \text{cbv-ty } e \ P \ e') \ e \ e'$ 

```

12 Congruence Relations

Show it's an equivalence relation (useful for rewriting/quotients).

```

lemma gcong-reflp: reflp ( $\simeq$ )
  by (simp add: gcong.refl reflpI)

```

```

lemma gcong-symp: symp ( $\simeq$ )
  by (simp add: gcong.sym sympI)

```

```

lemma gcong-transp: transp ( $\simeq$ )
  using gcong.trans transpI by blast

```

```

lemma gcong-equivp: equivp ( $\simeq$ )
  by (simp add: gcong-reflp gcong-symp gcong-transp equivpI)

```

13 Properties of Link Substitution

13.1 Basic Properties of Link Substitution

```

lemma FL-lmap-commute:
  fixes  $g :: 'ty \text{graph}$ 
  shows  $FL \ (\text{lmap } f \ g) = f \ ' \ (FL \ g)$ 
proof (induct  $g$  arbitrary:  $f$ )
  case (Nu  $g$ )
  have  $FL \ (\text{lmap } f \ (\text{Nu } g)) = FL \ (\text{Nu } (\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \ (x - 1) + 1)) \ g))$ 
  by simp

```

```

also have ... = {n. Suc n ∈ FL (lmap ( $\lambda x$ . if x = 0 then x else (f (x - 1) + 1)) g)}
by simp
also have ... = {n. Suc n ∈ ( $\lambda x$ . if x = 0 then x else (f (x - 1) + 1)) ‘ FL g}
using Nu by blast
also have ... = f ‘ {n. Suc n ∈ FL g}
using succ-after-before by presburger
also have ... = f ‘ FL (Nu g)
by simp
finally show ?case .
qed auto

```

```

lemma FL-lmap-deSuc-Suc:
  fixes k n :: nat
  and g :: 'ty graph
  shows {n. Suc n ∈ FL (lmap Suc g)} = FL g
apply (simp add: FL-lmap-commute)
by auto

```

```

lemma FL-Nu-lmap-Suc:
  FL g = FL (Nu (lmap Suc g))
by (simp add: FL-lmap-deSuc-Suc)

```

```

lemma lbind-hide:
  FL (lbind x g) = FL g - {x}
apply (simp add: lbind-def)
apply auto
apply (simp-all add: FL-lmap-commute image-iff)
done

```

13.2 Basic Properties of Simultaneous Link Substitution

```

lemma lsubst-cdom:
  assumes x ∈ set (map fst xs)
  shows lsubst xs x ∈ set (map snd xs)
using assms
proof (induct xs)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  apply (cases x = fst a)
  using lsubst-simp
  by auto
qed

```

```

lemma lsubst-image-aux:
  assumes distinct (map fst s)
  shows map (lsubst s) (map fst s) = map snd s
using assms
proof (induct s)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a s)
  then show ?case
  apply auto
  using lsubst-simp apply presburger
  using lsubst-simp apply auto

```

```

  by (metis fstI image-eqI)
qed

```

```

lemma lsubst-image:
  assumes distinct (map fst s)
  shows lsubst s ' set (map fst s) = set (map snd s)
by (metis assms list.set-map lsubst-image-aux)

```

```

lemma lsubst-zip-image:
  assumes length xs = length ys
  assumes distinct xs
  shows lsubst (zip xs ys) ' set xs = set ys
using assms
proof (induct xs arbitrary: ys)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  by (metis lsubst-image map-fst-zip map-snd-zip)
qed

```

13.3 Link Substitution Before/After Inverse Link Substitution

Swapping links.

```

abbreviation swap-list :: ('a * 'b) list => ('b * 'a) list
  where swap-list xs ≡ map (λ(x, y). (y, x)) xs

```

```

lemma swap-swap-list [simp]:
  swap-list (swap-list xs) = xs
apply (induct xs)
by auto

```

```

lemma swap-map-fst [simp]:
  map fst (swap-list xs) = map snd xs
apply (induct xs)
by auto

```

```

lemma swap-map-snd [simp]:
  map snd (swap-list xs) = map fst xs
apply (induct xs)
by auto

```

```

lemma lsubst-inv-cdom:
  assumes y ∈ set (map snd xs)
  shows lsubst (swap-list xs) y ∈ set (map fst xs)
by (metis assms lsubst-cdom swap-map-fst swap-map-snd)

```

```

lemma lsubst-inv-inv:
  assumes distinct (map fst xs)
  assumes y ∈ set (map snd xs)
  shows lsubst xs (lsbst (swap-list xs) y) = y
using assms
proof (induct xs arbitrary: y)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)

```

```

obtain  $x\ y'$  where
  decomp:  $(x, y') = a$ 
by (metis surj-pair)

then show ?case
proof (cases  $y = y'$ )
  case True
    have  $\text{lsbst } ((x, y') \# xs) (\text{lsbst } (\text{swap-list } ((x, y') \# xs))\ y) = y$ 
      by (simp add: True)
    then show ?thesis
      using decomp by fastforce
  next
    case False
    have  $P0: y \in \text{set } (\text{map snd } xs)$ 
      using Cons.prems(2) False decomp by fastforce
    then have  $\text{lsbst } (\text{swap-list } xs)\ y \in \text{set } (\text{map fst } xs)$ 
      using lsbst-inv-cdom by presburger
    then have  $P3: \text{lsbst } (\text{swap-list } xs)\ y \neq x$ 
      using Cons.prems(1) decomp by force

    have  $\text{lsbst } (a \# xs) (\text{lsbst } (\text{swap-list } (a \# xs))\ y) =$ 
       $\text{lsbst } ((x, y') \# xs) (\text{lsbst } (\text{swap-list } ((x, y') \# xs))\ y)$ 
      using decomp by simp
    also have  $\dots = \text{lsbst } ((x, y') \# xs) (\text{lsbst } ((y', x) \# \text{swap-list } xs)\ y)$ 
      by simp
    also have  $\dots = \text{lsbst } ((x, y') \# xs) (\text{lsbst } (\text{swap-list } xs)\ y)$ 
      by (simp add: False)
    also have  $\dots = \text{lsbst } xs (\text{lsbst } (\text{swap-list } xs)\ y)$ 
      by (simp add: P3)
    also have  $\dots = y$ 
      using Cons.hyps Cons.prems(1) P0 by auto
    finally show ?thesis .
qed
qed

```

```

lemma lsbst-ignore:
  assumes  $y \notin \text{set } (\text{map fst } xs)$ 
  shows  $\text{lsbst } xs\ y = y$ 
using assms
proof (induct  $xs$ )
  case Nil
  then show ?case
    by simp
next
  case (Cons  $a\ xs$ )
  then show ?case
    by (simp add: lsbst-simp)
qed

```

```

lemma lsbst-inv-invB:
  assumes  $y \notin \text{set } (\text{map fst } xs)$ 
  assumes  $y \notin \text{set } (\text{map snd } xs)$ 
  shows  $\text{lsbst } (\text{swap-list } xs) (\text{lsbst } xs\ y) = y$ 
proof -
  have  $\text{lsbst } (\text{swap-list } xs) (\text{lsbst } xs\ y) = \text{lsbst } (\text{swap-list } xs)\ y$ 
    using assms(1) lsbst-ignore by presburger
  also have  $\dots = y$ 
    proof -
      have  $y \notin \text{set } (\text{map fst } (\text{swap-list } xs))$ 
        by (metis assms(2) swap-map-fst)
      then show ?thesis
        using lsbst-ignore by presburger
    qed
  finally show ?thesis .
qed

```

```

lemma lsubst-inv-invC:
  assumes distinct (map snd xs)
  assumes  $y \notin \text{set } (\text{map fst } xs) \longrightarrow y \notin \text{set } (\text{map snd } xs)$ 
  shows lsubst (swap-list xs) (lsbst xs y) = y
by (metis assms(1,2) lsubst-inv-inv lsubst-inv-invB swap-map-fst swap-swap-list)

```

```

corollary lsubst-inv-inv2:
  assumes distinct (map snd xs)
  assumes  $y \in \text{set } (\text{map fst } xs)$ 
  shows lsubst (swap-list xs) (lsbst xs y) = y
by (metis assms(1,2) lsubst-inv-inv swap-map-fst swap-swap-list)

```

```

lemma map-lsubst-inv-inv:
  assumes distinct (map fst xs)
  assumes  $\text{set } ys \subseteq \text{set } (\text{map snd } xs)$ 
  shows map (lsbst xs o lsubst (swap-list xs)) ys = ys
using assms
proof (induct ys)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a ys)
  then show ?case
  apply auto
  using Cons.premis(2) lsubst-inv-inv by auto
qed

```

```

corollary map-lsubst-zip-inv-inv:
  assumes distinct xs
  assumes  $\text{set } ys' \subseteq \text{set } ys$ 
  assumes  $\text{length } xs = \text{length } ys$ 
  shows map (lsbst (zip xs ys) o lsubst (zip ys xs)) ys' = ys'
by (metis assms(1,2,3) map-fst-zip map-lsubst-inv-inv map-snd-zip zip-commute)

```

```

lemma lmap-lsubst-inv-inv-aux:
  assumes distinct (map fst xs)
  assumes distinct (map snd xs)
  assumes  $\forall y \in FL. g. y \geq k \longrightarrow y - k \in \text{set } (\text{map fst } xs)$ 
  shows lmap ( $\lambda x. \text{if } x < k \text{ then } x \text{ else } (\text{lsbst } (\text{swap-list } xs) (x - k) + k)$ )
    (lmap ( $\lambda x. \text{if } x < k \text{ then } x \text{ else } (\text{lsbst } xs (x - k) + k)$ ) g) = g
using assms
proof (induct g arbitrary: k rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  apply auto
  done
next
  case (Atom x1 x2)
  then show ?case
  apply auto
  by (simp add: FL.simps(2) comp-apply lsubst-inv-inv2 map-idI)
next
  case (Fusion x1 x2)
  then show ?case
  apply auto
  using lsubst-inv-inv2 apply fastforce
  using lsubst-inv-inv2 apply fastforce
  using lsubst-inv-inv2 apply fastforce
  using lsubst-inv-inv2 apply fastforce

```

```

using lsubst-inv-inv2 apply fastforce
using lsubst-inv-inv2 apply fastforce
done
next
case (Mol x1 x2)
then show ?case
apply auto
done
next
case (Nu g')
have  $\forall y \in FL (Nu g'). y \geq k \longrightarrow y - k \in set (map fst xs)$ 
using Nu.premis(3) by blast
then have P2:  $\forall y \in FL g'. y \geq Suc k \longrightarrow y - (Suc k) \in set (map fst xs)$ 
by (metis FL.simps(5) Suc-le-D Suc-le-mono diff-Suc-Suc mem-Collect-eq)

have P3:
 $(\lambda x. if x = 0 then x else (if x - 1 < k then x - 1 else lsubst (swap-list xs) (x - 1 - k) + k) + 1) =$ 
 $(\lambda x. if x < Suc k then x else lsubst (swap-list xs) (x - (Suc k)) + (Suc k))$ 
using ab-semigroup-add-class.add-ac(1) add.commute diff-diff-eq not0-implies-Suc not-less-eq
by fastforce

have P4:
 $(\lambda x. if x = 0 then x else (if x - 1 < k then x - 1 else lsubst xs (x - 1 - k) + k) + 1) =$ 
 $(\lambda x. if x < Suc k then x else lsubst xs (x - (Suc k)) + (Suc k))$ 
using add-cancel-right-right add-diff-inverse-nat diff-Suc-eq-diff-pred less-Suc-eq less-imp-diff-less
not-less-eq plus-1-eq-Suc by fastforce

have P5:
lmap  $(\lambda x. if x = 0 then x else (if x - 1 < k then x - 1 else lsubst (swap-list xs) (x - 1 - k) + k) + 1)$ 
 $(lmap (\lambda x. if x = 0 then x else (if x - 1 < k then x - 1 else lsubst xs (x - 1 - k) + k) + 1) g') =$ 
lmap  $(\lambda x. if x < Suc k then x else lsubst (swap-list xs) (x - (Suc k)) + (Suc k))$ 
 $(lmap (\lambda x. if x < Suc k then x else lsubst xs (x - (Suc k)) + (Suc k)) g')$ 
using P3 P4 by argo

show ?case
apply auto
using Nu.hyps Nu.premis(2) P2 P5 assms(1) by presburger
qed auto

corollary lmap-lsubst-inv-inv:
assumes distinct (map fst xs)
assumes distinct (map snd xs)
assumes  $FL g \subseteq set (map fst xs)$ 
shows lmap (lsbst (swap-list xs)) (lmap (lsbst xs) g) = g
proof -
have  $\forall y \in FL g. y \geq 0 \longrightarrow y - 0 \in set (map fst xs)$ 
apply auto
using assms(3) by auto
then have lmap  $(\lambda x. if x < 0 then x else (lsbst (swap-list xs) (x - 0) + 0))$ 
 $(lmap (\lambda x. if x < 0 then x else (lsbst xs (x - 0) + 0)) g) = g$ 
using assms(1,2) lmap-lsubst-inv-inv-aux by blast
then show ?thesis
by simp
qed

lemma lsubst-ignore-befores:
assumes  $xs = ys1 @ ys2$ 
assumes  $z \notin set (map fst ys1)$ 
shows lsubst xs z = lsubst ys2 z
using assms
proof (induct ys1 arbitrary: ys2 xs)

```

```

case Nil
then show ?case
apply auto
done
next
case (Cons a ys1)
then show ?case
apply auto
using lsubst-simp by presburger
qed

```

```

lemma lsubst-ignore-afters:
  assumes xs = ys1 @ ys2
  assumes z ∉ set (map fst ys2)
  shows lsubst xs z = lsubst ys1 z
using assms
proof (induct ys1 arbitrary: ys2 xs)
  case Nil
  then show ?case
  apply auto
  by (simp add: lsubst-ignore)
next
  case (Cons a ys1)
  then show ?case
  apply auto
  using lsubst-simp by presburger
qed

```

```

corollary lsubst-ignore-beforeafters:
  assumes xs = ys1 @ ys2 @ ys3
  assumes z ∉ set (map fst ys1)
  assumes z ∉ set (map fst ys3)
  shows lsubst xs z = lsubst ys2 z
using assms(1,2,3) lsubst-ignore-afters lsubst-ignore-befores by presburger

```

```

lemma lsubst-lemma1:
  assumes xs = ys1 @ [(k, v)] @ ys3
  assumes distinct (map fst xs)
  assumes z = k
  shows z ∉ set (map fst ys1)
  and z ∉ set (map fst ys3)
apply auto
using assms
apply (metis Un-iff append-Cons distinct-append distinct-map eq-key-imp-eq-value in-set-conv-decomp
  not-distinct-conv-prefix set-append)
using assms
apply (metis Un-iff append-Cons distinct-append distinct-map eq-key-imp-eq-value in-set-conv-decomp
  not-distinct-conv-prefix set-append)
done

```

```

lemma lsubst-lemma2:
  assumes z ∈ set (map fst xs)
  shows ∃ v ys1 ys3. (xs = ys1 @ [(z, v)] @ ys3)
using assms
proof (induct xs arbitrary: z)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a ys1)
  then show ?case
  apply auto
  apply (metis append-Nil eq-fst-iff)

```

by (meson in-set-conv-decomp-first list.set-intros(2))
qed

lemma *lsubst-lemma3*:
assumes *distinct* (map fst *xs*)
assumes $z \in \text{set } (\text{map fst } xs)$
shows $\exists v \text{ ys1 ys3. } (xs = \text{ys1} @ [(z, v)] @ \text{ys3})$
 $\wedge z \notin \text{set } (\text{map fst } \text{ys1})$
 $\wedge z \notin \text{set } (\text{map fst } \text{ys3})$
by (metis *assms*(1,2) *lsubst-lemma1*(1,2) *lsubst-lemma2*)

lemma *lsubst-lemma4*:
assumes *distinct* (map fst *xs*)
assumes $(k, v) \in \text{set } xs$
shows $\exists \text{ys1 ys3. } (xs = \text{ys1} @ [(k, v)] @ \text{ys3})$
 $\wedge k \notin \text{set } (\text{map fst } \text{ys1})$
 $\wedge k \notin \text{set } (\text{map fst } \text{ys3})$
by (metis *append.left-neutral append-Cons assms*(1,2) *in-set-conv-decomp-last lsubst-lemma1*(1,2))

lemma *lsubst-lemma5*:
assumes *distinct* (map fst *xs*)
assumes $(k, v) \in \text{set } xs$
shows *lsubst* *xs* *k* = *v*
by (metis *assms*(1,2) *lsubst.simps*(2) *lsubst-ignore-beforeafters lsubst-lemma4*)

lemma *lsubst-lemma6*:
assumes *distinct* (map fst *xs*)
assumes *distinct* (map fst *ys*)
assumes $k \in \text{set } (\text{map fst } xs)$
assumes $\text{set } xs = \text{set } ys$
shows *lsubst* *xs* *k* = *lsubst* *ys* *k*
using *assms*(1,2,3,4) *lsubst-lemma5* **by** *auto*

lemma *lsubst-lemma7*:
assumes *distinct* (map fst *xs*)
assumes *distinct* (map fst *ys*)
assumes $k \in \text{set } (\text{map fst } xs)$
assumes $\text{set } xs \subseteq \text{set } ys$
shows *lsubst* *xs* *k* = *lsubst* *ys* *k*
using *Un-iff assms*(1,2,3,4) *lsubst-lemma2 lsubst-lemma5 set-append* **by** *auto*

lemma *lsubst-distinct*:
assumes *distinct* (map fst *xs*)
assumes *distinct* (map fst *ys*)
assumes $\text{set } xs = \text{set } ys$
shows *lsubst* *xs* *k* = *lsubst* *ys* *k*
by (metis *assms*(1,2,3) *lsubst-ignore lsubst-lemma6*)

lemma *lmap-lsubst-distinct*:
assumes *distinct* (map fst *xs*)
assumes *distinct* (map fst *ys*)
assumes $\text{set } xs = \text{set } ys$
shows *lmap* (*lsubst* *xs*) *g* = *lmap* (*lsubst* *ys*) *g*
by (metis (no-types, lifting) *ext assms*(1,2,3) *lsubst-distinct*)

lemma *lsubst-filter*:

```

assumes  $z \in S$ 
assumes  $ys = \text{filter } (\lambda x. \text{fst } x \in S) \ xs$ 
shows  $\text{lsubst } xs \ z = \text{lsubst } ys \ z$ 
using assms
proof (induct xs arbitrary: ys S)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  apply (simp-all add: lsubst-simp)
  by auto
qed

```

```

lemma map-lsubst-filter:
  assumes  $\text{set } zs \subseteq S$ 
  assumes  $ys = \text{filter } (\lambda x. \text{fst } x \in S) \ xs$ 
  shows  $\text{map } (\text{lsubst } xs) \ zs = \text{map } (\text{lsubst } ys) \ zs$ 
using assms
proof (induct xs arbitrary: ys S)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  apply (simp-all add: lsubst-simp)
  by auto
qed

```

```

lemma fmap-lsubst-filter:
  assumes  $\text{fset } zs \subseteq S$ 
  assumes  $ys = \text{filter } (\lambda x. \text{fst } x \in S) \ xs$ 
  shows  $(\text{lsubst } xs) \ |\cdot| \ zs = (\text{lsubst } ys) \ |\cdot| \ zs$ 
proof –
  have  $\forall z \ |\cdot| \ zs. \ z \in S$ 
  using assms(1) by blast

  then have  $\forall z \ |\cdot| \ zs. (\text{lsubst } xs \ z = \text{lsubst } ys \ z)$ 
  using assms(2) lsubst-filter by blast

  then show ?thesis
  by auto
qed

```

13.4 Basic Property of Link Substitution Over Types

Free links of a left-folded *Mol*.

```

lemma FL-Mols1:
   $FL \ (Mols1 \ g \ gs) = FL \ g \cup \bigcup (FL \ \cdot \ \text{set } gs)$ 
apply (induct gs arbitrary: g)
apply (simp-all add: Mols1)
by auto

```

Free links after *i* nested *Nu* binders.

```

lemma FL-Nus1:
   $FL \ (Nus1 \ i \ g) = \{n. \ n + i \in FL \ g\}$ 
by (induct i arbitrary: g) auto

```

```

lemma app-prodrule-FL-equiv:
  assumes A1:  $\text{taus} = \text{taus-of-rhs } rhs$ 

```

```

assumes A3: map FLty taus = map FL Ts
shows FL (app-prodrule Ts rhs) = FLrhs rhs
proof –
obtain i C zs fusions taus' where
  prod-rhs: rhs = (i, (C, zs), fusions, taus')
by (metis prod.collapse)

have taus: taus' = taus
by (simp add: A1 prod-rhs taus-of-rhs-def)
then have prod-rhs2: rhs = (i, (C, zs), fusions, taus)
by (simp add: prod-rhs)

have FL-fusions:
   $\bigcup (FL \text{ ' set (fusions-of fusions)}) = (\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
apply (simp add: fusions-of-def)
by (simp add: case-prod-beta)

have
  FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    set zs  $\cup$   $\bigcup (FL \text{ ' set (fusions-of fusions)})$ 
by (simp add: FL-Mols1)
also have
  ... = set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
by (simp add: FL-fusions)
finally have P11:
  FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
by simp

have  $\bigcup (FL \text{ ' set Ts}) = \bigcup (\text{set (map FL Ts)})$ 
by simp
also have ... =  $\bigcup (\text{set (map FLty taus)})$ 
using A3 by presburger
also have ... =  $\bigcup (FLty \text{ ' set taus})$ 
by simp
finally have P6:  $\bigcup (FL \text{ ' set Ts}) = \bigcup (FLty \text{ ' set taus})$ 
by simp

have FL (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts) =
  FL (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
 $\cup$   $\bigcup (FL \text{ ' set (Ts)})$ 
by (simp add: FL-Mols1)
also have
  ... = (set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ )  $\cup$   $\bigcup (FL \text{ ' set Ts})$ 
using P11 by presburger
also have
  ... = set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$   $\cup$   $\bigcup (FLty \text{ ' set taus})$ 
using P6 by presburger
finally have
  FL (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts) =
    set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$   $\cup$   $\bigcup (FLty \text{ ' set taus})$ 
by blast
then have P30:
  FL (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
    {n. n + i  $\in$  set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 
       $\cup$   $(\bigcup ty \in \text{set taus. FLty ty})$ }
by (simp add: FL-Nus1)

have FL (app-prodrule Ts rhs) = FL (app-prodrule Ts (i, (C, zs), fusions, taus))
using prod-rhs2 by auto
also have ... =
  FL (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts))
by (simp add: app-prodrule-def)
also have ... =
  {n. n + i  $\in$  set zs  $\cup$   $(\bigcup (x, y) \in \text{set fusions. } \{x, y\})$ 

```

```

      ∪ (∪ ty ∈ set taus. FLty ty)}
    using P30 by blast
  also have ... = FLrhs (i, (C, zs), fusions, taus)
    apply (simp add: FLrhs-def)
    using FLfusion-set-def by auto
  also have ... = FLrhs rhs
    using prod-rhs2 by blast
  finally show ?thesis by simp
qed

```

13.5 Link Substitution Over Link Substitution

```

lemma lmap-lmap-commute-link-aux:
  assumes v ∈ set (map fst s)
  shows f (lsubst s v) =
    lsubst (map (λ(x,y). (x, f y)) s) v
using assms
proof (induct s)
  case Nil
  then show ?case
  by auto
next
  case (Cons a s)
  then show ?case
  apply auto
  apply (simp-all add: case-prod-beta lsubst-simp)
  by (simp add: rev-image-eqI)
qed

```

```

lemma lmap-lmap-commute-link:
  assumes v ∈ set xs
  assumes length xs = length ys
  shows f (lsubst (zip xs ys) v) =
    lsubst (zip xs (map f ys)) v
proof -
  have zip xs (map f ys) = map (λ(x,y). (x, f y)) (zip xs ys)
  by (simp add: zip-map2)
  then show ?thesis
  by (simp add: assms(1,2) lmap-lmap-commute-link-aux)
qed

```

```

lemma lmap-lmap-commute-links:
  assumes set vs ⊆ set xs
  assumes length xs = length ys
  shows map f (map (lsubst (zip xs ys)) vs) =
    map (lsubst (zip xs (map f ys))) vs
using assms
proof (induct vs arbitrary: xs ys)
  case Nil
  then show ?case
  by simp
next
  case (Cons a vs)
  then show ?case
  by (simp add: lmap-lmap-commute-link)
qed

```

```

lemma lmap-shift:
  shows (lsubst s) x + 1 = lsubst (map (λ(z, y). (z + 1, y + 1)) s) (x + 1)
proof (induct s)
  case Nil
  then show ?case
  by simp
next

```

```

case (Cons a s)
then show ?case
by (simp-all add: lsubst-simp case-prod-beta)
qed

```

```

lemma lmap-shift2:
  ( $\lambda x. \text{if } x = 0 \text{ then } x \text{ else } ((\text{lsubst } s) (x - 1) + 1)) y =$ 
    $\text{lsubst } ((0, 0) \# \text{map } (\lambda(x, y). (x + 1, y + 1)) s) y$ 
apply auto
using Suc-eq-plus1 Suc-pred lmap-shift by presburger

```

```

lemma lmap-lmap-commute:
  assumes  $FL\ G \subseteq \text{set } xs$ 
  assumes  $\text{length } xs = \text{length } ys$ 
  shows  $\text{lmap } f (\text{lmap } (\text{lsubst } (\text{zip } xs\ ys))\ G) = \text{lmap } (\text{lsubst } (\text{zip } xs\ (\text{map } f\ ys)))\ G$ 
using assms
proof (induct G arbitrary: xs ys f rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by simp
next
  case (Atom p vs)
  then show ?case
  using lmap-lmap-commute-links by force
next
  case (Fusion x1 x2)
  then show ?case
  apply auto
  apply (simp add: lmap-lmap-commute-link)
  by (simp add: lmap-lmap-commute-link)
next
  case (Mol G1 G2)
  then show ?case
  by auto
next
  case (Nu g)

```

```

have SucMinus:  $\forall S. \text{Suc } '(\lambda x. x - 1) ' (S - \{0\}) = S - \{0\}$ 
apply auto
by (simp add: image-iff)

```

```

have  $FL\ (Nu\ g) \subseteq \text{set } xs$ 
using Nu.prem1 by auto
then have  $\{n. \text{Suc } n \in FL\ g\} \subseteq \text{set } xs$ 
by auto
then have  $(\lambda x. x - 1) ' (FL\ g - \{0\}) \subseteq \text{set } xs$ 
using shift-down-alt by blast
then have  $\text{Suc } '(\lambda x. x - 1) ' (FL\ g - \{0\}) \subseteq \text{Suc } ' \text{set } xs$ 
by (meson image-mono)
then have  $(FL\ g - \{0\}) \subseteq \text{Suc } ' \text{set } xs$ 
using SucMinus by simp
then have  $FL\ g \subseteq \text{insert } 0\ (\text{Suc } ' \text{set } xs)$ 
by auto
then have  $P1: FL\ g \subseteq \text{set } (0 \# \text{map } \text{Suc } xs)$ 
by simp

```

```

have P5:  $\forall ys. \text{map } (\lambda(x, y). (x + 1, y + 1)) (\text{zip } xs\ ys) = \text{zip } (\text{map } \text{Suc } xs) (\text{map } \text{Suc } ys)$ 
apply auto
by (simp add: zip-map-map)

```

```

have IH:  $\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f\ (x - 1) + 1))$ 
   $(\text{lmap } (\text{lsubst } (\text{zip } (0 \# \text{map } \text{Suc } xs) (0 \# \text{map } \text{Suc } ys)))\ g) =$ 
   $\text{lmap } (\text{lsubst } (\text{zip } (0 \# \text{map } \text{Suc } xs)$ 
     $(\text{map } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f\ (x - 1) + 1)) (0 \# \text{map } \text{Suc } ys))))\ g$ 

```

by (*metis* (*lifting*) *Nu.hyps* *Nu.prem*s(2) *P1 length-Cons length-map*)

have *IH2*: $\text{map } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \ (x - 1) + 1)) \ (0 \ \# \ \text{map } \text{Suc } ys) =$
 $0 \ \# \ \text{map } \text{Suc } (\text{map } f \ ys)$
apply (*induct* *ys*) **by** *auto*

have *IH3*: $\forall \ ys. \ \text{lmap } (\text{lsubst } (\text{zip } xs \ ys)) \ (Nu \ g) =$
 $Nu \ (\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } ys))) \ g)$

proof

fix *ys*

have $\text{lmap } (\text{lsubst } (\text{zip } xs \ ys)) \ (Nu \ g) =$

$Nu \ (\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (\text{lsubst } (\text{zip } xs \ ys) \ (x - 1) + 1)) \ g)$

using *lmap.simps*(5) **by** *blast*

also have ... =

$Nu \ (\text{lmap } (\text{lsubst } ((0, 0) \ \# \ \text{map } (\lambda(x, y). (x + 1, y + 1)) \ (\text{zip } xs \ ys))) \ g)$

using *lmap-shift2* **by** *presburger*

also have ... =

$Nu \ (\text{lmap } (\text{lsubst } ((0, 0) \ \# \ \text{zip } (\text{map } \text{Suc } xs) \ (\text{map } \text{Suc } ys))) \ g)$

using *P5* **by** *presburger*

also have ... =

$Nu \ (\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } ys))) \ g)$

by *simp*

finally show

$\text{lmap } (\text{lsubst } (\text{zip } xs \ ys)) \ (Nu \ g) =$

$Nu \ (\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } ys))) \ g) .$

qed

have $\text{lmap } f \ (\text{lmap } (\text{lsubst } (\text{zip } xs \ ys)) \ (Nu \ g)) =$
 $\text{lmap } f \ (Nu \ (\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } ys))) \ g))$
using *IH3* **by** *presburger*

also have ... =

$Nu \ (\text{lmap } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \ (x - 1) + 1))$

$(\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } ys))) \ g))$

using *lmap.simps*(5) **by** *blast*

also have ... =

$Nu \ ((\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs)$

$(\text{map } (\lambda x. \text{if } x = 0 \text{ then } x \text{ else } (f \ (x - 1) + 1)) \ (0 \ \# \ \text{map } \text{Suc } ys)))) \ g))$

using *IH* **by** *blast*

also have ... =

$Nu \ ((\text{lmap } (\text{lsubst } (\text{zip } (0 \ \# \ \text{map } \text{Suc } xs) \ (0 \ \# \ \text{map } \text{Suc } (\text{map } f \ ys)))) \ g))$

using *IH2* **by** *argo*

also have ... = $\text{lmap } (\text{lsubst } (\text{zip } xs \ (\text{map } f \ ys))) \ (Nu \ g)$

using *IH3* **by** *presburger*

finally show *?case* .

qed *auto*

14 Properties of Lift and Graph Substitution

14.1 Basic Properties of Lift

lemma *lift-graph-lmap-commute*:

$\text{lift-graph } i \ (\text{lmap } f \ g) = \text{lmap } f \ (\text{lift-graph } i \ g)$

apply (*induct* *g* *arbitrary*: *f* *i*) **by** *simp-all*

lemma *lift-graph-freelinks*:

$FL \ (\text{lift-graph } i \ g) = FL \ g$

apply (*induct* *g* *arbitrary*: *i*) **by** *simp-all*

lemma *lifts-graph-freelinks*:

$FL \ ((\text{lift-graph } i \ \frown n) \ g) = FL \ g$

apply (*induct* *n*)

using *lift-graph-freelinks*

by *auto*

```

lemma lsubst-graph-freelinks:
  assumes length xs = length ys
  assumes set xs = FL G
  assumes distinct xs
  shows FL (lmap (lsubst (zip xs ys)) G) = set ys
proof –
  have P1: FL (lmap (lsubst (zip xs ys)) G) =
    (lsubst (zip xs ys)) ‘ FL G
    by (simp add: FL-lmap-commute)
  have P2: lsubst (zip xs ys) ‘ set xs = set ys
    by (metis assms(1,3) lsubst-image map-fst-zip map-snd-zip)
  then have P3: lsubst (zip xs ys) ‘ FL G = set ys
    by (simp add: assms(2))
  then show ?thesis
    by (simp add: FL-lmap-commute)
qed

```

```

lemma lsubst-graph-freelinks-subset:
  assumes length xs = length ys
  assumes FL G ⊆ set xs
  assumes distinct xs
  shows FL (lmap (lsubst (zip xs ys)) G) ⊆ set ys
proof –
  have P1: FL (lmap (lsubst (zip xs ys)) G) ⊆ (lsubst (zip xs ys)) ‘ FL G
    by (simp add: FL-lmap-commute)
  have P2: lsubst (zip xs ys) ‘ set xs = set ys
    by (metis assms(1,3) lsubst-image map-fst-zip map-snd-zip)
  then have P3: lsubst (zip xs ys) ‘ FL G ⊆ set ys
    using assms(2) by blast
  then show ?thesis
    by (simp add: FL-lmap-commute)
qed

```

14.2 Basic Properties of Graph Substitution

```

lemma subst-atom-freelinks:
  assumes set xs = FL g2
  assumes distinct xs
  shows FL (subst-p i xs g2 p ys) = set ys
proof (cases p)
  case (GConstr x1)
  then show ?thesis
    by simp
next
  case (GVar j)
  consider (lt) i < j | (eq) i = j | (gt) j < i
    by arith
  then show ?thesis
proof cases
  case lt
    then show ?thesis
    by (simp add: GVar)
  next
  case eq
    then show ?thesis
    by (simp add: GVar assms(1,2) lsubst-graph-freelinks)
  next
  case gt
    then show ?thesis
    by (simp add: GVar)
qed
next
  case (GAbs x31 x32 x33)
  then show ?thesis
  by simp

```

qed

lemma subst-graph-freelinks-equiv:

```
  assumes set xs = FL g2
  assumes distinct xs
  shows FL (subst-graph i xs g2 g) = FL g
using assms
proof (induct g arbitrary: i xs g2 rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by auto
next
  case (Atom x1 x2)
  then show ?case
  by (simp add: subst-atom-freelinks)
next
  case (Fusion x1 x2)
  then show ?case
  by simp
next
  case (Mol g1 g2)
  then show ?case
  by simp
next
  case (Nu g)
  then show ?case
  by simp
qed auto
```

lemma lift-graph-cong:

```
  fixes g1 g2 :: 'ty graph
  assumes g1  $\simeq$  g2
  shows lift-graph i g1  $\simeq$  lift-graph i g2
using assms
proof (induction arbitrary: i rule: gcong.induct)
  case (refl g)
  then show ?case
  apply auto
  done
next
  case (sym g1 g2)
  then show ?case
  apply auto
  done
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast
next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
```

```

case (nu-cong g g')
then show ?case
by (simp add: gcong.nu-cong)
next
case (nu-subst-fusion1 y g)
have P1: lift-graph i (Nu (lmap (λx. if x = 0 then y else x) g)) =
  Nu (lift-graph i (lmap (λx. if x = 0 then y else x) g))
  using lift-graph.simps(5) by blast
have P2: ... = Nu (lmap (λx. if x = 0 then y else x) (lift-graph i g))
  by (simp add: lift-graph-lmap-commute)
have P3: y ∈ FL (lift-graph i g)
  by (simp add: lift-graph-freelinks local.nu-subst-fusion1)
then have P5: Nu (lmap (λx. if x = 0 then y else x) (lift-graph i g)) ≃
  Nu (Mol (Fusion 0 y) (lift-graph i g))
  using gcong.nu-subst-fusion1 by blast
have P6: ... = lift-graph i (Nu (Mol (Fusion 0 y) g))
  by simp
then show ?case
  by (simp add: P2 P5 gcong.sym)
next
case (nu-subst-fusion2 g y)
have P1: lift-graph i (Nu (lmap (λx. if x = 0 then y else x) g)) =
  Nu (lift-graph i (lmap (λx. if x = 0 then y else x) g))
  using lift-graph.simps(5) by blast
have P2: ... = Nu (lmap (λx. if x = 0 then y else x) (lift-graph i g))
  by (simp add: lift-graph-lmap-commute)
have P3: 0 ∈ FL (lift-graph i g)
  by (simp add: lift-graph-freelinks local.nu-subst-fusion2)
then have P5: Nu (lmap (λx. if x = 0 then y else x) (lift-graph i g)) ≃
  Nu (Mol (Fusion 0 y) (lift-graph i g))
  using gcong.nu-subst-fusion2 by blast
have P6: ... = lift-graph i (Nu (Mol (Fusion 0 y) g))
  by simp
then show ?case
  by (simp add: P2 P5 gcong.sym)
next
case nu-nu-fusion1
then show ?case
  using gcong.nu-nu-fusion1 by auto
next
case nu-nu-fusion2
then show ?case
  by simp
next
case nu-zero
then show ?case
  by simp
next
case (nu-comm g)
then show ?case
  by (simp add: gcong.nu-comm lift-graph-lmap-commute)
next
case (nu-scope g2 g1)
then show ?case
  apply auto
  by (simp add: gcong.nu-scope lift-graph-lmap-commute)
qed

```

15 Theorems of Congruence

15.1 Free Links Equivalence of Congruent Graphs

We show that the set of free links over congruent graphs are equivalent.

lemma *freelinks-equiv-E6A*:

assumes A1: $G1 = \text{Nu} (\text{Mol} (\text{Fusion } 0 \ y) \ g)$
and A2: $G2 = \text{Nu} (\text{lmap} (\lambda \ x. \text{if } x = 0 \text{ then } y \text{ else } x) \ g)$

```

    and A3:  $y \in FL\ g$ 
  shows  $FL\ G1 = FL\ G2$ 
proof -
  have  $\{n. Suc\ n \in (\lambda\ x. \text{if } x = 0 \text{ then } y \text{ else } x) \text{ ' } FL\ g\} =$ 
     $\{n. Suc\ n \in FL\ (Mol\ (Fusion\ 0\ y)\ g)\}$ 
  apply simp
  using A3 by blast
  then show ?thesis
  by (simp add: A1 A2 FL-lmap-commute)
qed

```

```

lemma freelinks-equiv-E6B:
  assumes A1:  $G1 = Nu\ (Mol\ (Fusion\ 0\ y)\ g)$ 
    and A2:  $G2 = Nu\ (lmap\ (\lambda\ x. \text{if } x = 0 \text{ then } y \text{ else } x)\ g)$ 
    and A3:  $0 \in FL\ g$ 
  shows  $FL\ G1 = FL\ G2$ 
proof -
  have P1:  $FL\ (lmap\ (\lambda\ x. \text{if } x = 0 \text{ then } y \text{ else } x)\ g) =$ 
     $(\lambda\ x. \text{if } x = 0 \text{ then } y \text{ else } x) \text{ ' } FL\ g$ 
  using FL-lmap-commute by blast
  have P2:  $\{n. Suc\ n \in (\lambda\ x. \text{if } x = 0 \text{ then } y \text{ else } x) \text{ ' } FL\ g\} =$ 
     $\{n. Suc\ n \in FL\ (Mol\ (Fusion\ 0\ y)\ g)\}$ 
  apply auto
  using A3 by blast
  then show ?thesis
  by (simp add: A1 A2 FL-lmap-commute)
qed

```

```

lemma freelinks-equiv-E9:
  assumes G1 =  $Nu\ (Nu\ g)$ 
    and G2 =  $Nu\ (Nu\ (lmap\ (\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g))$ 
  shows  $FL\ G1 = FL\ G2$ 
proof -
  have P1:  $FL\ (lmap\ (\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g) =$ 
     $(\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) \text{ ' } FL\ g$ 
  using FL-lmap-commute by blast

  have P2:  $FL\ (Nu\ (Nu\ (lmap\ (\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x)\ g))) =$ 
     $\{n. Suc\ (Suc\ n) \in (\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) \text{ ' } FL\ g\}$ 
  using Collect-cong FL.simps(5) P1 mem-Collect-eq by auto
  have P3:  $\dots = \{n. Suc\ (Suc\ n) \in (\lambda\ x. \text{if } x = 0 \text{ then } 1 \text{ else if } x = 1 \text{ then } x \text{ else } x) \text{ ' } FL\ g\}$ 
  using P1 by simp
  have P4:  $\dots = \{n. Suc\ (Suc\ n) \in FL\ g\}$ 
  using diff-Suc-1' by fastforce
  have P5:  $\dots = FL\ (Nu\ (Nu\ g))$ 
  by auto
  show ?thesis
  using P2 P4 P5 assms(1,2) by argo
qed

```

```

lemma freelinks-equiv-E10:
  assumes A1:  $G1 = Mol\ (Nu\ g1)\ g2$ 
    and A2:  $G2 = Nu\ (Mol\ g1\ (lmap\ Suc\ g2))$ 
  shows  $FL\ G1 = FL\ G2$ 
proof -
  have  $FL\ G2 = FL\ (Nu\ (Mol\ g1\ (lmap\ Suc\ g2)))$ 
  by (simp add: A2)
  also have  $\dots = \{n. Suc\ n \in FL\ (Mol\ g1\ (lmap\ Suc\ g2))\}$ 
  by simp
  also have  $\dots = \{n. Suc\ n \in (FL\ g1 \cup FL\ (lmap\ Suc\ g2))\}$ 
  by simp
  also have  $\dots = \{n. Suc\ n \in FL\ g1\} \cup \{n. Suc\ n \in FL\ (lmap\ Suc\ g2)\}$ 
  by auto

```

```

also have ... = FL (Nu g1) ∪ {n. Suc n ∈ FL (lmap Suc g2)}
  by simp
also have ... = FL (Nu g1) ∪ FL (Nu (lmap Suc g2))
  by simp
also have ... = FL (Nu g1) ∪ FL g2
  by (simp add: FL-lmap-deSuc-Suc)
also have ... = FL (Mol (Nu g1) g2)
  by simp
also have ... = FL G1
  by (simp add: A1)
finally show ?thesis
  by simp
qed

```

Equivalence of free links over congruent graphs.

```

lemma freelinks-equiv:
  assumes g1 ≃ g2
  shows FL g1 = FL g2
using assms
proof (induction rule: gcong.induct)
  case (refl g)
  then show ?case by simp
next
  case (sym g1 g2)
  then show ?case by simp
next
  case (trans g1 g2 g3)
  then show ?case
    by presburger
next
  case (mol-zero g)
  then show ?case by simp
next
  case (mol-comm g1 g2)
  then show ?case
    by (simp add: sup-commute)
next
  case (mol-assoc a b c)
  then show ?case by (simp add: sup-assoc)
next
  case (mol-cong g1 g2 g1' g2')
  then show ?case
    using FL.simps(4) by blast
next
  case (nu-cong g g')
  then show ?case
    using FL.simps(5) by blast
next
  case (nu-subst-fusion1 y g)
  then show ?case
    using freelinks-equiv-E6A by blast
next
  case (nu-subst-fusion2 y g)
  then show ?case
    using freelinks-equiv-E6B by blast
next
  case (nu-nu-fusion1)
  then show ?case
    by auto
next
  case (nu-nu-fusion2)
  then show ?case by simp
next
  case (nu-zero)
  then show ?case by simp
next
  case (nu-comm g)

```

```

then show ?case
using freelinks-equiv-E9 by blast
next
case (nu-scope g1 g2)
then show ?case
using freelinks-equiv-E10 by blast
qed

```

15.2 Multiset Invariant Over Congruent Graphs

Checks that the left-most atom is a constructor atom. Used to distinguish the graph created by `app_prod` and an abstraction atom.

```

primrec f-of :: 'ty graph => ('ty p * nat) multiset
where
  f-of Zero = {#}
| f-of (Atom p xs) = {# (p, length xs) #}
| f-of (Fusion x y) = {#}
| f-of (Mol g1 g2) = f-of g1 + f-of g2
| f-of (Nu g) = f-of g

```

```

lemma lmap-id: lmap id g = g
apply (induct g arbitrary: f rule: graph.induct[of - %-. True %-. True])
apply auto
by (metis (full-types, lifting) ext Suc-eq-plus1 Suc-pred' neq0-conv)

```

```

lemma f-lmap-ignore[simp]:
  f-of (lmap f g) = f-of g
by (induct g arbitrary: f) auto

```

```

lemma f-cong-gcong:
  assumes g1  $\simeq$  g2
  shows f-of g1 = f-of g2
using assms
apply (induct set: gcong)
by auto

```

```

lemma abs-cong-inv:
  assumes Atom (GAbs xs1 ty1 e1) ys1  $\simeq$  Atom (GAbs xs2 ty2 e2) ys2
  shows xs1 = xs2  $\wedge$  ty1 = ty2  $\wedge$  e1 = e2  $\wedge$  length ys1 = length ys2
proof -
  have P1: f-of (Atom (GAbs xs1 ty1 e1) ys1) = f-of (Atom (GAbs xs2 ty2 e2) ys2)
  using assms f-cong-gcong by blast
  have P2: f-of (Atom (GAbs xs1 ty1 e1) ys1) = {# (GAbs xs1 ty1 e1, length ys1) #}
  by simp
  have P3: f-of (Atom (GAbs xs2 ty2 e2) ys2) = {# (GAbs xs2 ty2 e2, length ys2) #}
  by simp
  have P4: {# (GAbs xs1 ty1 e1, length ys1) #} = {# (GAbs xs2 ty2 e2, length ys2) #}
  using P1 P2 P3 by argo
  then show ?thesis
  by fastforce
qed

```

```

lemma abs-var-cong-inv:
   $\neg$ (Atom (GVar i) ys'  $\simeq$  Atom (GAbs xs ty1 e) ys)
proof -
  have  $\neg$ ((GVar i, length ys') = (GAbs xs ty1 e, length ys))
  by force
  then have  $\neg$ ({# (GVar i, length ys') #} = {# (GAbs xs ty1 e, length ys) #})
  by simp
  then have  $\neg$  f-of (Atom (GVar i) ys') = f-of (Atom (GAbs xs ty1 e) ys)
  by simp

```

```

then show ?thesis
  using f-cong-gcong by blast
qed

```

15.3 Lemmata with Lift Graph

```

lemma lift-graph-Nus1-equiv:
  lift-graph i (Nus1 j G) = Nus1 j (lift-graph i G)
apply (induct j)
by simp-all

```

```

lemma lift-graph-Mols1-equiv:
  lift-graph i (Mols1 G Gs) = Mols1 (lift-graph i G) (map (lift-graph i) Gs)
apply (induct Gs arbitrary: G)
by (simp-all add: Mols1)

```

```

lemma lift-graph-fusions-of:
  map (lift-graph i) (fusions-of fusions) = fusions-of fusions
proof (induction fusions)
case Nil
then show ?case
  by (simp add: fusions-of-def)
next
case (Cons p ps)
obtain x y where p-eq: p = (x, y)
  by (cases p) auto
then show ?case
  by (simp add: case-prod-beta fusions-of-def)
qed

```

```

lemma lift-graph-app-prodrule-equiv:
  lift-graph i (app-prodrule Ts rhs) =
    app-prodrule (map (lift-graph i) Ts) rhs
proof -
obtain j C zs fusions taus
  where rhs-eq: rhs = (j, (C, zs), fusions, taus)
  by (cases rhs) auto
have P1: lift-graph i (Atom (GConstr C) zs) = Atom (GConstr C) zs
  by simp

```

```

have P2: map (lift-graph i) (fusions-of fusions) = fusions-of fusions
  by (simp add: lift-graph-fusions-of)

```

```

have P3:
  lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    Mols1 (lift-graph i (Atom (GConstr C) zs))
      (map (lift-graph i) (fusions-of fusions))
  by (simp add: lift-graph-Mols1-equiv)

```

```

from P1 P2 P3 have P4:
  lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) =
    Mols1 (Atom (GConstr C) zs) (fusions-of fusions)
  by (simp add: lift-graph-fusions-of)

```

```

have P5:
  lift-graph i
    (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
    Nus1 j
      (Mols1 (lift-graph i (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)))
        (map (lift-graph i) Ts))
  by (simp add: lift-graph-Mols1-equiv lift-graph-Nus1-equiv)

```

```

from P4 P5 have P6:

```

```

lift-graph i
  (Nus1 j (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)) =
  Nus1 j
    (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
      (map (lift-graph i) Ts))
by metis

show ?thesis
  unfolding rhs-eq app-prodrule-def
  using P6 by simp
qed

```

```

lemma lift-graph-make-diffgraph-equiv:
  lift-graph i (make-diffgraph rhs) = make-diffgraph rhs
proof -
  obtain j C zs fusions taus where
    rhs: rhs = (j, (C, zs), fusions, taus)
  by (metis surj-pair)

  have P2: make-diffgraph rhs = Mols1 (Atom (GConstr C) zs) (fusions-of fusions)
  apply (simp add: make-diffgraph-def)
  using rhs by auto

  then show ?thesis
  by (simp add: lift-graph-Mols1-equiv lift-graph-fusions-of)
qed

```

```

lemma lift-graph-lbind-com:
  lift-graph i (lbind x g) = lbind x (lift-graph i g)
by (simp add: lbind-def lift-graph-lmap-commute)

```

```

lemma lift-graph-lbind-links-equiv:
  lift-graph i (fold lbind ys T) = fold lbind ys (lift-graph i T)
apply (induction ys arbitrary: T)
apply simp
by (simp add: lift-graph-lbind-com)

```

```

lemma lift-graph-lbinds-equiv:
  lift-graph i (lbinds ys T) = lbinds ys (lift-graph i T)
by (simp add: lbinds-def lift-graph-lbind-links-equiv)

```

```

lemma FL-lbinds:
  FL (lbinds ys g) = FL g - set ys
proof (induct ys arbitrary: g)
  case Nil
  then show ?case
  apply auto
  apply (simp-all add: lbinds-def)
  done
next
  case (Cons a ys)
  then show ?case
  apply auto
  apply (simp-all add: lbinds-def)
  apply (simp-all add: lbind-hide)
  done
qed

```

15.4 Lemmata with Subst Graph

```

lemma subst-graph-Nus1-equiv:
  subst-graph i xs g2 (Nus1 j G) = Nus1 j (subst-graph i xs g2 G)

```

apply (*induct j*)
by *simp-all*

lemma *subst-graph-Mols1-equiv*:
 $\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{Mols1 } G \text{ } Gs) = \text{Mols1 } (\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } G) \text{ } (\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } Gs)$
apply (*induct Gs arbitrary: G*)
by (*simp-all add: Mols1*)

lemma *subst-graph-fusions-of*:
 $\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } (\text{fusions-of fusions}) = \text{fusions-of fusions}$
proof (*induct fusions*)
case *Nil*
then show *?case*
by (*simp add: fusions-of-def*)
next
case (*Cons a fusions*)
then show *?case*
apply (*simp-all add: fusions-of-def*)
by (*simp add: case-prod-beta*)
qed

lemma *subst-typrod*:
 $\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{app-prodrule } Ts \text{ } rhs) = \text{app-prodrule } (\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } Ts) \text{ } rhs$
proof –
obtain *j C zs fusions taus*
where *rhs-eq: rhs = (j, (C, zs), fusions, taus)*
by (*cases rhs auto*)

have *P1: subst-graph i xs g2 (Atom (GConstr C) zs) = Atom (GConstr C) zs*
by *simp*

have *P2: map (subst-graph i xs g2) (fusions-of fusions) = fusions-of fusions*
by (*simp add: subst-graph-fusions-of*)

have *P3*:
 $\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})) =$
 $\text{Mols1 } (\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{Atom } (GConstr \text{ } C) \text{ } zs))$
 $\text{ } (\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } (\text{fusions-of fusions}))$
by (*simp add: subst-graph-Mols1-equiv*)

from *P1 P2 P3* **have** *P4*:
 $\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})) =$
 $\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})$
by *simp*

have *P5*:
 $\text{subst-graph } i \text{ } xs \text{ } g2$
 $\text{ } (\text{Nus1 } j \text{ } (\text{Mols1 } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})) \text{ } Ts)) =$
 $\text{Nus1 } j$
 $\text{ } (\text{Mols1 } (\text{subst-graph } i \text{ } xs \text{ } g2 \text{ } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})))$
 $\text{ } (\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } Ts))$
by (*simp add: subst-graph-Mols1-equiv subst-graph-Nus1-equiv*)

from *P4 P5* **have** *P6*:
 $\text{subst-graph } i \text{ } xs \text{ } g2$
 $\text{ } (\text{Nus1 } j \text{ } (\text{Mols1 } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions})) \text{ } Ts)) =$
 $\text{Nus1 } j$
 $\text{ } (\text{Mols1 } (\text{Mols1 } (\text{Atom } (GConstr \text{ } C) \text{ } zs) \text{ } (\text{fusions-of fusions}))$
 $\text{ } (\text{map } (\text{subst-graph } i \text{ } xs \text{ } g2) \text{ } Ts))$
by *metis*
then show *?thesis*

```

    unfolding rhs-eq app-prodrule-def
    using P6 by simp
qed

```

15.5 Lemmata of Link substitutions

```

lemma subst-p-lmap-commute:
  assumes FL G  $\subseteq$  set xs
  shows lmap f (subst-p i xs G p ys) = subst-p i xs G p (map f ys)
using assms
proof (cases p)
  case (GConstr x1)
  then show ?thesis
  by simp
next
  case (GVar j)
  consider (lt) i < j | (eq) i = j | (gt) j < i
  by arith
  then show ?thesis
proof cases
  case lt
  then show ?thesis
  by (simp add: GVar)
  next
  case eq
  then show ?thesis
  by (simp add: GVar assms(1) lmap-lmap-commute)
  next
  case gt
  then show ?thesis
  by (simp add: GVar)
qed
next
  case (GAbs x31 x32 x33)
  then show ?thesis
  by simp
qed

```

```

lemma subst-graph-lmap-commute:
  assumes FL G  $\subseteq$  set xs
  shows lmap f (subst-graph i xs G g1) = subst-graph i xs G (lmap f g1)
using assms
proof (induct g1 arbitrary: f rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by auto
next
  case (Atom x1 x2)
  then show ?case
  apply auto
  by (simp add: subst-p-lmap-commute)
next
  case (Fusion x1 x2)
  then show ?case
  by auto
next
  case (Mol g11 g12)
  then show ?case
  by auto
next
  case (Nu g1)
  then show ?case
  by auto
qed auto

```

```

lemma lmap-compose:
  lmap f (lmap g G) = lmap (f o g) G
proof (induct G arbitrary: f g rule: graph.induct[of - %-. True %-. True])
  case Zero
  then show ?case
  by simp
next
  case (Atom x1 x2)
  then show ?case
  by auto
next
  case (Fusion x1 x2)
  then show ?case
  by auto
next
  case (Mol G1 G2)
  then show ?case
  by auto
next
  case (Nu G)
  have P2: (λx. if x = 0 then x else f (x - 1) + 1) o (λx. if x = 0 then x else g (x - 1) + 1) =
    (λx. if x = 0 then x else f (g (x - 1)) + 1)
    using Suc-eq-plus1 Zero-not-Suc diff-Suc-1 by auto
  show ?case
  apply auto
  using Nu P2 by presburger
qed auto

```

```

lemma lmap-lim:
  ( $\forall x \in FL\ G. f\ x = g\ x$ )  $\implies lmap\ f\ G = lmap\ g\ G$ 
apply (induct G arbitrary: f g)
by simp-all

```

```

lemma lmap-graph-cong:
   $g1 \simeq g2 \implies lmap\ f\ g1 \simeq lmap\ f\ g2$ 
proof (induct arbitrary: f rule: gcong.induct)
  case (refl g)
  then show ?case
  by blast
next
  case (sym g1 g2)
  then show ?case
  by blast
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast
next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
  case (nu-cong g g')

```

```

then show ?case
by auto
next
case (nu-subst-fusion1 y g)

have y ∈ FL g
  by (simp add: local.nu-subst-fusion1)
then have P4: (λx. if x = 0 then x else f (x - 1) + 1) y
  ∈ FL (lmap (λx. if x = 0 then x else f (x - 1) + 1) g)
  using FL-lmap-commute by blast

have P6: (λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  o (λx. if x = 0 then x else f (x - 1) + 1) =
  (λx. if x = 0 then x else f (x - 1) + 1) o (λx. if x = 0 then y else x)
  by auto

have T1: lmap f (Nu (Mol (Fusion 0 y) g)) =
  Nu (Mol (Fusion 0 ((λx. if x = 0 then x else f (x - 1) + 1) y))
  (lmap (λx. if x = 0 then x else f (x - 1) + 1) g))
  by auto
also have T2: ... ≃
  Nu (lmap (λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  (lmap (λx. if x = 0 then x else f (x - 1) + 1) g))
  using P4 gcong.nu-subst-fusion1 by blast
have T3: ... =
  Nu (lmap
  ((λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  o (λx. if x = 0 then x else f (x - 1) + 1)) g)
  using lmap-compose by blast
have T4: ... =
  Nu (lmap
  ((λx. if x = 0 then x else f (x - 1) + 1) o (λx. if x = 0 then y else x)) g)
  by (simp add: P6)
have T5: ... =
  Nu (lmap (λx. if x = 0 then x else f (x - 1) + 1)
  (lmap (λx. if x = 0 then y else x) g))
  by (simp add: lmap-compose)
have T6: ... = lmap f (Nu (lmap (λx. if x = 0 then y else x) g))
  by simp
show ?case
  using P6 T2 T3 T5 T6 calculation by argo
next
case (nu-subst-fusion2 g y)
have 0 ∈ FL g
  by (simp add: local.nu-subst-fusion2)
then have P4: 0
  ∈ FL (lmap (λx. if x = 0 then x else f (x - 1) + 1) g)
  by (simp add: FL-lmap-commute)

have P6: (λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  o (λx. if x = 0 then x else f (x - 1) + 1) =
  (λx. if x = 0 then x else f (x - 1) + 1) o (λx. if x = 0 then y else x)
  by auto

have T1: lmap f (Nu (Mol (Fusion 0 y) g)) =
  Nu (Mol (Fusion 0 ((λx. if x = 0 then x else f (x - 1) + 1) y))
  (lmap (λx. if x = 0 then x else f (x - 1) + 1) g))
  by auto
also have T2: ... ≃
  Nu (lmap (λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  (lmap (λx. if x = 0 then x else f (x - 1) + 1) g))
  using P4 gcong.nu-subst-fusion2 by blast
have T3: ... =
  Nu (lmap
  ((λx. if x = 0 then (λx. if x = 0 then x else f (x - 1) + 1) y else x)
  o (λx. if x = 0 then x else f (x - 1) + 1)) g)
  using lmap-compose by blast

```

```

have T4: ... =
  Nu (lmap
    ((λx. if x = 0 then x else f (x - 1) + 1) o (λx. if x = 0 then y else x)) g)
  by (simp add: P6)
have T5: ... =
  Nu (lmap (λx. if x = 0 then x else f (x - 1) + 1)
    (lmap (λx. if x = 0 then y else x) g))
  by (simp add: lmap-compose)
have T6: ... = lmap f (Nu (lmap (λx. if x = 0 then y else x) g))
  by simp
show ?case
  using P6 T2 T3 T5 T6 calculation by argo
next
case nu-nu-fusion1
then show ?case
  apply auto
  using gcong.nu-nu-fusion1 by auto
next
case nu-nu-fusion2
then show ?case
  by auto
next
case nu-zero
then show ?case
  by auto
next
case (nu-comm g)

have P1:
  (λx. if x = 0 then 1 else if x = 1 then x else x)
  o (λx. if x = 0 then x else (if x - 1 = 0 then x - 1 else f (x - 1 - 1) + 1) + 1) =
  (λx. if x = 0 then x else (if x - 1 = 0 then x - 1 else f (x - 1 - 1) + 1) + 1)
  o (λx. if x = 0 then 1 else if x = 1 then x else x)
  by auto

show ?case
  apply auto
  by (metis (no-types, lifting) P1 gcong.nu-comm lmap-compose)
next
case (nu-scope g1 g2)
have lmap (Suc o f) g2 = lmap ((λx. if x = 0 then x else f (x - 1) + 1) o Suc) g2
  by (simp add: lmap-lim)
then have P3: lmap Suc (lmap f g2) = lmap (λx. if x = 0 then x else f (x - 1) + 1) (lmap Suc g2)
  by (simp add: lmap-compose)

have E1:
  lmap f (Mol (Nu g1) g2) =
  Mol (Nu (lmap (λx. if x = 0 then x else f (x - 1) + 1) g1)) (lmap f g2)
  by simp
have E2: ... ≈
  Nu (Mol (lmap (λx. if x = 0 then x else f (x - 1) + 1) g1)
    (lmap (λx. if x = 0 then x else f (x - 1) + 1) (lmap Suc g2)))
  by (metis gcong.nu-scope P3)
have E3: ... = Nu (lmap (λx. if x = 0 then x else f (x - 1) + 1) (Mol g1 (lmap Suc g2)))
  by simp
have E4: ... = lmap f (Nu (Mol g1 (lmap Suc g2)))
  by simp

show ?case
  using E1 E2 E3 E4 by argo
qed

```

```

lemma lmap-lim2:
  assumes  $ys = xs1 \text{ @ } xs2$ 
  assumes  $set (map \text{fst } xs1) \cap FLG = \{\}$ 
  shows  $\forall x \in FLG. \text{lsubst } xs2 \ x = \text{lsubst } ys \ x$ 
using lsubst-ignore-befores assms apply auto
by (metis IntI emptyE)

lemma lmap-lim-helper:
  assumes  $set \ ys = set \ xs1 \cup set \ xs2$ 
  shows  $set \ ys = set \ (xs1 \text{ @ } xs2)$ 
by (simp add: assms)

lemma lmap-lim3:
  assumes  $set \ ys = set \ xs1 \cup set \ xs2$ 
  assumes distinct ( $map \text{fst } ys$ )
  assumes distinct ( $map \text{fst } xs2$ )
  assumes  $set (map \text{fst } xs1) \cap FLG = \{\}$ 
  shows  $\forall x \in FLG. \text{lsubst } xs2 \ x = \text{lsubst } ys \ x$ 
by (smt (verit, best) Un-commute Un-iff assms(1,2,3,4) disjoint-iff image-Un list.set-map lsubst-ignore
  lsubst-lemma7 sup-ge1)

lemma lmap-lim4:
  assumes  $set \ ys = set \ xs1 \cup set \ xs2$ 
  assumes distinct ( $map \text{fst } ys$ )
  assumes distinct ( $map \text{fst } xs2$ )
  assumes  $set (map \text{fst } xs1) \cap FL \ G = \{\}$ 
  shows  $\text{lmap } (\text{lsubst } xs2) \ G = \text{lmap } (\text{lsubst } ys) \ G$ 
by (meson assms(1,2,3,4) lmap-lim lmap-lim3)

lemma lmap-lim5:
  assumes  $xs = filter (\lambda x. \text{fst } x \in FLG) \ ys$ 
  shows  $\exists xs1. (set \ ys = set \ xs1 \cup set \ xs \wedge set (map \text{fst } xs1) \cap FLG = \{\})$ 
using assms
proof (induct ys arbitrary: xs)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a ys)
  then show ?case
  apply auto
  by (metis (no-types, lifting) Un-insert-left image-insert insert-disjoint(2) list.simps(15))
qed

lemma lmap-restriction:
  assumes distinct ( $map \text{fst } ys$ )
  assumes  $xs = filter (\lambda x. \text{fst } x \in FL \ G) \ ys$ 
  shows  $\text{lmap } (\text{lsubst } xs) \ G = \text{lmap } (\text{lsubst } ys) \ G$ 
using assms
proof –
  have P1: distinct ( $map \text{fst } xs$ )
  using assms(1,2) distinct-map-filter by blast
  have P2:  $\exists xs1. (set \ ys = set \ xs1 \cup set \ xs$ 
     $\wedge set (map \text{fst } xs1) \cap FL \ G = \{\})$ 
  using assms(2) lmap-lim5 by blast
  then show ?thesis
  using P1 assms(1) lmap-lim4 by blast
qed

```

```

lemma lmap-ty-lim4:
  assumes set ys = set xs1  $\cup$  set xs2
  assumes distinct (map fst ys)
  assumes distinct (map fst xs2)
  assumes set (map fst xs1)  $\cap$  ALty ty = {}
  shows lmap-ty (lsubst xs2) ty = lmap-ty (lsubst ys) ty
using assms
proof (induct ty)
  case (TBase x1 x2)
  then show ?case
  apply auto
  by (simp add: lmap-lim3)
next
  case (TArrow ty1 ty2 x3)
  then show ?case
  apply auto
  by (simp add: lmap-lim3)
next
  case (TLI x1 x2 x3 x4)
  then show ?case
  apply auto
  apply (simp-all add: lmap-lim3)
  by (smt (verit, ccfv-SIG) UN-absorb Un-empty inf-sup-distrib1 multiset.map-cong)
qed

```

```

lemma lmap-ty-lim5:
  assumes set xs  $\subseteq$  set ys
  assumes distinct (map fst ys)
  assumes distinct (map fst xs)
  assumes set (map fst xs) = ALty ty
  shows lmap-ty (lsubst xs) ty = lmap-ty (lsubst ys) ty
proof –
  obtain xs1 where
    xs1: xs1 = filter ( $\lambda x. \text{fst } x \notin \text{ALty } ty$ ) ys
    by presburger
  then have P1: set ys =
    set (filter ( $\lambda x. \text{fst } x \notin \text{ALty } ty$ ) ys)  $\cup$ 
    set (filter ( $\lambda x. \text{fst } x \in \text{ALty } ty$ ) ys)
    by auto

  have P2: set (filter ( $\lambda x. \text{fst } x \in \text{ALty } ty$ ) ys) = set xs
    apply auto
    apply (metis (no-types, lifting) append-Cons assms(1,2,3,4) eq-key-imp-eq-value in-mono
      in-set-conv-decomp lsubst-lemma3)
    using assms(1) apply blast
    by (metis assms(4) in-set-zipE zip-map-fst-snd)

  then have P1: set ys = set xs1  $\cup$  set xs
    using P1 xs1 by argh

  have P2: set (map fst xs1)  $\cap$  ALty ty = {}
    apply auto
    by (simp add: xs1)

  show ?thesis
    using P1 P2 assms(2,3) lmap-ty-lim4 by blast
qed

```

15.6 Graph Substitution Over Congruence Graphs and Expressions

```

lemma subst-graph-cong:
  assumes g1  $\simeq$  g2
  assumes set xs = FL G
  assumes distinct xs
  shows subst-graph i xs G g1  $\simeq$  subst-graph i xs G g2
using assms

```

```

proof (induction arbitrary: i xs G rule: gcong.induct)
  case (refl g)
  then show ?case
  by (simp add: gcong.refl)
next
  case (sym g1 g2)
  then show ?case
  by blast
next
  case (trans g1 g2 g3)
  then show ?case
  using gcong.trans by blast
next
  case (mol-zero g)
  then show ?case
  by simp
next
  case (mol-comm g1 g2)
  then show ?case
  by simp
next
  case (mol-assoc a b c)
  then show ?case
  by simp
next
  case (mol-cong g1 g1' g2 g2')
  then show ?case
  by (simp add: gcong.mol-cong)
next
  case (nu-cong g g')
  then show ?case
  by (simp add: gcong.nu-cong)
next
  case (nu-subst-fusion1 y g)
  then show ?case
  apply auto
  by (metis (no-types, lifting) gcong.nu-subst-fusion1 order-refl subst-graph-freelinks-equiv
    subst-graph-lmap-commute)
next
  case (nu-subst-fusion2 g y)
  then show ?case
  apply auto
  by (metis (no-types, lifting) dual-order.refl gcong.nu-subst-fusion2 subst-graph-freelinks-equiv
    subst-graph-lmap-commute)
next
  case nu-nu-fusion1
  then show ?case
  apply auto
  using gcong.nu-nu-fusion1 by fastforce
next
  case nu-nu-fusion2
  then show ?case
  by auto
next
  case nu-zero
  then show ?case
  by simp
next
  case (nu-comm g)
  then show ?case
  apply auto
  by (metis (no-types, lifting) gcong.nu-comm subsetI subst-graph-lmap-commute)
next
  case (nu-scope g2 g1)
  then show ?case
  apply auto
  by (metis gcong.nu-scope subset-refl subst-graph-lmap-commute)

```

qed

16 Properties of Environments

lemma *insert-at-shift*:

insert-at (*xs* @ *ys*) (*j* + *length xs*) *y* = *xs* @ *insert-at ys j y*

apply (*induction xs*)

by *auto*

Inserting at any position increases the environment length by exactly one.)

lemma *length-insert-at*:

length (*insert-at xs j y*) = *length xs* + 1

proof (*induction xs arbitrary: j*)

case *Nil*

then show ?*case*

apply (*cases j*)

apply *auto*

done

next

case (*Cons a xs*)

then show ?*case*

apply (*cases j*)

apply *simp*

by *simp*

qed

Relations between the before and the after inserted environments.

lemma *shift-eq*:

assumes $j \leq \text{length } xs$

shows (*insert-at xs j y*) ! *j* = *y*

using *assms*

proof (*induction xs arbitrary: j*)

case *Nil*

then show ?*case*

apply (*cases j*)

apply *auto*

done

next

case (*Cons y ys*)

then show ?*case*

apply (*cases j*)

apply *auto*

done

qed

lemma *shift-gt*:

assumes $i \leq \text{length } xs$

shows $j < i \implies (\text{insert-at } xs \ i \ y) ! j = xs ! j$

using *assms*

proof (*induction xs arbitrary: i j*)

case *Nil*

then show ?*case*

apply (*cases i*)

apply *simp*

by *simp*

next

case (*Cons a xs*)

then show ?*case*

apply (*cases i*)

apply *auto*

using *less-Suc-eq-0-disj* **by** *fastforce*

qed

```

lemma shift-lt:
   $i < j ==>$ 
   $(\text{insert-at } xs \ i \ y) ! j = xs ! (j - 1)$ 
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  apply (cases i)
  apply simp
  by simp
next
  case (Cons a xs)
  then show ?case
  apply (cases i)
  apply auto
  done
qed

```

Inserting y at the front and then U at position $\text{Suc } i$ is the same as first inserting U at position i and then y at the front.

```

lemma shift-commute:
   $\text{insert-at } (\text{insert-at } xs \ i \ y1) \ 0 \ y2$ 
   $= \text{insert-at } (\text{insert-at } xs \ 0 \ y2) \ (\text{Suc } i) \ y1$ 
proof (induction xs arbitrary: i j)
  case Nil
  then show ?case
  by simp
next
  case (Cons a xs)
  then show ?case
  by simp
qed

```

```

lemma lmap-ty-inv-inv:
  assumes distinct ( $\text{map fst } xs$ )
  assumes distinct ( $\text{map snd } xs$ )
  assumes  $ALty \ ty \subseteq \text{set } (\text{map fst } xs)$ 
  shows  $\text{lmap-ty } (\text{lsubst } (\text{swap-list } xs)) \ (\text{lmap-ty } (\text{lsubst } xs) \ ty) = ty$ 
using assms
proof (induct ty arbitrary: xs)
  case (TBase x1 x2)
  then show ?case
  apply auto
  by (metis  $ALty.simps(1)$   $TBase.prems(3)$  map-lsubst-inv-inv swap-map-snd swap-swap-list)
next
  case (TArrow ty1 ty2 x3)
  then show ?case
  apply auto
  by (metis  $ALty.simps(2)$   $TArrow.prems(3)$  map-lsubst-inv-inv swap-map-fst swap-swap-list)
next
  case ( $TLI \ taus \ alpha \ zs \ ys$ )

  have  $\forall ty \in \# \ taus. \ ALty \ ty \subseteq \text{set } (\text{map fst } xs)$ 
  using  $TLI.prems(3)$  by auto

  then have  $\forall ty \in \# \ taus. \ (\text{lmap-ty } (\text{lsubst } (\text{swap-list } xs))) \ (\text{lmap-ty } (\text{lsubst } xs) \ ty) = ty$ 
  using  $TLI.hyps$   $TLI.prems(1,2)$  by blast

  then have  $\text{image-mset } ((\text{lmap-ty } (\text{lsubst } (\text{swap-list } xs))) \circ (\text{lmap-ty } (\text{lsubst } xs))) \ taus = taus$ 
  by (simp add: multiset.map-ident-strong)

  then have  $P1: \text{image-mset } (\text{lmap-ty } (\text{lsubst } (\text{swap-list } xs))) \ (\text{image-mset } (\text{lmap-ty } (\text{lsubst } xs)) \ taus) = taus$ 

```

```

by (simp add: image-mset.compositionality)

have P2: set zs  $\subseteq$  set (map fst xs)
  using TLI.premis(3) by auto

have P3: fset ys  $\subseteq$  set (map fst xs)
  using TLI.premis(3) by auto

show ?case
apply auto
  using P1 apply order
apply (metis P2 TLI.premis(2) map-lsubst-inv-inv swap-map-snd swap-swap-list)
  using P3 TLI.premis(2) lsubst-inv-inv2 apply fastforce
  by (metis (no-types, lifting) P3 TLI.premis(2) imageI lsubst-inv-inv2 subset-eq)
qed

```

```

lemma FL-ty-lmap:
  assumes A1: FLty ty = set xs
  assumes A2: length xs = length ys
  assumes A3: distinct xs
  shows FLty (lsbst-ty xs ys ty) = set ys
apply (cases ty)
apply (simp-all add: FLty-def)
using A1 A2 A3 FLty.simps(1) lsubst-ty-def lsubst-zip-image by auto

```

```

lemma FL-ty-lmap2:
  assumes A1: FLty ty = set (map fst xs)
  assumes A3: distinct (map fst xs)
  shows FLty (lmap-ty (lsbst xs) ty) = set (map snd xs)
by (metis A1 A3 FL-ty-lmap length-map lsubst-ty-def zip-map-fst-snd)

```

```

lemma FL-ty-lmap3:
  assumes FLty ty = set (map fst xs2)
  assumes set ys = set xs1  $\cup$  set xs2
  assumes distinct (map fst ys)
  assumes distinct (map fst xs2)
  shows FLty (lmap-ty (lsbst xs2) ty) = FLty (lmap-ty (lsbst ys) ty)
using assms
proof (cases ty)
  case (TBase x11 x12)
  then show ?thesis
  apply auto
  using assms(1,2,3,4) lsubst-lemma7 apply auto[1]
  using assms(1,2,3,4) lsubst-lemma7 apply auto[1]
  done
next
  case (TArrow x21 x22 x23)
  then show ?thesis
  apply auto
  using assms(1,2,3,4) lsubst-lemma7 apply auto[1]
  using assms(1,2,3,4) lsubst-lemma7 apply auto[1]
  done
next
  case (TLI x31 x32 x33 x34)
  then show ?thesis
  apply auto
  using assms(1,2,3,4) lsubst-lemma7 apply auto[1]

```

```

using assms(1,2,3,4) lsubst-lemma7 apply auto[1]
done
qed

```

```

lemma FL-typing-rel-eq-lemma1:
  assumes distinct (map fst xs)
  assumes distinct (map fst xs2)
  assumes FL G = set (map fst xs2)
  assumes set xs2  $\subseteq$  set xs
  shows FL (lmap (lsbst xs) G) = FL (lmap (lsbst xs2) G)
by (metis assms(1,2,3,4) lmap-lim lsubst-lemma7)

```

```

lemma FL-typing-rel-eq-lemma2:
  assumes distinct (map fst xs)
  assumes distinct (map fst xs2)
  assumes FLty ty = set (map fst xs2)
  assumes set xs2  $\subseteq$  set xs
  shows FLty (lmap-ty (lsbst xs) ty) = FLty (lmap-ty (lsbst xs2) ty)
using FL-ty-lmap3 assms(1,2,3,4) by blast

```

```

lemma FL-typing-rel-eq-lemma3:
  assumes distinct (map fst xs)
  assumes distinct (map fst xs2)
  assumes FL G = FLty ty
  assumes FL G = set (map fst xs2)
  assumes set xs2  $\subseteq$  set xs
  shows FL (lmap (lsbst xs) G) = FLty (lmap-ty (lsbst xs) ty)
by (metis FL-lmap-commute FL-ty-lmap2 FL-typing-rel-eq-lemma1
  FL-typing-rel-eq-lemma2 assms(1,2,3,4,5) lsubst-image)

```

```

lemma FL-typing-rel-eq-lemma4:
  assumes distinct (map fst xs)
  assumes FLG  $\subseteq$  set (map fst xs)
  assumes xs2 = filter ( $\lambda x. \text{fst } x \in \text{FLG}$ ) xs
  shows (distinct (map fst xs2)
     $\wedge$  set xs2  $\subseteq$  set xs
     $\wedge$  FLG = set (map fst xs2))
apply auto
using assms(1,3) distinct-map-filter apply blast
apply (simp add: assms(3))
using assms(2,3) apply auto[1]
by (simp add: assms(3))

```

```

lemma FL-typing-rel-eq-lemma5:
  assumes distinct (map fst xs)
  assumes FLG  $\subseteq$  set (map fst xs)
  shows  $\exists xs2. (\text{distinct } (\text{map fst } xs2)
    \wedge \text{set } xs2 \subseteq \text{set } xs
    \wedge \text{FLG} = \text{set } (\text{map fst } xs2))$ 
by (meson FL-typing-rel-eq-lemma4 assms(1,2))

```

```

lemma FL-typing-rel-eq-lemma6:
  assumes distinct (map fst xs)
  assumes FL G = FLty ty
  assumes FL G  $\subseteq$  set (map fst xs)
  shows FL (lmap (lsbst xs) G) = FLty (lmap-ty (lsbst xs) ty)
by (meson FL-typing-rel-eq-lemma3 FL-typing-rel-eq-lemma5
  assms(1,2,3))

```

```

lemma FL-typing-rel-eq-lemma7:
  assumes set xs = ALty ty
  assumes length xs = length ys
  assumes distinct xs
  assumes distinct ys
  assumes FL G  $\subseteq$  set xs
  assumes FL G = FLty ty
  shows FL (lmap (lsubst (zip xs ys)) G) = FLty (lsubst-ty xs ys ty)
by (metis FL-typing-rel-eq-lemma6 assms(2,3,5,6) lsubst-ty-def
    map-fst-zip)

lemma FL-typing-rel-eq-lemma8:
  assumes FL T = set ys
  assumes set xs  $\subseteq$  FL T
  shows FL (lbinds (filter ( $\lambda y. y \notin$  set xs) ys) T) = set xs
by (metis Diff-Diff-Int FL-lbinds Int-commute Un-Int-eq(4) assms(1,2)
    set-diff-eq set-filter sup.order-iff)

lemma lsubst-ideal-aux:
  assumes s = s1 @ s2
  assumes distinct (map fst s)
  assumes map fst s1 = map snd s1
  shows lsubst s z = lsubst s2 z
using assms
proof (induct s1 arbitrary: s2 z s)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a s3)
  then show ?case
  apply auto
  by (metis list.set-map lsubst-ignore lsubst-simp map-eq-conv)
qed

lemma lsubst-ideal:
  assumes set s = set s1  $\cup$  set s2
  assumes distinct (map fst s)
  assumes distinct (map fst s2)
  assumes map fst s1 = map snd s1
  shows lsubst s z = lsubst s2 z
by (smt (verit) UnE Un-upper2 assms(1,2,3,4) image-iff list.set-map
    lsubst-ignore lsubst-lemma5 lsubst-lemma7 map-eq-conv
    surjective-pairing)

lemma lmap-union-ideal:
  assumes set s = set s1  $\cup$  set s2
  assumes distinct (map fst s)
  assumes distinct (map fst s2)
  assumes map fst s1 = map snd s1
  shows lmap (lsubst s) g = lmap (lsubst s2) g
using assms(1,2,3,4) lsubst-ideal by presburger

lemma lmap-ty-union-ideal:
  assumes set s = set s1  $\cup$  set s2
  assumes distinct (map fst s)
  assumes distinct (map fst s2)
  assumes map fst s1 = map snd s1
  shows lmap-ty (lsubst s) ty = lmap-ty (lsubst s2) ty
using assms(1,2,3,4) lsubst-ideal by presburger

```

```

lemma FL-typing-LRHS:
  assumes  $\Gamma \vdash \{P\} \ G : ty$ 
  assumes  $G = \text{Graph } g$ 
  shows  $FL\ g = FLty\ ty$ 
using assms
proof (induction arbitrary:  $g$  set: typing)
  case (TyVar  $i \ \Gamma \ xs \ ty \ ys \ P$ )
  then show ?case
  using FL-ty-lmap2 by force
next
  case (TyArrow  $xs \ ty1 \ \Gamma \ P \ e \ ty2 \ ys$ )
  then show ?case
  apply auto
  done
next
  case (TyApp  $\Gamma \ P \ e1 \ ty2 \ ty1 \ xs \ e2$ )
  then show ?case
  apply auto
  done
next
  case (TyProd  $a \ xs \ rhs \ P \ taus \ Ts \ \Gamma$ )
  have  $\forall i < \text{length } Ts. FLty\ (\text{taus-of-rhs } rhs \ ! \ i) = FL\ (Ts \ ! \ i)$ 
  by (simp add: TyProd.IH TyProd.hyps(2))
  then have  $P2: \text{map } FLty\ (\text{taus-of-rhs } rhs) = \text{map } FL\ Ts$ 
  using TyProd.hyps(2,3) map-equality-iff by blast
  then show ?case
  apply auto
  using TyProd.hyps(4) TyProd.prem1 app-prodrule-FL-equiv apply blast
  using TyProd.hyps(4) TyProd.prem1 app-prodrule-FL-equiv by blast
next
  case (TyCong  $\Gamma \ P \ g1 \ ty \ g2$ )
  then show ?case
  apply simp
  by (simp add: freelinks-equiv)
next
  case (TyAlpha  $\Gamma2 \ P \ G \ ty \ xs \ ys$ )
  then show ?case
  by (metis FL-typing-rel-eq-lemma7 exp.inject(1))
next
  case (TyCase  $\Gamma \ P \ e0 \ ty1 \ \Gamma2 \ e1 \ ty2 \ e2 \ T$ )
  then show ?case
  by auto
next
  case (TyLIIntro  $lhs \ rhs \ P \ \Gamma$ )
  then show ?case
  apply auto
  apply (metis TyLIIntro-freelinks-equiv surj-pair)
  apply (metis TyLIIntro-freelinks-equiv surj-pair)
  done
next
  case (TyLITrans  $\Gamma \ P \ T1 \ a2 \ xs2 \ tys1 \ a1 \ xs1 \ ys1 \ T2 \ tys2 \ ys2 \ ys$ )
  then show ?case
  by force
next
  case (TyLIIntro0  $\Gamma \ P \ T \ a \ xs$ )
  then show ?case
  by (simp add: fset-of-list.rep-eq)
next
  case (TyLIElim0  $ys \ ys' \ xs \ T \ \Gamma \ P \ a$ )
  then show ?case
  by (metis FL-typing-rel-eq-lemma8 FLty.simps(1,3) exp.inject(1) fset-of-list.rep-eq)
qed

```

```

lemma LLty-union-ideal-aux:
  assumes finite S
  shows  $\exists ys. (set (map fst ys) = S$ 
     $\wedge distinct (map fst ys)$ 
     $\wedge distinct (map snd ys)$ 
     $\wedge map fst ys = map snd ys)$ 
proof -
  obtain xs where xs:
    set xs = S distinct xs
  using assms by (meson finite-distinct-list)

  let ?ys = map ( $\lambda x. (x, x)$ ) xs

  have P1: set (map fst ?ys) = S using xs by auto
  moreover have distinct (map fst ?ys)
    by (simp add: map-idl xs(2))
  moreover have P2: map fst ?ys = map snd ?ys
    by auto
  ultimately show ?thesis
    apply auto
    by (metis (no-types, lifting) P1 P2 list.map-comp list.set-map
      map-eq-conv)
qed

```

```

lemma LLty-union-ideal:
  assumes A1: distinct (map fst xs)
  assumes A2: distinct (map snd xs)
  assumes A3: set (map fst xs) = FLty ty
  assumes A5: set (map snd xs)  $\cap$  ALty ty - FLty ty = {}
  shows  $\exists ys. (set (map fst ys) = ALty ty$ 
     $\wedge distinct (map fst ys)$ 
     $\wedge distinct (map snd ys)$ 
     $\wedge lmap\text{-}ty (lsubst ys) ty = lmap\text{-}ty (lsubst xs) ty$ 
     $\wedge lmap (lsubst ys) G = lmap (lsubst xs) G)$ 
proof -
  have finite (ALty ty - FLty ty)
    by (simp add: finite-ALty)
  then obtain ys ys1 where
    P3: set (map fst ys1) = ALty ty - FLty ty
     $\wedge distinct (map fst ys1)$ 
     $\wedge distinct (map snd ys1)$ 
     $\wedge ys = ys1 @ xs$ 
     $\wedge map fst ys1 = map snd ys1$ 
    by (meson LLty-union-ideal-aux)

  have set (map fst ys1)  $\cap$  set (map fst xs) = {}
    by (metis Diff-disjoint P3 A3 inf-commute)
  then have P5: distinct (map fst ys)
    by (simp add: P3 A1)

  have set (map snd ys1)  $\cap$  set (map snd xs) = {}
    by (metis Int-Diff P3 A5 inf-commute)
  then have P7: distinct (map snd ys)
    by (simp add: P3 A2)

  have P10: set (map fst ys) = ALty ty
  proof -
    have set (map fst ys) =
      set (map fst ys1)  $\cup$  set (map fst xs)
      by (simp add: P3)
    also have  $\dots = (ALty ty - FLty ty) \cup FLty ty$ 
      using A3 P3 by blast
    also have  $\dots = ALty ty$ 
      using FLty-subset-ALty by blast
  qed

```

```

    finally show ?thesis .
qed

have P8: lmap-ty (lsubst ys) ty = lmap-ty (lsubst xs) ty
  using P3 P5 lsubst-ideal-aux by presburger
have P9: lmap (lsubst ys) G = lmap (lsubst xs) G
  using P3 P5 lsubst-ideal-aux by presburger

show ?thesis
  using P10 P5 P7 P8 P9 by blast
qed

lemma TyAlphaWhenTyVar:
  assumes A0:  $\Gamma \vdash \{P\}$  Graph  $g : ty$ 
  assumes A1:  $set (map fst xs) = FLty\ ty$ 
  assumes A2:  $distinct (map fst xs)$ 
  assumes A3:  $distinct (map snd xs)$ 
  assumes A4:  $set (map snd xs) \cap ALty\ ty - FLty\ ty = \{\}$ 
  shows  $\Gamma \vdash \{P\}$  Graph (lmap (lsubst xs) g) : lmap-ty (lsubst xs) ty
proof -
  obtain ys where
    ys:  $set (map fst ys) = ALty\ ty$ 
     $\wedge distinct (map fst ys)$ 
     $\wedge distinct (map snd ys)$ 
     $\wedge lmap-ty (lsubst ys) ty = lmap-ty (lsubst xs) ty$ 
     $\wedge lmap (lsubst ys) g = lmap (lsubst xs) g$ 
  using assms LLty-union-ideal by metis

  then have  $\Gamma \vdash \{P\}$  Graph (lmap (lsubst ys) g) : lmap-ty (lsubst ys) ty
    by (metis A0 FL-typing-LRHS FLty-subset-ALty TyAlpha length-map
      lsubst-ty-def zip-map-fst-snd)
  then have  $\Gamma \vdash \{P\}$  Graph (lmap (lsubst xs) g) : lmap-ty (lsubst xs) ty
    using ys by argo
  then show ?thesis .
qed

lemma subst-make-diffgraph-ignore:
  subst i xs g2 (Graph (make-diffgraph rhs)) = Graph (make-diffgraph rhs)
apply (simp add: make-diffgraph-def)
by (simp add: split-beta subst-graph-Mols1-equiv
  subst-graph-fusions-of)

lemma subst-mol-distribute:
  subst i xs g2 (Graph (Mol T1 T2)) =
  Graph (Mol (subst-graph i xs g2 T1) (subst-graph i xs g2 T2))
by simp

lemma subst-graph-lbind-com:
  assumes set xs = FL G
  shows subst-graph i xs G (lbind x g) = lbind x (subst-graph i xs G g)
apply (simp add: lbind-def)
by (simp add: assms subst-graph-lmap-commute)

lemma subst-graph-lbinds-com:
  assumes set xs = FL G
  shows subst-graph i xs G (lbinds ys g) = lbinds ys (subst-graph i xs G g)
using assms
apply (simp add: lbinds-def)

```

```

proof (induct ys arbitrary: g i xs G)
  case Nil
  then show ?case
  by simp
next
  case (Cons a ys)
  then show ?case
  apply auto
  by (simp add: subst-graph-lbind-com)
qed

```

```

lemma ALty-lmap:
  assumes A1: ALty ty = set (map fst xs)
  assumes A2: set xs  $\subseteq$  set ys
  assumes A3: distinct (map fst ys)
  assumes A4: distinct (map fst xs)
  shows ALty (lmap-ty (lsubst ys) ty) = set (map snd xs)
using assms
proof (induct ty arbitrary: xs ys)
  case (TBase x1 x2)
  then show ?case
  apply auto
  apply (metis imageI lsubst-lemma5 snd-conv subset-code(1))
  by (metis fst-conv image-eqI lsubst-lemma5 subset-iff)
next
  case (TArrow ty1 ty2 x3)
  then show ?case
  apply auto
  apply (metis image-eqI lsubst-lemma5 snd-eqD subsetD)
  by (metis fst-conv imageI lsubst-lemma5 subset-eq)
next
  case (TLI tys a vs zs)
  have P1:
     $\bigcup$  (set-mset (image-mset ALty tys))  $\cup$  set vs  $\cup$  fset zs
    = set (map fst xs)
  using ALty.simps(3) TLI.prem(1) by presburger

  have P45:
    ALty (lmap-ty (lsubst xs) (TLI tys a vs zs)) =
    ALty (lmap-ty (lsubst ys) (TLI tys a vs zs))
  using TLI.prem(1,2,3,4) lmap-ty-lim5 by presburger

```

obtain xs-tys **where**

```

xs-tys: xs-tys = filter ( $\lambda x$ . fst x  $\in$   $\bigcup$  (set-mset (image-mset ALty tys))) xs
by presburger

```

have xs-tys-P1: set xs-tys $\subseteq$ set xs

using xs-tys **by** auto

have xs-tys-P2: distinct (map fst xs-tys)

using TLI.prem(4) distinct-map-filter xs-tys **by** blast

have xs-tys-P3:

$\bigcup$ (set-mset (image-mset ALty tys)) = set (map fst xs-tys)

using xs-tys **apply** auto

using P1 **by** fastforce

obtain sigma-of-tau **where**

sigma-of-tau: sigma-of-tau = (λty . filter (λx . fst x $\in$ ALty ty) xs-tys)

by presburger

have

```

  ∀ ty ∈# tys. ALty ty ⊆ set (map fst xs-tys)
  using xs-tys-P3 by auto
then have sigma-of-tau-P1:
  ∀ ty ∈# tys. (ALty ty = set (map fst (sigma-of-tau ty)))
  apply (simp-all add: sigma-of-tau)
  by auto

then have P9:
  (⋃ ty ∈ set-mset tys. set (sigma-of-tau ty)) = set xs-tys
  apply auto
  apply (simp-all add: sigma-of-tau)
  by (simp add: xs-tys)
then have P20:
  (⋃ ty ∈ set-mset tys. (set (map snd (sigma-of-tau ty)))) =
    set (map snd xs-tys)
  by (metis (no-types, lifting) ext image-UN image-set)

have sigma-of-tau-P2:
  ∀ ty ∈# tys. ∀ xs-ty.
    (xs-ty = sigma-of-tau ty --> (set xs-ty ⊆ set xs-tys
    ∧ distinct (map fst xs-ty)
    ∧ ALty ty = set (map fst xs-ty)))
  apply auto
  apply (simp add: sigma-of-tau)
  apply (simp add: distinct-map-filter sigma-of-tau xs-tys)
  using TLI.prem1(4) distinct-map-filter apply blast
  apply (simp add: sigma-of-tau-P1)
  by (simp add: sigma-of-tau)

have P4:
  ∀ ty ∈# tys.
    ALty (lmap-ty (lsubst (sigma-of-tau ty)) ty) = set (map snd (sigma-of-tau ty))
  by (simp add: TLI.hyps sigma-of-tau-P2)
then have P6:
  ∀ ty ∈# tys.
    ALty (lmap-ty (lsubst xs) ty) = set (map snd (sigma-of-tau ty))
  by (meson TLI.hyps TLI.prem1(4) order-trans sigma-of-tau-P2
    xs-tys-P1)
then have P7:
  ∀ ty ∈# tys.
    ALty (lmap-ty (lsubst ys) ty) = set (map snd (sigma-of-tau ty))
  by (meson TLI.hyps TLI.prem1(2,3) sigma-of-tau-P2 subset-trans
    xs-tys-P1)
then have P8:
  (⋃ ty ∈ set-mset tys. (ALty (lmap-ty (lsubst ys) ty))) =
  (⋃ ty ∈ set-mset tys. (set (map snd (sigma-of-tau ty))))
  by blast
then have P12:
  (⋃ ty ∈ set-mset tys. (ALty (lmap-ty (lsubst ys) ty))) =
    set (map snd xs-tys)
  using P20 by order
then have P14:
  ⋃ (set-mset (image-mset ALty (image-mset (lmap-ty (lsubst ys)) tys))) =
    set (map snd xs-tys)
  by auto

have vs-aux1: set (map fst ((filter (λx. fst x ∈ set vs) xs))) = set vs
  apply auto
  using P1 list.set-map by auto
have vs-aux-distinct: distinct (map fst ((filter (λx. fst x ∈ set vs) xs)))
  by (simp add: TLI.prem1(4) distinct-map-filter)
then have vs-aux1:
  set (map (lsubst (filter (λx. fst x ∈ set vs) xs)) vs) =
    set (map snd (filter (λx. fst x ∈ set vs) xs))
  by (metis (lifting) image-set lsubst-image vs-aux1)
have vs-cdom2:

```

```

map (lsubst (filter (λx. fst x ∈ set vs) xs)) vs = map (lsubst xs) vs
apply auto
by (metis lsubst-filter)
then have vs-cdom:
  set (map (lsubst xs) vs) =
    set (map snd (filter (λx. fst x ∈ set vs) xs))
using vs-aux1 by presburger
then have vs-cdom3:
  (lsubst xs) ‘ set vs =
    set (map snd (filter (λx. fst x ∈ set vs) xs))
by auto

have zs-aux1: set (map fst ((filter (λx. fst x ∈ fset zs) xs))) = fset zs
apply auto
using P1 list.set-map by auto
have zs-aux-distinct: distinct (map fst ((filter (λx. fst x ∈ fset zs) xs)))
by (simp add: TLI.prem3(4) distinct-map-filter)
then have zs-aux1:
  fset ((lsubst (filter (λx. fst x ∈ fset zs) xs)) |† zs) =
    set (map snd (filter (λx. fst x ∈ fset zs) xs))
by (metis fimage.rep-eq lsubst-image zs-aux1)
have zs-cdom2:
  (lsubst (filter (λx. fst x ∈ fset zs) xs)) |† zs = (lsubst xs) |† zs
by (metis dual-order.refl fmap-lsubst-filter)
then have zs-cdom:
  fset ((lsubst xs) |† zs) =
    set (map snd (filter (λx. fst x ∈ fset zs) xs))
using zs-aux1 by presburger
then have zs-cdom3:
  (lsubst xs) ‘ fset zs =
    set (map snd (filter (λx. fst x ∈ fset zs) xs))
by auto

have P50:
  set (map snd xs-tys) =
    (⋃ (set-mset (image-mset ALty (image-mset (lmap-ty (lsubst xs)) tys))))
apply auto
apply (metis (no-types, lifting) P12 P6 P8 UN-E imageI list.set-map snd-eqD)
using P12 P6 P8 by auto

have P40:
  set xs =
    set (filter (λx. fst x ∈ ⋃ (set-mset (image-mset ALty tys))) xs) ∪
    set (filter (λx. fst x ∈ set vs) xs) ∪
    set (filter (λx. fst x ∈ fset zs) xs)
using P1 by auto
then have P41:
  set xs =
    set xs-tys ∪
    set (filter (λx. fst x ∈ set vs) xs) ∪
    set (filter (λx. fst x ∈ fset zs) xs)
using xs-tys by presburger
then have P42:
  set (map snd xs) =
    set (map snd xs-tys) ∪
    set (map snd (filter (λx. fst x ∈ set vs) xs)) ∪
    set (map snd (filter (λx. fst x ∈ fset zs) xs))
by auto
then have P43:
  set (map snd xs) =
    (⋃ (set-mset (image-mset ALty (image-mset (lmap-ty (lsubst xs)) tys)))) ∪
    lsubst xs ‘ fset zs ∪
    lsubst xs ‘ set vs
using P50 vs-cdom3 zs-cdom3 by force
then have P44:
  set (map snd xs) = ALty (lmap-ty (lsubst xs) (TLI tys a vs zs))
using ALty.simps(3) P42 P50 lmap-ty.simps(3) vs-cdom zs-cdom by presburger

```

```

then show ?case
  using P45 by presburger
qed

```

17 Soundness

17.1 Substitution Lemma

We first show that lifting preserves well-typedness. The lifting operation shifts free variable indices in an expression. This lemma shows that lifting preserves typing: if e is well-typed in environment Γ , then lifting e by i yields a well-typed term in the environment extended by inserting τ' at position i . This property is crucial for the substitution lemma, particularly under the abstraction case.

```

lemma lift-type:
  assumes  $T: \Gamma \vdash \{P\} \ e : ty$ 
  and  $LE: i \leq \text{length } \Gamma$ 
  shows  $\text{insert-at } \Gamma \ i \ (ys, ty2) \vdash \{P\} \ (\text{lift } i \ e) : ty$ 
using T LE
proof (induction arbitrary: i ys ty2 rule: typing.induct)
  case (TyVar j  $\Gamma$  xs s ys ty P)
  note j-lt = TyVar.hyps(1)
  note lookup = TyVar.hyps(2)
  note len-vs = TyVar.hyps(3)
  note le-i = TyVar.prem1

  show ?case
proof (cases  $j < i$ )
  case True
  then have P1:  $\text{lift-atom } i \ (GVar \ j) = GVar \ j$ 
    by simp

  have len-ins:  $\text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2)) = \text{length } \Gamma + 1$ 
    by (simp add: length-insert-at)

  from j-lt have j-lt':  $j < \text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2))$ 
using True le-i len-ins by linarith

  from le-i True have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! j = \Gamma ! j$ 
    by (simp add: shift-gt)

  with lookup have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! j = (xs, ty)$ 
    using TyVar.hyps(4) by force

  with j-lt' len-vs
  show ?thesis
    by (metis P1 TyVar.hyps(5,6,7,8) lift.simps(1) lift-graph.simps(2) lookup typing.TyVar)
next
  case False
  then have i-le-j:  $i \leq j$  by simp

  have P1:  $\text{lift-atom } i \ (GVar \ j) = GVar \ (Suc \ j)$ 
    by (simp add: False)

  have len-ins:  $\text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2)) = \text{length } \Gamma + 1$ 
    by (simp add: length-insert-at)

  from j-lt have  $Suc \ j \leq \text{length } \Gamma$ 
    by simp
  then have Sucj-lt-ins:  $Suc \ j < \text{length } (\text{insert-at } \Gamma \ i \ (ys, ty2))$ 
    by (simp add: len-ins)

  from i-le-j have  $i < Suc \ j$  by simp
  then have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! Suc \ j = \Gamma ! j$ 
    by (simp add: shift-lt)

  with lookup have  $(\text{insert-at } \Gamma \ i \ (ys, ty2)) ! Suc \ j = (xs, ty)$ 
    using TyVar.hyps(4) by presburger

```

```

with Sucj-lt-ins len-vs
show ?thesis
by (metis P1 TyVar.hyps(5,6,7,8) lift.simps(1) lift-graph.simps(2) lookup typing.TyVar)
qed

next
case (TyLIIntro lhs rhs P  $\Gamma$ )
then show ?case
by (simp add: lift-graph-make-diffgraph-equiv typing.TyLIIntro)

next
case (TyLITrans  $\Gamma$  P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2)
then show ?case
by (metis lift.simps(1) lift-graph.simps(4) typing.TyLITrans)

next
case (TyLIElim0 zs ys' xs T  $\Gamma$  P a)
have insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$  lift i (Graph T) : TLI  $\{\#\}$  a xs zs
using TyLIElim0.IH TyLIElim0.prem by blast
then have insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$  Graph (lift-graph i T) : TLI  $\{\#\}$  a xs zs
by auto
then have insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$ 
Graph (lbinds (filter ( $\lambda y. y \notin \text{set } xs$ ) ys') (lift-graph i T)) : TBase a xs
by (simp add: TyLIElim0.hyps(1,2) lift-graph-freelinks typing.TyLIElim0)
then have insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$ 
Graph (lift-graph i (lbinds (filter ( $\lambda y. y \notin \text{set } xs$ ) ys') T)) : TBase a xs
by (simp add: lift-graph-lbinds-equiv)
then have insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$ 
lift i (Graph (lbinds (filter ( $\lambda y. y \notin \text{set } xs$ ) ys') T)) : TBase a xs
by simp
then show ?case
by blast

next
case (TyLIIntro0  $\Gamma$  P T a xs)
then show ?case
by (simp add: typing.TyLIIntro0)

next
case (TyArrow xs ty1  $\Gamma$  P e ty2' vs)
note IH = TyArrow.IH
note le = TyArrow.prem

have Suc-le: Suc i  $\leq$  length ((xs, ty1)  $\#$   $\Gamma$ )
using le by simp

have body-typed:
insert-at ((xs, ty1)  $\#$   $\Gamma$ ) (Suc i) (ys, ty2)  $\vdash \{P\}$  lift (Suc i) e : ty2'
using IH Suc-le by blast

have env-eq:
insert-at ((xs, ty1)  $\#$   $\Gamma$ ) (Suc i) (ys, ty2)
= insert-at (insert-at  $\Gamma$  i (ys, ty2)) 0 (xs, ty1)
by (simp add: shift-commute)

have (xs, ty1)  $\#$  insert-at  $\Gamma$  i (ys, ty2)  $\vdash \{P\}$  lift (Suc i) e : ty2'
using body-typed by auto

then show ?case
by (simp add: TyArrow.hyps(2,3) typing.TyArrow)

next
case (TyApp  $\Gamma$  P e1 ty3 ty1 xs e2)
note IH1 = TyApp.IH(1)
note IH2 = TyApp.IH(2)
note le = TyApp.prem

have
insert-at  $\Gamma$  i (ys, ty2')  $\vdash \{P\}$  lift i e1 : TArrow ty3 ty1 xs

```

```

insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2'$ )  $\vdash \{P\}$  lift  $i$   $e2$  :  $ty3$ 
  apply (simp add: IH1 le)
  by (simp add: IH2 le)

then show ?case
  apply auto
  by (meson IH1 IH2 le typing.TyApp)

next
case (TyCong  $\Gamma$   $P$   $g1$   $ty$   $g2$ )
note IH = TyCong.IH
note cong = TyCong.hyps(2)
note le = TyCong.premis

have insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2'$ )  $\vdash \{P\}$  lift  $i$  ( $Graph$   $g1$ ) :  $ty$  for  $ys$   $ty2'$ 
  using IH le by blast
then show ?case
  by (metis lift.simps(1) lift-graph-cong local.cong typing.TyCong)

next
case (TyAlpha  $\Gamma2$   $P$   $g$   $ty$   $xs$   $vs$ )
note IH = TyAlpha.IH
note le = TyAlpha.premis(1)
note xs-FL = TyAlpha.hyps(2)
note len-xs-vs = TyAlpha.hyps(3)

have insert-at  $\Gamma2$   $i$  ( $ys$ ,  $ty2'$ )  $\vdash \{P\}$  lift  $i$  ( $Graph$   $g$ ) :  $ty$  for  $ys$   $ty2'$ 
  using IH le by blast

then show ?case
  by (metis TyAlpha.hyps(4,5,6) len-xs-vs lift.simps(1) lift-graph-freelinks
    lift-graph-lmap-commute typing.TyAlpha xs-FL)

next
case (TyProd  $a$   $xs$   $rhs$   $P$   $taus$   $Ts$   $\Gamma$ )

have P2:
  lift-graph  $i$  (app-prodrule  $Ts$   $rhs$ ) = app-prodrule (map (lift-graph  $i$ )  $Ts$ )  $rhs$ 
  by (simp add: lift-graph-app-prodrule-equiv)

then show ?case
  using P2 TyProd.IH TyProd.hyps(1,2,3,4) TyProd.premis typing.TyProd by auto

next
case (TyCase  $\Gamma$   $P$   $e0$   $ty1$   $\Gamma2$   $e1$   $ty2'$   $e2$   $T$ )
have IHe0: insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$   $e0$  :  $ty1$ 
  by (simp add: TyCase.IH(1) TyCase.premis)

have insert-at ( $\Gamma2$  @  $\Gamma$ ) ( $i$  + length  $\Gamma2$ ) ( $ys$ ,  $ty2$ )
   $\vdash \{P\}$  lift ( $i$  + length  $\Gamma2$ )  $e1$  :  $ty2'$ 
  by (simp add: TyCase.IH(2) TyCase.premis)
then have IHe1:
   $\Gamma2$  @ insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )
   $\vdash \{P\}$  lift ( $i$  + length  $\Gamma2$ )  $e1$  :  $ty2'$ 
  by (simp add: insert-at-shift)

have IHe2: insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$   $e2$  :  $ty2'$ 
  by (simp add: TyCase.IH(3) TyCase.premis)

have
  insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$ 
  Case (lift  $i$   $e0$ )  $\Gamma2$   $T$  (lift ( $i$  + length  $\Gamma2$ )  $e1$ ) (lift  $i$   $e2$ ) :  $ty2'$ 
  using IHe0 IHe1 IHe2 typing.TyCase by blast
then have insert-at  $\Gamma$   $i$  ( $ys$ ,  $ty2$ )  $\vdash \{P\}$  lift  $i$  (Case  $e0$   $\Gamma2$   $T$   $e1$   $e2$ ) :  $ty2'$ 
  by simp
then show ?case .

qed

```

```

lemma weakening-left:
  assumes A1:  $\Gamma \vdash \{P\} \ e : ty$ 
  shows  $x \# \Gamma \vdash \{P\} \ (\text{lift } 0 \ e) : ty$ 
by (metis assms insert-at.simps(1) lift-type surj-pair zero-le)

```

```

lemma lifts-graph2:
  ( $\text{lift } 0 \rightsquigarrow n$ ) (Graph g) = Graph ((lift-graph 0  $\rightsquigarrow$  n) g)
proof (induct n arbitrary: g)
  case 0
  then show ?case
  apply auto
  done
next
  case (Suc n)
  then show ?case
  apply auto
  done
qed

```

```

lemma weakening-lefts:
  assumes A1:  $\Gamma \vdash \{P\} \ e : ty$ 
  shows  $\Gamma 2 @ \Gamma \vdash \{P\} \ (\text{lift } 0 \rightsquigarrow \text{length } \Gamma 2) \ e : ty$ 
using assms
proof (induct  $\Gamma 2$  arbitrary: e)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a  $\Gamma 2$ )
  then show ?case
  apply auto
  by (simp add: weakening-left)
qed

```

```

lemma subst-lemma:
  fixes i :: nat
  and e1 :: type exp
  and g2 :: type graph
  and ty1 ty2 :: type
  and  $\Gamma 1 \ \Gamma 2 :: tyenv$ 
  assumes A1:  $\Gamma 1 \vdash \{P\} \ e1 : ty1$ 
  and A2:  $\Gamma 2 \vdash \{P\} \ \text{Graph } g2 : ty2$ 
  and A3:  $i \leq \text{length } \Gamma 2$ 
  and A4:  $\Gamma 1 = \text{insert-at } \Gamma 2 \ i \ (xs, ty2)$ 
  and A5:  $\text{set } xs = FL \ g2$ 
  and A6:  $\text{distinct } xs$ 
  shows  $\Gamma 2 \vdash \{P\} \ \text{subst } i \ xs \ g2 \ e1 : ty1$ 
using assms
proof (induct arbitrary:  $\Gamma 2 \ i \ ty2 \ g2 \ xs \ \text{set: typing}$ )
  case (TyVar j  $\Gamma \ xs' \ s \ ys \ ty' \ P$ )
  note TV = TyVar

  have len- $\Gamma 1$ :  $\text{length } \Gamma = \text{length } \Gamma 2 + 1$ 
  by (simp add: TyVar.prem(3) length-insert-at)

  have subst-GVar:
    subst i xs g2 (Graph (Atom (GVar j) ys)) =
      (if  $i < j$  then Graph (Atom (GVar (j - 1)) ys)
       else if  $j = i \wedge \text{length } xs = \text{length } ys$ 
        then Graph (lmap (lsubst (zip xs ys)) g2)

```

```

    else Graph (Atom (GVar j) ys))

by simp

consider (lt) i < j | (eq) i = j | (gt) j < i
  by arith
then show ?case
proof cases
  case lt

    then have subst-ij:
      subst i xs g2 (Graph (Atom (GVar j) ys))
      = Graph (Atom (GVar (j - 1)) ys)
    by (simp add: subst-GVar)

    from TyVar.hyps(1) TyVar.prem(4) len-Γ1 lt
    have j1-lt-len-Γ2: j - 1 < length Γ2
    by (simp add: less-diff-conv)

    from TyVar.prem(3) lt
    have lookup-shift:
      Γ ! j = Γ2 ! (j - 1)
    by (simp add: shift-lt)

    have P2B: set ys ∩ ALty ty' - FLty ty' = {}
    by (simp add: TyVar.hyps(8))

    from TyVar.hyps(4) lookup-shift
    have P2: Γ2 ! (j - 1) = (xs', ty')
    by simp

    moreover have length xs' = length ys
    by (simp add: TyVar.hyps(2,3))

    ultimately have
      Γ2 ⊢ {P} Graph (Atom (GVar (j - 1)) ys) : lmap-ty (lsubst (zip xs' ys)) ty'
    by (metis P2B TyVar.hyps(2,3,5,6,7) j1-lt-len-Γ2 lsubst-ty-def typing.TyVar
      zip-map-fst-snd)

    thus ?thesis
      using TyVar.hyps(2,3,5,6,7) P2 j1-lt-len-Γ2 subst-ij typing.TyVar P2B
    by presburger
next
  case eq
  have subst-eq:
    subst i xs g2 (Graph (Atom (GVar j) ys)) =
      (if length xs = length ys
       then Graph (lmap (lsubst (zip xs ys)) g2)
       else Graph (Atom (GVar j) ys))
    by (simp add: eq)

    from TyVar.hyps(2) TyVar.prem(4) eq
    have (xs', ty') = (xs, ty2)
  proof -
    from TyVar.prem(3)
    have Γ ! j = (insert-at Γ2 i (xs, ty2)) ! j
    by simp
    with eq TyVar.hyps(2) show ?thesis
      using TyVar.hyps(4) TyVar.prem(2) shift-eq by fastforce
  qed
  then have xs'-xs: xs' = xs and ty'-ty2: ty' = ty2
  by auto

  have len-xs-ys: length xs = length ys
  using TyVar.hyps(2,3) xs'-xs by force

```

```

have P2B: set ys  $\cap$  ALty ty' - FLty ty' = {}
  by (simp add: TyVar.hyps(8))

have subst-eq':
  subst i xs g2 (Graph (Atom (GVar j) ys))
    = Graph (lmap (lsubst (zip xs ys)) g2)
  using subst-eq len-xs-ys by simp

have P3: s = zip xs ys
  by (metis TyVar.hyps(2,3) xs'-xs zip-map-fst-snd)

have  $\Gamma 2 \vdash \{P\}$  Graph (lmap (lsubst s) g2) : lmap-ty (lsubst s) ty2
  using P2B TyAlphaWhenTyVar TyVar.hyps(2,3,5,6,7) TyVar.prem(1) ty'-ty2
  by auto

then show ?thesis
  using subst-eq' ty'-ty2 xs'-xs P2B P3
  by argo
next
case gt

have subst-gt:
  subst i xs g2 (Graph (Atom (GVar j) ys))
    = Graph (Atom (GVar j) ys)
  using gt subst-GVar by simp

from TyVar.prem(4) gt
have lookup-same:
   $\Gamma ! j = \Gamma 2 ! j$ 
  by (simp add: TyVar.prem(2,3) shift-gt)

from TyVar.hyps(2) lookup-same
have lookup2:  $\Gamma 2 ! j = (xs', ty')$ 
  using TyVar.hyps(4) by argo

have P2B: set ys  $\cap$  ALty ty' - FLty ty' = {}
  by (simp add: TyVar.hyps(8))

have j-gt-len- $\Gamma 2$ :  $j < \text{length } \Gamma 2$ 
proof -
  from TyVar.hyps(1) len- $\Gamma 1$  have  $j < \text{length } \Gamma 2 + 1$  by simp
  with gt show ?thesis
  using TyVar.prem(2) dual-order.strict-trans1 by blast
qed

have length xs' = length ys
  using TyVar.hyps(3)
  by (simp add: TyVar.hyps(2))

then have  $\Gamma 2 \vdash \{P\}$  Graph (Atom (GVar j) ys) : lsubst-ty xs' ys ty'
  by (simp add: P2B TyVar.hyps(5,6,7) j-gt-len- $\Gamma 2$  lookup2 lsubst-ty-def
    typing.TyVar)

thus ?thesis
  using P2B TyVar.hyps(2,3,5,6,7) j-gt-len- $\Gamma 2$  lookup2 subst-gt typing.TyVar
  by auto
qed
next
case (TyApp  $\Gamma$  P e1 ty2 ty1 xs e2)
show ?case
apply auto
by (meson TyApp.hyps(2,4) TyApp.prem(1,2,3,4,5) typing.TyApp)

next
case (TyProd a zs rhs P taus Ts  $\Gamma$ )
have P1: subst-graph i xs g2 (app-prodrule Ts rhs) =

```

app-prodrule (*map* (*subst-graph* *i* *xs* *g2*) *Ts*) *rhs*
by (*simp* *add*: *subst-typrod*)

have *P10*:

$\forall j < \text{length } Ts. \Gamma 2 \vdash \{P\} \text{ subst } i \text{ xs } g2 \text{ (Graph (Ts ! j)) : taus ! j}$

proof (*intro* *allI* *impI*)

fix *j* **assume** *Hj*: $j < \text{length } Ts$

from *TyProd.hyps*(4)[*rule-format*, *of* *j*] *Hj*

have *IHj*:

$\Gamma \vdash \{P\} \text{ Graph } (Ts ! j) : \text{taus ! j} \wedge$

$(\forall x \text{ xa } xb \text{ xc.}$

$x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$

$(\forall xd. \text{set } xd = FL \text{ xc} \longrightarrow$

$\text{set } xd = FLty \text{ xb} \longrightarrow$

$\Gamma = \text{insert-at } x \text{ xa } (xd, xb) \longrightarrow$

$\text{distinct } xd \longrightarrow$

$x \vdash \{P\} \text{ subst } xa \text{ xd } xc \text{ (Graph (Ts ! j)) : taus ! j}))$

by *auto*

then have *IHj-subst*:

$\forall x \text{ xa } xb \text{ xc.}$

$x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$

$(\forall xd. \text{set } xd = FL \text{ xc} \longrightarrow$

$\text{set } xd = FLty \text{ xb} \longrightarrow$

$\Gamma = \text{insert-at } x \text{ xa } (xd, xb) \longrightarrow$

$\text{distinct } xd \longrightarrow$

$x \vdash \{P\} \text{ subst } xa \text{ xd } xc \text{ (Graph (Ts ! j)) : taus ! j})$

by *auto*

show $\Gamma 2 \vdash \{P\} \text{ subst } i \text{ xs } g2 \text{ (Graph (Ts ! j)) : taus ! j}$

using *Hj* *TyProd.prem*s(1,2,3,4,5)

$\langle j < \text{length } Ts \implies \Gamma \vdash \{P\} \text{ Graph } (Ts ! j) : \text{taus ! j} \wedge (\forall x \text{ xa } xb \text{ xc. } x \vdash \{P\} \text{ Graph } xc : xb \longrightarrow xa \leq \text{length } x \longrightarrow$

$(\forall xd. \Gamma = \text{insert-at } x \text{ xa } (xd, xb) \longrightarrow \text{set } xd = FL \text{ xc} \longrightarrow \text{distinct } xd \longrightarrow x \vdash \{P\} \text{ subst } xa \text{ xd } xc \text{ (Graph (Ts ! j)) : taus ! j})) \rangle$

by *presburger*

qed

then show *?case*

apply *auto*

using *P1* *TyProd.hyps*(1,2,3,5) *typing.TyProd* **by** *auto*

next

case (*TyCase* Γ *P* *e0* *ty1'* $\Gamma 2'$ *e1* *ty2'* *e2* *T*)

have *IHe0*: $\Gamma 2 \vdash \{P\} \text{ subst } i \text{ xs } g2 \text{ e0 : ty1'}$

using *TyCase.hyps*(2) *TyCase.prem*s(1,2,3,4,5) **by** *presburger*

have $\Gamma 2 \vdash \{P\} \text{ Graph } g2 : ty2$

by (*simp* *add*: *TyCase.prem*s(1))

then have *P5*: $\Gamma 2' @ \Gamma 2 \vdash \{P\} \text{ (lift } 0 \text{ } \rightsquigarrow \text{length } \Gamma 2') \text{ (Graph } g2) : ty2$

by (*simp* *add*: *weakening-lefts*)

have $i \leq \text{length } \Gamma 2$

by (*simp* *add*: *TyCase.prem*s(2))

then have *P4*: $i + \text{length } \Gamma 2' \leq \text{length } (\Gamma 2' @ \Gamma 2)$

by *auto*

have $\Gamma = \text{insert-at } \Gamma 2 \text{ } i \text{ (xs, ty2)}$

by (*simp* *add*: *TyCase.prem*s(3))

then have *IHe1*:

$\Gamma 2' @ \Gamma = \text{insert-at } (\Gamma 2' @ \Gamma 2) \text{ (i + length } \Gamma 2') \text{ (xs, ty2)}$

by (*simp* *add*: *insert-at-shift*)

then have *IHe1B*:

$\Gamma 2' @ \Gamma 2 \vdash \{P\}$

```

  subst (i + length  $\Gamma 2'$ ) xs ((lift-graph 0  $\rightsquigarrow$  (length  $\Gamma 2'$ )) g2) e1 : ty2'
by (metis P4 P5 TyCase.hyps(4) TyCase.prem(4,5) lifts-graph2 lifts-graph-freelinks)

have IHe2:  $\Gamma 2 \vdash \{P\}$  subst i xs g2 e2 : ty2'
  using TyCase.hyps(6) TyCase.prem(1,2,3,4,5) by presburger

have T1:  $\Gamma 2 \vdash \{P\}$ 
  Case (subst i xs g2 e0)  $\Gamma 2'$  T
    (subst (i + length  $\Gamma 2'$ ) xs ((lift-graph 0  $\rightsquigarrow$  (length  $\Gamma 2'$ )) g2) e1)
  (subst i xs g2 e2)
  : ty2'
  using IHe0 IHe1B IHe2 typing.TyCase by blast
then show ?case
  by simp
next
case (TyCong  $\Gamma' P g1' ty' g2'$ )

have IHFL1: FL g2 = set xs
  by (simp add: TyCong.prem(4))

have P1:  $\Gamma' \vdash \{P\}$  (Graph g1') : ty'
  by (simp add: TyCong.hyps(1))
have P2:  $\Gamma 2 \vdash \{P\}$  (subst i xs g2 ((Graph g1'))) : ty'
  using IHFL1 TyCong.hyps(2) TyCong.prem(1,2,3,5) by presburger
have P5:  $\Gamma' \vdash \{P\}$  (Graph g2') : ty'
  using P1 TyCong.hyps(3) typing.TyCong by presburger

have IH:  $g1' \simeq g2'$ 
  by (simp add: TyCong.hyps(3))
then show ?case
  by (metis IHFL1 P2 TyCong.prem(5) subst.simp(1) subst-graph-cong typing.TyCong)
next
case (TyAlpha  $\Gamma 2 P g ty xs ys$ )
then show ?case
  by (metis subset-refl subst.simp(1) subst-graph-freelinks-equiv subst-graph-lmap-commute
    typing.TyAlpha)
next
case (TyArrow xs' ty1'  $\Gamma' P e' ty2' ys'$ )

have PremT:  $\Gamma 2 \vdash \{P\}$  (Graph g2) : ty2
  by (simp add: TyArrow.prem(1))
have PremLE:  $0 \leq \text{length } \Gamma 2$ 
  by simp

then have LIFT1: insert-at  $\Gamma 2$  0 (xs', ty1')  $\vdash \{P\}$  (lift 0 (Graph g2)) : ty2
  using PremT PremLE lift-type
  by presburger
then have LIFT: (xs', ty1')  $\# \Gamma 2 \vdash \{P\}$  (lift 0 (Graph g2)) : ty2
  by simp

have LE: Suc i  $\leq \text{length } ((xs', ty1') \# \Gamma 2)$ 
  by (simp add: TyArrow.prem(2,4) le-SucI length-insert-at)
have ENV: (xs', ty1')  $\# \Gamma' = \text{insert-at } ((xs', ty1') \# \Gamma 2)$  (Suc i) (xs, ty2)
  by (simp add: TyArrow.prem(3))

have IH: (xs', ty1')  $\# \Gamma 2 \vdash \{P\}$  (subst (Suc i) xs (lift-graph 0 g2)) e' : ty2'
  by (metis ENV LE LIFT1 TyArrow.hyps(2) TyArrow.prem(4,5) insert-at.simp(1) lift.simp(1)
    lift-graph-freelinks)
show ?case
  by (metis IH Suc-eq-plus1 TyArrow.hyps(3,4) subst.simp(1) subst-graph.simp(2) subst-p.simp(2)
    typing.TyArrow)
next
case (TyLIIntro lhs rhs P  $\Gamma$ )
then show ?case
  using subst-make-diffgraph-ignore typing.TyLIIntro by presburger
next

```

```

case (TyLITrans  $\Gamma$   $P$   $T1$   $a2$   $xs2$   $ty1$   $a1$   $xs1$   $ys1$   $T2$   $ty2$   $ys2$   $ys$ )
then show ?case
by (smt (z3) subst.simps(1) subst-graph.simps(4) typing.TyLITrans)
next
case (TyLIElim0  $ys$   $ys'$   $vs$   $T$   $\Gamma$   $P$   $a$ )
then show ?case
by (simp add: subst-graph-freelinks-equiv subst-graph-lbinds-com
      typing.TyLIElim0)
next
case (TyLIIntro0  $\Gamma$   $P$   $T$   $a$   $vs$ )
then show ?case
by (simp add: typing.TyLIIntro0)
qed

```

17.2 Inversion Lemma

```

lemma typing-inv-graph-abs-aux:
  assumes  $T: \Gamma \vdash \{P\} \ e' : ty$ 
  assumes  $ty: ty = TArrow\ ty1\ ty2\ zs$ 
    and  $E: e' = Graph\ g'$ 
    and  $C: g' \simeq Atom\ (GAbs\ xs\ ty3\ e)\ ys$ 
  shows  $(xs, ty1) \# \Gamma \vdash \{P\} \ e : ty2$ 
using assms
proof (induction arbitrary:  $xs\ ty1\ ty2\ zs\ e\ ys\ g'$  rule: typing.induct)
  case (TyAlpha  $\Gamma 2' P g ty' xs' ys')$ 

    have  $P1: FL\ g \subseteq set\ xs'$ 
      by (simp add: TyAlpha.hyps(6))
    have  $P2: FL\ (lmap\ (lsubst\ (zip\ xs'\ ys'))\ g) \subseteq set\ ys'$ 
      by (simp add: P1 TyAlpha.hyps(3,4) lsubst-graph-freelinks-subset)
    have  $P3: lmap\ (lsubst\ (zip\ xs'\ ys'))\ g \simeq Atom\ (GAbs\ xs\ ty3\ e)\ ys$ 
      using TyAlpha.prem(2,3) by blast
    then have  $P4: FL\ (lmap\ (lsubst\ (zip\ xs'\ ys'))\ g) = FL\ (Atom\ (GAbs\ xs\ ty3\ e)\ ys)$ 
      by (simp add: freelinks-equiv)
    then have  $P7: set\ ys \subseteq set\ ys'$ 
      using P2 by auto

    then have  $P8: distinct\ xs'$ 
      by (simp add: TyAlpha.hyps(4))
    then have  $P9: distinct\ ys'$ 
      by (simp add: TyAlpha.hyps(5))

    have  $P6: lmap\ (lsubst\ (zip\ ys'\ xs'))\ (lmap\ (lsubst\ (zip\ xs'\ ys'))\ g) \simeq$ 
       $lmap\ (lsubst\ (zip\ ys'\ xs'))\ (Atom\ (GAbs\ xs\ ty3\ e)\ ys)$ 
      using P3 lmap-graph-cong by blast
    have  $P7: lmap\ (lsubst\ (zip\ ys'\ xs'))\ (lmap\ (lsubst\ (zip\ xs'\ ys'))\ g) = g$ 
      by (metis P1 P8 P9 TyAlpha.hyps(3) dual-order.refl lmap-lsubst-inv-inv map-fst-zip map-snd-zip
        zip-commute)
    have  $P8: g \simeq lmap\ (lsubst\ (zip\ ys'\ xs'))\ (Atom\ (GAbs\ xs\ ty3\ e)\ ys)$ 
      using P6 P7 by auto
    then have  $P9: g \simeq Atom\ (GAbs\ xs\ ty3\ e)\ (map\ (lsubst\ (zip\ ys'\ xs'))\ ys)$ 
      by simp

    have  $P14: FLty\ ty' = FL\ g$ 
      using FL-typing-LRHS TyAlpha.hyps(1) by presburger
    then have  $P15: FLty\ ty' \subseteq set\ xs'$ 
      by (simp add: P1)

    have  $P15B: set\ xs' = ALty\ ty'$ 
      by (simp add: TyAlpha.hyps(2))

    have  $P14: FLty\ ty' = FL\ g$ 
      using FL-typing-LRHS TyAlpha.hyps(1) by presburger
    then have  $P15: FLty\ ty' = set\ xs'$ 
      by (metis (no-types, lifting) ALty.simps(2) ALty-lmap FL-lmap-commute
        FL-typing-rel-eq-lemma7 FLty.simps(2) P1 P15B P7 TyAlpha.hyps(3,4,5))

```

```

TyAlpha.premis(1) dual-order.refl lsubst-image lsubst-ty-def map-fst-zip
map-snd-zip)

have P10: lsubst-ty xs' ys' ty' = TArrow ty1 ty2 zs
  by (simp add: TyAlpha.premis(1))
have P12: lsubst-ty ys' xs' (lsbst-ty xs' ys' ty') = lsubst-ty ys' xs' (TArrow ty1 ty2 zs)
  using P10 by presburger
have P13: ty' = lsubst-ty ys' xs' (TArrow ty1 ty2 zs)
  by (metis P1 P10 P14 P15 P15B TyAlpha.hyps(3,4,5) lmap-ty-inv-inv
    lsubst-ty-def map-fst-zip map-snd-zip zip-commute)
then have P16: ty' = TArrow ty1 ty2 (map (lsbst (zip ys' xs')) zs)
  by (simp add: lsubst-ty-def)

show ?case
  using P16 P9 TyAlpha.IH by blast
next
case (TyVar i Γ xs ty ys P)
then show ?case
  apply auto
  by (simp add: abs-var-cong-inv)
next
case (TyArrow xs' ty1' Γ P e2' ty2' ys')
then show ?case
  by (metis abs-cong-inv exp.inject(1) type.inject(2))
next
case (TyApp Γ P e1 ty1 ty2 xs e2)
then show ?case
  apply auto
  done
next
case (TyProd a xs rhs P taus Ts Γ)
then show ?case
  apply auto
  done
next
case (TyCong Γ P e1 ty e2)
then show ?case
  apply simp
  by (meson gcong.trans)
next
case (TyCase Γ P e0 ty1 Γ2 e1 ty2 e2 T)
then show ?case
  by auto
next
case (TyLIIntro lhs rhs P Γ)
obtain i C zs fusions taus where
  prod-rhs: rhs = (i, (C, zs), fusions, taus)
by (metis prod.collapse)
obtain a vs where
  prod-lhs: lhs = (a, vs)
by (metis prod.collapse)
have P1: make-LI (lhs, rhs) = TLI (mset taus) a vs (fset-of-list zs |∪| FLfusion-fset fusions)
  using prod-rhs prod-lhs make-LI-def by simp
then show ?case
  by (simp add: TyLIIntro.premis(1))
next
case (TyLITrans Γ P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2 ys)
then show ?case
  apply auto
  done
next
case (TyLIElim0 ys ys' xs T Γ P a)
then show ?case
  apply auto
  done
next
case (TyLIIntro0 Γ P T a xs)

```

```

then show ?case
apply auto
done
qed

```

```

corollary typing-inv-graph-abs:
  assumes  $T: \Gamma \vdash \{P\} \text{ Graph } (GAbs \text{ xs } ty3 \text{ e}) \text{ ys} : TArrow \text{ ty1 } \text{ ty2 } \text{ zs}$ 
  shows  $(\text{xs}, \text{ty1}) \# \Gamma \vdash \{P\} \text{ e} : \text{ty2}$ 
by (meson assms gcong.refl typing-inv-graph-abs-aux)

```

17.3 Subject Reduction

Subject reduction: If a term e reduces to e' by a call-by-value step, and e is well-typed, then e' is also well-typed with the same type. This ensures that evaluation preserves typing.

```

lemma weakening-right:
  assumes  $\Gamma \vdash \{P\} \text{ e} : \tau$ 
  shows  $\Gamma @ \Gamma2 \vdash \{P\} \text{ e} : \tau$ 
using assms
proof (induct set: typing)
  case (TyVar i  $\Gamma$  xs ty ys P)
  then show ?case
  by (simp add: nth-append-left typing.TyVar)
next
  case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
  then show ?case
  by (simp add: typing.TyArrow)
next
  case (TyApp  $\Gamma$  P e1 ty1 ty2 xs e2)
  then show ?case
  by (simp add: typing.TyApp)
next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  then show ?case
  by (simp add: typing.TyProd)
next
  case (TyCong  $\Gamma$  P g1 ty g2)
  then show ?case
  using typing.TyCong by presburger
next
  case (TyAlpha  $\Gamma2$  P g ty xs ys)
  then show ?case
  by (simp add: typing.TyAlpha)
next
  case (TyCase  $\Gamma$  P e0 ty1  $\Gamma2$  e1 ty2 e2 T)
  then show ?case
  by (simp add: typing.TyCase)
next
  case (TyLIIntro lhs rhs P  $\Gamma$ )
  then show ?case
  by (simp add: typing.TyLIIntro)
next
  case (TyLITrans  $\Gamma$  P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2 ys)
  then show ?case
  by (simp add: typing.TyLITrans)
next
  case (TyLIElim0 ys ys' xs T  $\Gamma$  P a)
  then show ?case
  by (simp add: typing.TyLIElim0)
next
  case (TyLIIntro0  $\Gamma$  P T a xs)
  then show ?case
  by (simp add: typing.TyLIIntro0)
qed

```

```

lemma subst-lemma0:
  assumes A1:  $(xs, ty) \# \Gamma 1 \vdash \{P\} \ e1 : ty1$ 
  assumes A2: distinct xs
  assumes A3:  $FL\ g = set\ xs$ 
  assumes A4:  $\Box \vdash \{P\} \ (Graph\ g) : ty$ 
  shows  $\Gamma 1 \vdash \{P\} \ subst\ 0\ xs\ g\ e1 : ty1$ 
proof -
  have P1:  $\Gamma 1 \vdash \{P\} \ Graph\ g : ty$ 
    by (metis A4 append-Nil weakening-right)
  then show ?thesis
    using A1 A2 A3 A4 subst-lemma by auto
qed

```

```

lemma subst-lemma-aux:
  assumes A1:  $\Gamma = map\ fst\ XsGs$ 
  assumes A2:  $\Gamma @ \Gamma 1 \vdash \{P\} \ e1 : ty1$ 
  assumes A3:  $\forall ((xs, ty), g) \in set\ XsGs. ($ 
     $distinct\ xs \wedge FL\ g = set\ xs \wedge$ 
     $\Box \vdash \{P\} \ Graph\ g : ty)$ 
  shows  $\Gamma 1 \vdash \{P\} \ substs\ XsGs\ e1 : ty1$ 
using assms
proof (induct XsGs arbitrary:  $\Gamma \Gamma 1\ e1\ ty1$ )
  case Nil
  then show ?case
    by (simp add: substs)
next
  case (Cons a  $\Gamma$ )
  then show ?case
    apply (simp add: substs)
    using subst-lemma0 by force
qed

```

```

lemma subst-lemma:
  assumes A1:  $\Gamma @ \Gamma 1 \vdash \{P\} \ e1 : ty1$ 
  assumes A2:  $length\ \Gamma = length\ gs$ 
  assumes A3:  $XsGs = zip\ \Gamma\ gs$ 
  assumes A4:  $\forall ((xs, ty), g) \in set\ XsGs. ($ 
     $distinct\ xs \wedge FL\ g = set\ xs \wedge$ 
     $\Box \vdash \{P\} \ Graph\ g : ty)$ 
  shows  $\Gamma 1 \vdash \{P\} \ substs\ XsGs\ e1 : ty1$ 
using subst-lemma-aux apply auto
using A1 A2 A3 A4 map-fst-zip split-beta by auto

```

Subject reduction: If a term e reduces to e' by a call-by-value step, and e is well-typed, then e' is also well-typed with the same type. This ensures that evaluation preserves typing.

```

lemma subject-reduction:
  assumes  $\Gamma \vdash \{P\} \ e : \tau$ 
  and  $e \multimap \{P\} \ e'$ 
  shows  $\Gamma \vdash \{P\} \ e' : \tau$ 
using assms
proof (induct arbitrary:  $e'$  set: typing)
  case (TyVar i  $\Gamma\ xs\ ty\ ys\ P$ )
  then show ?case
    apply auto
    done
next
  case (TyArrow xs ty1  $\Gamma\ P\ e\ ty2\ ys$ )
  then show ?case
    apply auto
    done
next
  case (TyApp  $\Gamma\ P\ e1\ ty1\ ty2\ zs\ e2$ )

  note  $T\text{-fun} = TyApp.hyps(1)$ 
  have  $\Gamma \vdash \{P\} \ e1 : TArrow\ ty1\ ty2\ zs$ 

```

```

by (simp add: T-fun)

note T-arg = TyApp.hyps(3)
have  $\Gamma \vdash \{P\} \ e2 : ty1$ 
  by (simp add: T-arg)

note IH-fun = TyApp.hyps(2)
note IH-arg = TyApp.hyps(4)
note Step = TyApp.premis

show ?case
proof (cases rule: cbv-ty-AppE[OF Step])
  case (1 v xs lam ty e ys)

  have P1:  $(xs, ty1) \# \Gamma \vdash \{P\} \ e : ty2 ==>$ 
     $\Gamma \vdash \{P\} \ Graph \ (Atom \ (GAbs \ xs \ ty1 \ e) \ ys) : TArrow \ ty1 \ ty2 \ ys$ 
    using 1(3,4,7) FL-typing-LRHS T-arg TyArrow by auto

  have P2:  $\Gamma \vdash \{P\} \ Graph \ (Atom \ (GAbs \ xs \ ty \ e) \ ys) : TArrow \ ty1 \ ty2 \ zs$ 
    using 1(5) T-fun
    by (metis 1(6) TyCong)
  have P3:  $(xs, ty1) \# \Gamma \vdash \{P\} \ e : ty2$ 
    using P2 typing-inv-graph-abs by auto
  then show ?thesis
    using 1(1,3,4,7) T-arg subst-lemma by force
next
  case (2 e1')
  then show ?thesis
  apply auto
  using IH-fun T-arg typing.TyApp by blast
next
  case (3 e')
  then show ?thesis
  apply auto
  using IH-arg T-fun typing.TyApp by blast
qed
next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  then show ?case
  apply auto
  done
next
  case (TyCong  $\Gamma \ P \ g1 \ ty \ g2$ )
  then show ?case
  by auto
next
  case (TyAlpha  $\Gamma \ 2 \ P \ g \ ty \ xs \ ys$ )
  then show ?case
  apply auto
  done
next
  case (TyCase  $\Gamma \ P \ e0' \ ty1' \ \Gamma2' \ e1' \ ty2' \ e2' \ T$ )

  note Step = TyCase.premis

  have P1:  $\Gamma \vdash \{P\} \ e0' : ty1'$ 
    by (simp add: TyCase.hyps(1))
  have P3:  $\Gamma2' @ \Gamma \vdash \{P\} \ e1' : ty2'$ 
    by (simp add: TyCase.hyps(3))
  have P5:  $\Gamma \vdash \{P\} \ e2' : ty2'$ 
    by (simp add: TyCase.hyps(5))
  have P7:  $Case \ e0' \ \Gamma2' \ T \ e1' \ e2' \ -> \{P\} \ e'$ 
    by (simp add: Step)

  show ?case
proof (cases rule: cbv-ty-CaseE[OF Step])

```

```

  case (1 v XsGs)
  then show ?thesis
    using P3 substs-lemma by presburger
next
  case (2 v)
  then show ?thesis
  apply auto
    using P5 by blast
next
  case (3 e0')
  then show ?thesis
  apply auto
    using P3 P5 TyCase.hyps(2) typing.TyCase by blast
qed
next
  case (TyLIIntro lhs rhs P  $\Gamma$ )
  then show ?case
  apply auto
done
next
  case (TyLITrans  $\Gamma$  P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2 ys)
  then show ?case
  apply auto
done
next
  case (TyLIElim0 ys ys' xs T  $\Gamma$  P a)
  then show ?case
  apply auto
done
next
  case (TyLIIntro0  $\Gamma$  P T a xs)
  then show ?case
  apply auto
done
qed

```

Multi-step subject reduction: If e reduces to e' in zero or more steps, then e' has the same type as e .

```

corollary subject-reduction':
  assumes  $e \rightarrow^*\{P\} e'$ 
  and  $\Gamma \vdash \{P\} e : \tau$ 
  shows  $\Gamma \vdash \{P\} e' : \tau$ 
using assms
  by (induct set: rtranclp) (iprover intro: subject-reduction)+

```

17.4 Canonical

```

lemma LI-isnot-Arrow:
  make-LI (lhs, rhs)  $\neq$  TArrow ty1 ty2 zs
proof -
  obtain i C zs fusions taus where
    prod-rhs: rhs = (i, (C, zs), fusions, taus)
  by (metis prod.collapse)
  obtain a vs where
    prod-lhs: lhs = (a, vs)
  by (metis prod.collapse)
  show ?thesis
    by (simp add: make-LI-def prod-rhs prod-lhs)
qed

```

```

lemma canonical-forms-arrow:
  assumes  $T: \Gamma \vdash \{P\} v : ty$ 
  assumes  $V: is-val v$ 
  assumes  $A3: \Gamma = []$ 
  assumes  $A4: ty = TArrow ty1 ty2 zs$ 
  shows  $\exists xs e' ys g.$ 
    ( $v = Graph\ g \wedge$ 
     $g \simeq Atom\ (GAbs\ xs\ ty1\ e')\ ys \wedge$ 

```

```

    distinct xs ∧
    set xs = FLty ty1)
using assms
proof (induct arbitrary: zs set: typing)
  case (TyVar i Γ xs ty ys P)
  then show ?case
  apply auto
  done
next
  case (TyArrow xs ty1 Γ P e ty2 ys)
  then show ?case
  by auto
next
  case (TyApp Γ P e1 ty1 ty2 xs e2)
  then show ?case
  apply auto
  done
next
  case (TyProd a xs rhs P taus Ts Γ)
  then show ?case
  apply auto
  done
next
  case (TyCong Γ P g1 ty g2)
  show ?case
  apply auto
  by (metis TyCong.hyps(2,3) TyCong.prem(1,2,3) cong-is-val exp.inject(1) gcong.sym gcong.trans)
next
  case (TyAlpha Γ2 P g' ty xs' ys')
  have P1: is-val (Graph g')
    using TyAlpha.prem(1) is-graph-val-ignore-lmap is-val.simp(1) by blast
  then show ?case
  apply auto
  by (metis (no-types, lifting) ext A4 P1 TyAlpha.hyps(2,3,4,5,6)
    TyAlpha.prem(2,3) exp.inject(1) lmap.simp(2) lmap-graph-cong
    lmap-ty.simp(2) lmap-ty-inv-inv lsubst-ty-def map-eq-map-tailrec
    map-fst-zip map-snd-zip subset-code(1) zip-commute)
next
  case (TyCase Γ P e0 ty1 Γ2 e1 ty2 e2 T)
  then show ?case
  apply auto
  done
next
  case (TyLIIntro lhs rhs P Γ)
  then show ?case
  by (simp add: LI-isnot-Arrow)
next
  case (TyLITrans Γ P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2 ys)
  then show ?case
  apply auto
  done
next
  case (TyLIElim0 ys ys' xs T Γ P a)
  then show ?case
  apply auto
  done
next
  case (TyLIIntro0 Γ P T a xs)
  then show ?case
  apply auto
  done
qed

```

17.5 Progress

lemma *Nus-is-val*:
is-graph-val g = is-graph-val (Nus1 i g)

apply (*induct i arbitrary: g*)
by *auto*

lemma *Mols1-is-val*:
 $(is-graph-val\ g \wedge (\forall g \in set\ gs.\ is-graph-val\ g)) = is-graph-val\ (Mols1\ g\ gs)$
proof (*induct gs arbitrary: g*)
case *Nil*
then show *?case*
apply *auto*
apply (*simp-all add: Mols1*)
done
next
case (*Cons a gs*)
then show *?case*
apply (*simp-all add: Mols1*)
by (*metis is-graph-val.simps(4)*)
qed

lemma *fusions-is-val*:
 $\forall fus \in set\ (fusions-of\ fs).\ is-graph-val\ fus$
apply (*simp add: fusions-of-def*)
by *auto*

lemma *app-prodrule-is-val*:
assumes *A1*: $\forall i < length\ Ts.\ is-val\ (Graph\ (Ts\ !\ i))$
shows *is-graph-val (app-prodrule Ts rhs)*
proof –
have *P0*: $\forall T \in set\ Ts.\ is-graph-val\ T$
by (*metis assms in-set-conv-nth is-val.simps(1)*)

obtain *i C zs fusions taus* **where**
 $prod-rhs: rhs = (i, (C, zs), fusions, taus)$
by (*metis prod.collapse*)

have *is-graph-val (Atom (GConstr C) zs)*
by *simp*

then have *is-graph-val (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))*
using *Mols1-is-val fusions-is-val* **by** *blast*

then have *is-graph-val (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts)*
using *Mols1-is-val P0* **by** *blast*

then have *is-graph-val (Nus1 i (Mols1 (Mols1 (Atom (GConstr C) zs) (fusions-of fusions)) Ts))*
using *Nus-is-val* **by** *blast*

then show *?thesis*
apply (*simp add: app-prodrule-def*)
using *prod-rhs* **by** *auto*
qed

lemma *diffgraph-is-val*:
 $is-graph-val\ (make-diffgraph\ rhs)$
proof –
obtain *i C zs fusions taus* **where**
 $prod-rhs: rhs = (i, (C, zs), fusions, taus)$
by (*metis prod.collapse*)

have *P1*: $make-diffgraph\ rhs =$
 $Mols1\ (Atom\ (GConstr\ C)\ zs)\ (fusions-of\ fusions)$
using *prod-rhs make-diffgraph-def* **apply** *simp*
done

```

have P2: is-graph-val (Mols1 (Atom (GConstr C) zs) (fusions-of fusions))
  using Mols1-is-val fusions-is-val is-atom-val.simps(1)
  is-graph-val.simps(2) by blast
have P2: is-graph-val (make-diffgraph rhs)
  by (simp add: P1 P2)

then show ?thesis .
qed

```

```

lemma is-val-lbinds-commute:
  is-val (Graph T) = is-val (Graph (lbinds xs T))
apply (simp add: lbinds-def)
proof (induct xs arbitrary: T)
  case Nil
  then show ?case
  apply auto
  done
next
  case (Cons a xs)
  then show ?case
  apply (simp add: lbind-def)
  by (simp add: is-graph-val-ignore-lmap)
qed

```

Progress: closed, well-typed terms are values or can step.

```

lemma progress:
  assumes A1:  $\Gamma \vdash \{P\} e : ty$ 
  assumes A2:  $\Gamma = []$ 
  shows is-val e  $\vee (\exists e'. e \rightarrow \{P\} e')$ 
  using assms
proof (induction arbitrary: ty e' set: typing)
  case (TyVar i  $\Gamma$  xs ty ys P)
  then show ?case
  apply auto
  done
next
  case (TyArrow xs ty1  $\Gamma$  P e ty2 ys)
  then show ?case
  apply auto
  done
next
  case (TyApp  $\Gamma$  P e1 ty1 ty2 xs e2)
  have P1: is-val e1  $\vee (\exists a. e1 \rightarrow \{P\} a)$ 
    by (simp add: TyApp.IH(1) TyApp.prem)
  have P2: is-val e2  $\vee (\exists a. e2 \rightarrow \{P\} a)$ 
    by (simp add: TyApp.IH(2) TyApp.prem)

  have Step:
    (is-val e1  $\wedge$  is-val e2)  $\vee$ 
    ( $\exists a. e1 \rightarrow \{P\} a$ )  $\vee$  ( $\exists a. e2 \rightarrow \{P\} a$ )
  using P1 P2 by blast

  from Step show ?case
proof (elim disjE)
  assume VV: is-val e1  $\wedge$  is-val e2
  then have V1: is-val e1 and V2: is-val e2 by auto

  from V1 obtain g1 where E1: e1 = Graph g1
  by (cases e1) auto
  from V2 obtain g2 where E2: e2 = Graph g2
  by (cases e2) auto

  have P3C:
     $\exists xs e' ys. g1 \simeq \text{Atom } (GAbs \ xs \ ty1 \ e') \ ys \wedge \text{set } xs = FLty \ ty1 \wedge \text{distinct } xs$ 
  by (metis E1 TyApp.hyps(1) TyApp.prem V1 exp.inject(1) canonical-forms-arrow)

```

```

    then show ?case
    by (metis E1 E2 FL-typing-LRHS TyApp.hyps(2) V2 beta-v-ty is-val.simps(1))
next
  assume S: ( $\exists a. e1 \multimap_{\{P\}} a$ )
  then show ?case
  by force
next
  assume S: ( $\exists a. e2 \multimap_{\{P\}} a$ )
  then show ?case
  using P1 by blast
qed
next
  case (TyProd a xs rhs P taus Ts  $\Gamma$ )
  have P1:  $\forall i < \text{length } Ts. \text{is-val } (\text{Graph } (Ts ! i))$ 
  by (meson TyProd.IH TyProd.premis cbv-ty-GraphE)
  then show ?case
  apply auto
  by (simp add: app-prodrule-is-val)
next
  case (TyCong  $\Gamma P e1 ty e2$ )
  then show ?case
  by (metis cbv-ty-GraphE cong-is-val)
next
  case (TyAlpha  $\Gamma 2 P g ty xs ys$ )
  then show ?case
  by (metis cbv-ty-GraphE is-graph-val-ignore-lmap is-val.simps(1))
next
  case (TyCase  $\Gamma P e0 ty1 \Gamma 2 e1 ty2 e2 T$ )
  then show ?case
  apply auto
  apply (metis cbv-ty.intros(3) exp.exhaust is-val.simps(1,2,3))
  by (metis cbv-ty.intros(3) exp.exhaust is-val.simps(1,2,3))
next
  case (TyLIIntro lhs rhs P  $\Gamma$ )
  then show ?case
  apply auto
  by (simp add: diffgraph-is-val)
next
  case (TyLITrans  $\Gamma P T1 a2 xs2 tys1 a1 xs1 ys1 T2 tys2 ys2 ys$ )
  then show ?case
  apply auto
  done
next
  case (TyLIElim0 ys ys' xs T  $\Gamma P a$ )
  then show ?case
  by (meson cbv-ty-GraphE is-val-lbinds-commute)
next
  case (TyLIIntro0  $\Gamma P T a xs$ )
  then show ?case
  apply auto
  done
qed

```

Progress: a simpler corollary.

```

corollary progress':
  assumes  $\Box \vdash_{\{P\}} e : \tau$ 
  shows  $\text{is-val } e \vee (\exists e'. e \multimap_{\{P\}} e')$ 
using assms progress by auto

```

17.6 Soundness

Type soundness theorem: If a closed term is well-typed and reduces (in zero or more steps) to some term e' , then e' is either a value or can take a further step. This combines subject reduction and progress to establish the standard type soundness property of the language.

```

theorem type-soundness:
  assumes WT:  $\Box \vdash_{\{P\}} e : \tau$ 

```

```

    and STEPS:  $e \rightarrow_{\{P\}}^* e'$ 
  shows  $\text{is-val } e' \vee (\exists u. e' \rightarrow_{\{P\}} u)$ 
proof -
  have  $\Box \vdash_{\{P\}} e' : \tau$  using STEPS WT subject-reduction' by blast
  thus ?thesis using progress' by blast
qed

end

```